

TIS Project Reference Booklet

Documentation version: 3.1

Generated: 2026-07-21 23:04 Middle East Daylight Time

Branch: dev

Git commit SHA: ae85904

Source of truth: Markdown files under docs/ are authoritative. This PDF is a generated snapshot and must not be edited manually.

Source Documents Included

- docs/README.md
- docs/AI_PROJECT_CONTEXT.md
- docs/DOCUMENTATION_UPDATE_POLICY.md
- docs/TIS_MASTER_CONTEXT.md
- docs/PROJECT_STATE.md
- docs/CHANGE_HISTORY.md
- docs/adr/README.md
- docs/engineering/README.md
- docs/engineering/TIS_MODULE_MAP.md
- docs/engineering/REPOSITORY_ARCHITECTURE.md
- docs/engineering/USER_AND_SYSTEM_FLOWS.md
- docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
- docs/engineering/DEVELOPMENT_STANDARDS.md
- docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md
- docs/engineering/PRODUCT_ROADMAP.md
- docs/engineering/REJECTED_DECISIONS.md
- docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md
- docs/engineering/AI_OPTIMIZATION_GUIDE.md
- docs/engineering/PROJECT_GOVERNANCE.md
- docs/engineering/KNOWLEDGE_LIFECYCLE.md
- docs/engineering/DOCUMENTATION_AUTOMATION.md
- docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md
- docs/engineering/SELF_EVOLVING_WORKFLOW.md
- docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md

- docs/engineering/AI_CODING_WORKFLOW.md
- docs/engineering/FUTURE_AUTOMATION_ROADMAP.md
- docs/adr/0001-separate-nextjs-landing-website.md
- docs/adr/0002-separate-saas-identity-and-operational-users.md
- docs/adr/0003-paddle-payment-architecture.md
- docs/adr/0004-webhook-only-payment-confirmation.md
- docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md
- docs/adr/0006-documentation-as-source-knowledge-management-system.md
- docs/adr/0007-landing-page-visual-system-strategy.md
- docs/history/academic-calendar/README.md
- docs/history/engineering-handbook/2026-06-26-kms-v3-engineering-handbook.md
- docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3b-engineering-layers.md
- docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3c-governance-ai-traceability.md
- docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3d-lifecycle-foundation.md
- docs/history/engineering-handbook/2026-06-27-production-memory-stability-guardrails.md
- docs/history/engineering-handbook/2026-07-21-kms-automatic-enforcement.md
- docs/history/engineering-handbook/README.md
- docs/history/landing-page/README.md
- docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md
- docs/history/platform-knowledge/README.md
- docs/history/provisioning/2026-06-26-kms-foundation.md
- docs/history/provisioning/README.md
- docs/history/README.md
- docs/history/saas-onboarding/README.md
- docs/history/subscriptions/README.md
- docs/history/workforce-planning/README.md
- docs/marketing/landing_page_source_of_truth.md
- docs/marketing/tis_landing_page_master_content.md
- docs/location-data-roadmap.md

docs/README.md

TIS Documentation

This folder is the source of truth for the TIS Knowledge Management System (KMS).

Markdown files are authoritative. The PDF booklet is a generated snapshot and must never be edited manually.

Repository enforcement begins with root `AGENTS.md` and `.kms-impact.yml`. `scripts/check_kms_impact.py` validates the declaration, changed paths, declared Markdown updates, and generated artifacts. It never rewrites documentation.

First Read For AI Coding Conversations

For any new Codex or ChatGPT coding conversation, load this file first:

```
1 docs/AI_PROJECT_CONTEXT.md
```

Then load:

```
1 docs/TIS_MASTER_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/DOCUMENTATION_UPDATE_POLICY.md
1 relevant ADRs under docs/adr/
1 relevant module history under docs/history/
1 engineering handbook docs under docs/engineering/
```

If the task touches the public website, also read:

```
1 docs/marketing/landing_page_source_of_truth.md
1 docs/marketing/tis_landing_page_master_content.md
1 docs/adr/0001-separate-nextjs-landing-website.md
1 docs/adr/0007-landing-page-visual-system-strategy.md
```

Core Documents

```
1 docs/AI_PROJECT_CONTEXT.md: compact onboarding file for future Codex and ChatGPT conversations.
1 docs/TIS_MASTER_CONTEXT.md: long-term product, architecture, SaaS, workflow, roadmap, and critical-rules source of truth.
1 docs/PROJECT_STATE.md: living status file for branch strategy, priority, milestones, known issues, and next work.
1 docs/DOCUMENTATION_UPDATE_POLICY.md: non-negotiable KMS and Knowledge Impact Assessment policy.
1 docs/CHANGE_HISTORY.md: chronological summary of meaningful changes.
```

Engineering Handbook

```
1 docs/engineering/README.md: engineering onboarding order and handbook index.
1 docs/engineering/TIS_MODULE_MAP.md: complete module map with purpose, files, maturity, docs, risks, and guardrails.
```

- 1 docs/engineering/REPOSITORY_ARCHITECTURE.md: repository structure and ownership boundaries.
- 1 docs/engineering/USER_AND_SYSTEM_FLOWS.md: public customer, SaaS identity, payment, provisioning, operational login, platform owner, KMS, and developer onboarding flows.
- 1 docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md: conceptual data model and tenant/identity isolation rules.
- 1 docs/engineering/DEVELOPMENT_STANDARDS.md: non-negotiable engineering rules.
- 1 docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md: UI/UX principles by product surface.
- 1 docs/engineering/PRODUCT_ROADMAP.md: completed, current, next, and future roadmap.
- 1 docs/engineering/REJECTED_DECISIONS.md: major rejected alternatives and long-term consequences.
- 1 docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md: screenshot, diagram, naming, storage, and visual update standards.
- 1 docs/engineering/AI_OPTIMIZATION_GUIDE.md: definitive onboarding guide for future AI assistants.
- 1 docs/engineering/PROJECT_GOVERNANCE.md: governance, approvals, quality gates, and decision traceability.
- 1 docs/engineering/KNOWLEDGE_LIFECYCLE.md: lifecycle from planning through production and maintenance.
- 1 docs/engineering/DOCUMENTATION_AUTOMATION.md: current/future KMS automation model.
- 1 docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md: formal KIA engineering standard.
- 1 docs/engineering/SELF_EVOLVING_WORKFLOW.md: official workflow that keeps software and KMS synchronized.
- 1 docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md: propagation map between source docs, manifest, PDF, and Knowledge Center.
- 1 docs/engineering/AI_CODING_WORKFLOW.md: disciplined workflow for future AI coding assistants.
- 1 docs/engineering/FUTURE_AUTOMATION_ROADMAP.md: future automation opportunities.

Decision And History Documents

- 1 docs/adr/: Architecture Decision Records for major accepted decisions.
- 1 docs/history/: module-based history preserving deeper before/after context.

Current module history areas:

- 1 docs/history/subscriptions/
- 1 docs/history/landing-page/
- 1 docs/history/academic-calendar/
- 1 docs/history/workforce-planning/
- 1 docs/history/saas-onboarding/
- 1 docs/history/provisioning/
- 1 docs/history/platform-knowledge/
- 1 docs/history/engineering-handbook/

Supporting Documents

- 1 docs/location-data-roadmap.md: location data roadmap and related implementation notes.

- 1 docs/marketing/landing_page_source_of_truth.md: boundary between the public Next.js landing website and the FastAPI application portal.
- 1 docs/marketing/tis_landing_page_master_content.md: approved marketing foundation and landing page content direction.

Generated Snapshot

Generated files:

- 1 static/docs/TIS_Project_Reference_Booklet.pdf
- 1 static/docs/docs_manifest.json

The PDF is generated from approved Markdown docs. The manifest records the generated timestamp, documentation version, branch, commit SHA, included sources, and source hashes.

KMS Synchronization

Validate KIA, regenerate the PDF and manifest, and verify freshness with:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Complete read-only validation:

```
.\.venv\Scripts\python.exe scripts\kms.py check
```

The command reuses the dependency-light generator and strict impact checker. It must not require LaTeX, Playwright, Chromium, network calls, or system fonts.

Knowledge Impact Assessment

Every future implementation report must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until KIA is assessed. If included source docs change, regenerate the PDF.

Update `.kms-impact.yml` during every task. CI validates it on pull requests and dev; deployment from master depends on the same KMS gate.

Phase Boundary

Phase 2A and Phase 2B establish KMS governance, ADRs, module history, AI context, and PDF/manifest generation.

KMS v3.0 Phase 3D completes the KMS v1.0 lifecycle foundation with self-evolving documentation workflow, KIA standard, dependency map, AI coding workflow, and future automation roadmap. The Platform Owner Knowledge Center is implemented as a read-only owner utility. Do not add a regenerate button or app-side Markdown rewriting unless explicitly approved.

docs/AI_PROJECT_CONTEXT.md

TIS AI Project Context

This is the first file future Codex or ChatGPT coding conversations should load. It is a compact project onboarding reference; detailed source of truth remains in the other Markdown docs.

What TIS Is

TIS is Teacher Information System, a developing SaaS academic operations platform for schools and school groups. It connects teacher information, staffing and workload planning, academic calendars, observations, branch context, SaaS onboarding, billing, provisioning, and future AI-assisted academic decision support.

Public URLs:

- 1 Public website: <https://tisplatform.com>
- 1 Application portal: <https://app.tisplatform.com>
- 1 Render must set `TIS_PUBLIC_BASE_URL=https://app.tisplatform.com` so transactional emails and background Workspace Activation emails generate production login and static asset URLs instead of local-development fallbacks.

Important routes:

- 1 Operational login: `/login`
- 1 SaaS signup: `/saas/signup`
- 1 SaaS login: `/saas/login`
- 1 SaaS account: `/saas/account`
- 1 Platform console: `/platform`

Current Architecture

The operational app is a FastAPI application at the repository root.

Key files and folders:

- 1 `main.py`: primary FastAPI app and many route handlers.
- 1 `auth.py`: authentication, roles, platform identity helpers, permissions, sessions.
- 1 `authorization.py`: protected route rules and access-denied handling.
- 1 `permission_registry.py`: permission keys, groups, defaults, and developer-assignable permissions.
- 1 `location_service.py`: global location picker lookup/validation. Must stay memory-conscious and scoped; do not restore full unbounded dataset parsing for normal picker requests.
- 1 `ui_shell.py`: shared app shell/navigation/page metadata.
- 1 `models.py`, `database.py`, `db_migrations.py`: data model, DB setup, local schema repair/migration logic.
- 1 `routers/`: modular operational routes.
- 1 `saas/`: SaaS account, onboarding, payment, billing, and provisioning services/routes.
- 1 `saas/entitlement_service.py`: provider-confirmed commercial entitlement and paid branch-capacity resolution.
- 1 `saas/subscription_portal_service.py`: customer subscription portal view model.

- 1 saas/subscription_change_service.py, saas/subscription_plan_change_service.py, and saas/subscription_cancellation_service.py: provider-authoritative quantity, plan, and cancellation workflows.
- 1 saas/subscription_lifecycle_service.py: centralized lifecycle state and allowed-action policy.
- 1 saas/billing_history_service.py: provider-sourced billing history and short-lived invoice download resolution.
- 1 saas/payment_lifecycle_reconciliation_service.py: guarded reconciliation for attributable finalized payment evidence.
- 1 scripts/sync_paddle_price_ids.py: environment-specific Paddle initial checkout price mapping sync into subscription_plan_prices.provider_price_id.
- 1 templates/: Jinja templates.
- 1 static/: static assets and generated documentation output.
- 1 tests/: pytest coverage.

The public marketing website is separate:

- 1 tis-landing-website/
- 1 Next.js / Node runtime
- 1 Source of truth for public landing implementation

Legacy FastAPI landing files are not the source of truth:

- 1 templates/landing.html
- 1 static/landing/landing.css

Engineering Handbook

For deeper onboarding, read:

- 1 docs/engineering/README.md
- 1 docs/engineering/TIS_MODULE_MAP.md
- 1 docs/engineering/REPOSITORY_ARCHITECTURE.md
- 1 docs/engineering/USER_AND_SYSTEM_FLOWS.md
- 1 docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
- 1 docs/engineering/DEVELOPMENT_STANDARDS.md
- 1 docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md
- 1 docs/engineering/PRODUCT_ROADMAP.md
- 1 docs/engineering/REJECTED_DECISIONS.md
- 1 docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md
- 1 docs/engineering/AI_OPTIMIZATION_GUIDE.md
- 1 docs/engineering/PROJECT_GOVERNANCE.md
- 1 docs/engineering/KNOWLEDGE_LIFECYCLE.md
- 1 docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md

- 1 docs/engineering/SELF_EVOLVING_WORKFLOW.md
- 1 docs/engineering/AI_CODING_WORKFLOW.md

These files explain module ownership, repository boundaries, end-to-end flows, and what must not be changed casually.

Paddle Initial Checkout Configuration

Initial subscription checkout uses Paddle price IDs stored in `subscription_plan_prices.provider_price_id`. Use `scripts/sync_paddle_price_ids.py` with a structured sandbox or production mapping JSON to configure these values. Paddle credentials and endpoints remain environment variables. Real mapping files are ignored by Git; keep sandbox and live provider price IDs separate. If a mapping is missing, checkout fails closed before Paddle is called and customers receive a support-oriented Secure Payment message while internal diagnostics keep plan code, billing interval, and currency context.

Paddle transaction checkout uses a dedicated public SaaS payment launcher at `/saas/payment`. Configure `PADDLE_CHECKOUT_BASE_URL` to that page so Paddle returns transaction checkout URLs with `_ptxn` appended to the launcher instead of the operational app root. The launcher uses Paddle.js with `PADDLE_CLIENT_TOKEN` and `PADDLE_ENVIRONMENT`; never expose `PADDLE_API_KEY` in HTML or JavaScript.

Domains And Routing

The public website lives at `https://tisplatform.com`. The app portal lives at `https://app.tisplatform.com`.

SaaS account routes are under `/saas`. Platform-owner SaaS administration routes are under `/saas-admin`. Operational tenant workflows use routes such as `/dashboard`, `/teachers`, `/subjects`, `/planning`, `/timetable`, `/academic-calendar`, and `/observations`.

Completed M1-M5 Milestones

M1: SaaS identity foundation and separation between platform, tenant, and SaaS account identities.

M2: SaaS onboarding flow for organization, contacts, branches, academic setup, and review.

M3: Billing and plan foundation with plan catalog, checkout, billing status, and payment service boundaries.

M4: Tenant provisioning foundation with pending organizations, provisioning jobs, retry/run actions, and platform owner oversight.

M5: Platform access and owner controls, including platform owner/developer identities, permissions, and platform console behavior.

Completed M7 Subscription Management

M7 includes a normalized entitlement foundation, a customer Subscription Management portal, paid branch-quantity management, upgrades and scheduled downgrades, Paddle-authoritative previews and proration, scheduled cancellation and reversal, a centralized lifecycle/action policy, provider-sourced billing history, protected invoice downloads, and webhook/reconciliation safeguards.

Commercial state fails closed when ownership, provider evidence, or local relationships are ambiguous. TIS does not independently calculate replacement monetary values. Immediate changes require provider-confirmed outcomes; scheduled changes remain pending until provider/webhook evidence reaches the effective boundary.

Current SaaS Account Verification State

Phase 1 TIS Account email verification recovery is accepted. Valid verification links now mark the SaaS account email verified/active and redirect to the TIS Account login page with a professional success notice so the customer can continue school workspace setup.

Expired or invalid verification links no longer dead-end. They show a recovery page with a resend verification form. Resend verification handles unverified accounts, already verified accounts, and unknown email addresses with safe customer-facing messaging that does not reveal account existence. Password-based accounts that remain unverified are blocked from starting or continuing school workspace setup.

This Phase 1 verification recovery work did not change payment, billing, provisioning, database schema, migrations, operational modules, or the Next.js landing website. Google/Microsoft login remains future work and was not implemented.

Current SaaS Customer-Facing Language State

Phase 2 TIS Account wording cleanup is accepted for customer-facing account and school workspace setup pages. Customer-visible account/setup pages now avoid presenting "SaaS" and technical identifiers as product language, while internal /saas routes, modules, models, and stored statuses remain unchanged.

The customer journey should use professional labels such as "TIS Account", "Account Dashboard", "School Workspace Setup", "Organization Profile", "Branch Setup", "Academic Setup", "Subscription Setup", "Secure Payment", and "Workspace Activation". Customer templates should label internal billing, payment, onboarding, and activation statuses through customer-safe display labels instead of raw database statuses such as `tenant_active`, provisioning states, checkout session states, provider identifiers, plan IDs, school group IDs, attempt UUIDs, or provider subscription/transaction IDs.

The shared TIS Account customer shell uses an official TIS logo image so customer account/setup forms inherit official branding. The light account shell uses the full-color horizontal logo variant, and transactional account emails use an existing official dark-blue wordmark asset. This wording/logo pass did not change payment, billing, provisioning behavior, database schema, migrations, operational modules, or the Next.js landing website. Google/Microsoft login remains future work and was not implemented.

Current TIS Account Guided Setup Framework State

Phase 3A shared guided setup framework is accepted for the TIS Account dashboard only. The customer account shell now supports a shared setup console with the official TIS logo, an 8-step journey stepper, a current-step/status area, one primary next action, and concise guidance. The account page uses this framework to answer "What should I do next?" without dashboard statistics.

The 8 customer-facing journey steps are TIS Account, Email Verification, School Workspace Setup, Review & Confirmation, Subscription Selection, Secure Payment, Workspace Activation, and Enter TIS Platform. The display state is calculated from existing account, onboarding, billing, payment, and activation data without changing stored statuses.

This Phase 3A work did not redesign onboarding forms, subscription/payment pages, or billing/status pages. It did not change payment, billing, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login, internal /saas route names, or admin views.

Phase 3B redesigned the five School Workspace Setup onboarding pages on top of the Phase 3A shared shell: Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup. The pages now use consistent guided-wizard sections, a single shared-shell primary CTA, secondary Back/Save Draft actions, concise guidance, and lower visual clutter while preserving all existing form actions, field names, validation behavior, draft behavior, and onboarding state transitions.

Phase 3B did not redesign subscription/payment/billing/status pages and did not change payment, billing, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login,

internal /saas route names, or admin views.

Phase 3C redesigned the customer-facing Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages on top of the Phase 3A shared shell and Phase 3B guided style. These pages now use one shared-shell primary CTA, customer-safe payment/activation labels, clearer browser-return messaging, and explicit guidance that TIS Platform access becomes available after Workspace Activation.

Phase 3C did not change payment behavior, billing behavior, provisioning behavior, webhook logic, checkout start/launch behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login, internal /saas route names, stored statuses, or admin views.

Current Priority

Current priority is automatic KMS synchronization enforcement and reliable post-M7 engineering context:

- 1 Markdown is source of truth.
- 1 PDF is generated snapshot.
- 1 Change history preserves chronological change context.
- 1 ADRs preserve major decisions.
- 1 Module history preserves deeper area-specific evolution.
- 1 Platform Owner Knowledge Center is implemented as a read-only owner utility.
- 1 The Knowledge Center uses protected routes for PDF view/download and does not link directly to static PDF paths.
- 1 KMS v3.0 Phase 3A adds a true engineering handbook with module map, repository architecture, workflows, and developer onboarding.
- 1 KMS v3.0 Phase 3B adds database architecture, development standards, UI/UX philosophy, product roadmap, and stronger human/AI developer guidance.
- 1 KMS v3.0 Phase 3C adds rejected decisions, visual documentation framework, AI optimization guidance, project governance, and decision traceability.
- 1 KMS v3.0 Phase 3D completes KMS v1.0 lifecycle standards, dependency mapping, AI coding workflow, and future automation roadmap.
- 1 Root AGENTS.md makes KMS onboarding mandatory for Codex tasks.
- 1 .kms-impact.yml is the machine-readable task declaration.
- 1 scripts/kms.py sync is the single local synchronization command; it validates KIA before writing, regenerates the PDF/manifest, then verifies freshness.
- 1 scripts/kms.py check is the single read-only local and CI validation command.
- 1 scripts/check_kms_impact.py validates major-change classification, declared Markdown updates, and generated-artifact freshness.
- 1 GitHub Actions block pull-request integration and production deployment when KMS validation fails.

Critical Rules

- 1 Do not touch SaaS flows unless explicitly approved.
- 1 Do not touch operational logic unless required by the approved task.
- 1 Do not touch database migrations or tis.db unless explicitly approved.

- 1 Do not weaken tenant isolation.
- 1 Do not bypass permissions or platform owner checks.
- 1 Do not merge platform user, SaaS account, and tenant user concepts.
- 1 Do not change landing page implementation unless explicitly approved.
- 1 Do not add a KMS regenerate button until explicitly approved.
- 1 Do not push or commit unless explicitly requested.
- 1 Treat production memory as a hard budget. Do not add unbounded full-dataset caches, duplicate production template renders, startup-heavy work, or warning-level debug spam on normal requests.

KMS Policy

Every implementation must include a Knowledge Impact Assessment:

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

If included docs change, synchronize:

```
..\venv\Scripts\python.exe scripts\kms.py sync
```

Development Workflow

- 1 Read this file first.
- 2 Read root AGENTS.md, docs/TIS_MASTER_CONTEXT.md, and docs/PROJECT_STATE.md.
- 3 Read docs/engineering/README.md.
- 4 Read docs/engineering/DEVELOPMENT_STANDARDS.md.
- 5 Read docs/engineering/AI_OPTIMIZATION_GUIDE.md.
- 6 Read docs/engineering/AI_CODING_WORKFLOW.md.
- 7 Read relevant engineering docs, ADRs, module history, and supporting docs.
- 8 Inspect code before editing.
- 9 Keep changes scoped.
- 10 Update .kms-impact.yml and affected KMS docs when meaningful behavior, architecture, product state, module map, repository ownership, data model, design philosophy, roadmap, governance, decision traceability, automation, lifecycle, or workflow changes.
- 11 Run scripts/kms.py sync if included source docs changed.
- 12 Run scripts/kms.py check for final read-only validation.
- 13 Run implementation validation.
- 14 Report KIA in final response.

Landing Page Situation

The public landing implementation is in `tis-landing-website/`. Marketing docs live under `docs/marketing/`. Do not modify legacy FastAPI landing files unless explicitly approved.

Next Planned Work

Review the automatic KMS enforcement baseline, keep M7 subscription documentation synchronized as follow-up fixes evolve, and later consider an explicit owner-only regenerate action. Any regenerate action may rebuild artifacts from reviewed Markdown only and must not rewrite source prose.

docs/DOCUMENTATION_UPDATE_POLICY.md

TIS Documentation Update Policy

This policy defines the non-negotiable rules for keeping the TIS Knowledge Management System reliable.

Source Of Truth

Markdown files under docs/ are the source of truth.

The PDF booklet at static/docs/TIS_Project_Reference_Booklet.pdf is a generated snapshot. It must never be edited manually. It must be regenerated whenever included Markdown source files change.

Required Knowledge Impact Assessment

Every approved implementation must end with a Knowledge Impact Assessment (KIA).

Required final report template:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until the KIA is included, even when no documentation changes are needed.

Every task must also update `.kms-impact.yml`. The declaration is the machine-readable form of the KIA and records `knowledge_impact`, a summary, affected areas, KMS files updated, a no-impact reason, and the explicit major-change override. `scripts/check_kms_impact.py` validates it against the actual Git diff.

`knowledge_impact: yes` requires changed authoritative Markdown. `knowledge_impact: no` requires a specific reason. When a path is conservatively classified as major but the change is genuinely non-behavioral, `major_change_override: yes` is allowed only with that written reason.

When Documentation Must Be Updated

Update documentation when a task changes:

- 1 product vision or positioning,
- 1 architecture or module boundaries,
- 1 SaaS signup, login, onboarding, billing, payment, or provisioning flows,
- 1 platform owner or permission behavior,
- 1 operational workflows such as calendar, planning, timetable, teachers, subjects, or observations,
- 1 deployment assumptions,
- 1 branch/release/project status,
- 1 landing page source-of-truth or visual strategy,
- 1 roadmap or known issues,
- 1 documentation system behavior.

Which Docs To Update

Use this guide:

- 1 docs/AI_PROJECT_CONTEXT.md: compact onboarding context for future AI coding conversations; update when the high-level project situation changes.
- 1 docs/TIS_MASTER_CONTEXT.md: durable product, architecture, workflow, roadmap, and critical rules.
- 1 docs/PROJECT_STATE.md: current branch strategy, priorities, milestone status, known issues, and next work.
- 1 docs/CHANGE_HISTORY.md: chronological summary of meaningful changes.
- 1 docs/adr/: major architectural or product decisions.
- 1 docs/history/: deeper module-specific before/after history.
- 1 Supporting docs under docs/marketing/ or other folders when those areas change.

ADR Rule

Create or update an ADR when a change affects a major long-term decision, including identity boundaries, payment architecture, provisioning strategy, deployment architecture, public website architecture, documentation governance, or visual system strategy.

Do not create ADRs for routine bug fixes unless they introduce or reverse a significant decision.

Module History Rule

Update docs/history/<module>/ when a meaningful module area changes and the previous documented state should be preserved.

docs/CHANGE_HISTORY.md remains the chronological summary. Module history stores deeper context.

KMS Synchronization

Validate KIA, regenerate the PDF and manifest, and verify freshness with:

```
..\env\Scripts\python.exe scripts\kms.py sync
```

The generator must remain dependency-light:

- 1 use existing report lab,
- 1 no LaTeX,
- 1 no Playwright or Chromium,
- 1 no external network calls,
- 1 no system font dependency.

Complete read-only validation:

```
..\env\Scripts\python.exe scripts\kms.py check
```

The check command must never write documentation. GitHub Actions run it for pull requests and dev; the production deployment workflow requires it before triggering deployment from master. The sync command writes generated artifacts only and never rewrites authoritative Markdown.

Prohibited KMS Content

KMS documentation describes system design only. Never include customer or organization information, personal data, subscription rows, invoices, transactions, real production identifiers, real webhook payloads, credentials, secrets, environment values, database row contents, or test-customer personal details.

Reporting Rule

Every final implementation report must mention:

- 1 whether knowledge was impacted,
- 1 which docs changed,
- 1 whether change history changed,
- 1 whether an ADR was needed,
- 1 whether module history changed,
- 1 whether the PDF was regenerated,
- 1 whether `AI_PROJECT_CONTEXT.md` changed,
- 1 validation results.

App Behavior Rule

The application may later detect documentation freshness and expose docs through owner-only protected routes. It must not silently rewrite Markdown source docs. Source docs are edited through reviewed development work.

docs/TIS_MASTER_CONTEXT.md

TIS Master Context

Documentation version: 3.0

Last major context update: 2026-06-27

Product Identity

TIS stands for Teacher Information System. It is a developing SaaS academic operations platform for schools, school groups, academic leaders, supervisors, and platform owners who need one trusted place for academic staffing, teacher records, planning, calendars, observations, branch context, and future intelligence.

TIS is not only a teacher directory. It is intended to become the operational backbone for academic decision-making across a school or multi-branch organization.

Public product presence:

- 1 Public website: <https://tisplatform.com>
- 1 Application portal: <https://app.tisplatform.com>
- 1 Operational login: </login>
- 1 SaaS signup: </saas/signup>
- 1 SaaS login: </saas/login>
- 1 SaaS account area: </saas/account>

Product Vision

TIS helps schools move away from scattered spreadsheets and disconnected operational files. The product vision is to give academic leaders a structured, secure, and tenant-isolated platform where teacher information, teaching loads, staffing needs, observations, calendars, and branch-level context can be managed together.

As the platform matures, TIS should also become the trusted data foundation for AI-assisted academic operations. Future AI features should depend on verified school data, clear permissions, and careful subscription packaging.

Business Goal

The business goal is to establish TIS as a subscription-based SaaS platform for academic operations. The platform should support school onboarding, plan selection, payment, provisioning, account management, and scalable multi-tenant usage.

Near-term business priorities:

- 1 Convert interested schools from public landing page traffic into SaaS accounts.
- 1 Support clear onboarding from signup through school setup.
- 1 Provide reliable billing and payment status visibility.
- 1 Enable platform owners to manage pending organizations and provisioning.
- 1 Preserve trust by keeping tenant data isolated and operational flows stable.

Educational Goal

The educational goal is to reduce administrative friction so schools can make better academic decisions. TIS should help leadership teams identify staffing gaps, workload problems, incomplete academic coverage, observation follow-up needs, and calendar conflicts earlier.

The platform should support educational quality by making academic operations easier to see, review, and improve.

Target Customers

Primary customers:

- 1 Private schools
- 1 Multi-branch school groups
- 1 Academic leadership teams
- 1 Principals and vice principals
- 1 Department heads and supervisors
- 1 School operations and HR-adjacent academic staff

Internal platform users:

- 1 Platform Owner
- 1 Platform Co-Owner
- 1 Platform Developer

Tenant users:

- 1 Administrators
- 1 Coordinators
- 1 Supervisors
- 1 Teachers and academic staff, depending on enabled workflows

FastAPI App Architecture

The operational TIS application is a FastAPI application at the repository root. It uses Python, SQLAlchemy, Jinja templates, static assets, and modular routers.

Core app areas:

- 1 `main.py`: primary FastAPI app, route registration, app-level workflows, startup checks, platform console routes, dashboard and configuration flows.
- 1 `models.py`: main SQLAlchemy models.
- 1 `database.py`: database connection/session setup.
- 1 `db_migrations.py`: local schema migration and repair logic.
- 1 `auth.py`: authentication, role normalization, platform identity helpers, permission helpers, session handling.
- 1 `authorization.py`: route permission rules and access-denied response helpers.
- 1 `permission_registry.py`: permission groups, labels, defaults, developer-assignable permissions, and system owner permissions.
- 1 `role_permission_service.py`: role permission persistence and related helpers.

- 1 `ui_shell.py`: shared application shell context, navigation, visual identity, and page metadata.
- 1 `routers/`: modular feature routers for users, teachers, subjects, planning, timetable, academic calendar, and observations.
- 1 `templates/`: Jinja templates for the operational app and SaaS app pages.
- 1 `static/`: CSS, JavaScript, images, branding assets, and generated public artifacts.
- 1 `tests/`: pytest coverage for tenant isolation, SaaS phases, platform access, permissions, email, branding, and related workflows.

Important operational route families:

- 1 `/login`: operational app login.
- 1 `/dashboard`: tenant operational dashboard.
- 1 `/platform`: platform console for platform identities.
- 1 `/system-configuration`: branch, year, branding, role permissions, and configuration workflows.
- 1 `/teachers`, `/subjects`, `/planning`, `/timetable`, `/academic-calendar`, `/observations`: core academic operations.

Next.js Landing Architecture

The public marketing website lives in `tis-landing-website/`. It is separate from the FastAPI operational portal.

Landing architecture:

- 1 Runtime: Next.js / Node.
- 1 Public website domain: `https://tisplatform.com`.
- 1 Local development URL: `http://localhost:3000`.
- 1 Main page: `tis-landing-website/src/app/page.tsx`.
- 1 App layout: `tis-landing-website/src/app/layout.tsx`.
- 1 Global styles: `tis-landing-website/src/app/globals.css`.
- 1 Logo component: `tis-landing-website/src/components/tis-logo.tsx`.
- 1 Public assets: `tis-landing-website/public/`.

The application portal remains separate:

- 1 App domain: `https://app.tisplatform.com`.
- 1 Runtime: FastAPI / Python with a relational database.

Legacy landing files in the FastAPI app are not the source of truth:

- 1 `templates/landing.html`
- 1 `static/landing/landing.css`

Those legacy files must not be modified unless explicitly approved.

Multi-Tenant SaaS Strategy

TIS is designed as a multi-tenant SaaS platform. Tenant isolation is a critical rule. School groups, branches, users, academic years, teachers, planning data, timetable data, observations, and configuration records must remain scoped to the correct organization and branch context.

The platform distinguishes between:

- 1 Platform identities: platform owners, co-owners, and developers.
- 1 Tenant identities: users belonging to a school group and branch context.
- 1 SaaS account identities: accounts that move through signup, onboarding, billing, and provisioning.

Platform users may inspect or switch organization context through controlled platform workflows. Tenant users must remain inside their authorized school, branch, academic year, and permission scope.

Critical tenant strategy:

- 1 Do not weaken tenant isolation.
- 1 Do not bypass permission checks.
- 1 Do not assume a platform identity is a tenant identity.
- 1 Do not assume a SaaS account is already provisioned into an operational tenant.
- 1 Keep onboarding, billing, and tenant provisioning as distinct stages.

SaaS Routes And Account Experience

Core SaaS routes:

- 1 /saas/signup: public SaaS account creation.
- 1 /saas/login: SaaS account login.
- 1 /saas/account: SaaS account dashboard.

Related SaaS areas include plan selection, onboarding organization details, contacts, branch setup, academic setup, onboarding review, billing status, checkout summary, checkout return, checkout cancel, sessions, security, and profile pages.

Platform owner SaaS administration exists under /saas-admin for pending organizations, payments, and provisioning workflows.

TIS Account Email Verification Recovery

The accepted Phase 1 verification recovery improvement strengthens the public TIS Account setup journey without changing payment, billing, provisioning, tenant activation, database schema, migrations, operational modules, or the landing website.

Current verification behavior:

- 1 Valid email verification links mark the account email verified/active and redirect to /saas/login with a professional success notice: the customer is told their email has been verified and asked to sign in to continue school workspace setup.
- 1 Expired or invalid verification links show a recovery page instead of a dead-end generic error.
- 1 The recovery page includes a resend verification form.
- 1 Resend verification safely handles unverified accounts, already verified accounts, and unknown email addresses without exposing whether an account exists.

- 1 Password-based accounts that are still pending verification cannot start or continue school workspace setup.
- 1 New customer-facing verification language in this flow uses "TIS Account" and "school workspace setup".

Google and Microsoft sign-in remain future work. OAuth-based verification bypass rules should not be expanded without a separate approved identity decision.

TIS Account Customer-Facing Wording And Branding

The accepted Phase 2 customer-facing wording cleanup improves the public TIS Account and school workspace setup experience without renaming internal /saas routes, modules, models, or stored statuses.

Customer-visible account/setup pages should use professional labels including:

- 1 TIS Account
- 1 Account Setup
- 1 Account Dashboard
- 1 School Workspace Setup
- 1 Organization Profile
- 1 Branch Setup
- 1 Academic Setup
- 1 Subscription Setup
- 1 Secure Payment
- 1 Workspace Activation

Customer pages should not expose "SaaS" as product copy, raw database statuses, tenant/provisioning terminology, provider transaction/subscription identifiers, attempt UUIDs, checkout session internals, plan IDs, or school group IDs. Those internal concepts may remain in backend code, admin views, stored data, tests, and documentation where they describe architecture or platform-owner operations.

The shared customer account shell includes the official full-color horizontal TIS logo on the light account background. Transactional TIS Account emails use an existing official dark-blue TIS wordmark URL. This Phase 2 cleanup did not change payment, billing, provisioning behavior, tenant activation behavior, database schema, migrations, operational modules, or the Next.js landing website. Google/Microsoft login remains future work.

TIS Account Guided Setup Framework

The accepted Phase 3A implementation introduces the shared customer setup framework for the TIS Account dashboard only. The account page now behaves like a guided onboarding console rather than a dense admin dashboard.

The shared framework provides:

- 1 Official TIS logo/header.
- 1 An 8-step customer journey stepper.
- 1 Current step and status banner.
- 1 One primary next action.
- 1 Main content area.
- 1 Help/guidance area.
- 1 Clear messaging that TIS Platform access becomes available after Workspace Activation.

The journey steps are TIS Account, Email Verification, School Workspace Setup, Review & Confirmation, Subscription Selection, Secure Payment, Workspace Activation, and Enter TIS Platform. Step state is derived from existing account, pending organization, onboarding progress, billing, payment, and activation data. Internal route names, stored statuses, billing/payment/provisioning behavior, database schema, migrations, operational modules, the landing website, and OAuth behavior remain unchanged.

Phase 3B applies this shared framework to the five School Workspace Setup onboarding pages: Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup. These pages now use a consistent guided wizard structure with grouped sections, one shared-shell primary CTA, secondary Back/Save Draft actions, concise guidance, and reduced visual clutter.

The Phase 3B onboarding redesign preserves existing form actions, field names, validation behavior, draft behavior, route names, and onboarding state transitions. Subscription/payment/status pages remain future Phase 3 work.

Phase 3C applies the same guided framework to Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages. These pages use one shared-shell primary CTA, customer-safe status labels, concise supporting cards, and explicit messaging that browser return from checkout does not itself confirm payment and that TIS Platform access becomes available after Workspace Activation.

The Phase 3C redesign preserves payment behavior, billing behavior, provisioning behavior, webhook logic, checkout start/launch behavior, stored statuses, route names, database schema, migrations, operational modules, the landing website, OAuth behavior, and admin views.

M1-M5 Completed Milestone Summary

M1: Identity and SaaS foundation

- 1 Established core SaaS account concepts.
- 1 Added initial signup/login/account flows.
- 1 Clarified separation between platform, tenant, and SaaS account identities.
- 1 Added supporting tests for identity and SaaS phase behavior.

M2: Onboarding foundation

- 1 Added structured onboarding stages for organization information, contacts, branches, academic setup, and review.
- 1 Improved the path from SaaS signup toward a provisionable organization.
- 1 Preserved the distinction between pending SaaS organizations and operational tenants.

M3: Billing and plan foundation

- 1 Added pricing, plan catalog, billing status, checkout summary, return, and cancel flows.
- 1 Created billing/payment service modules to keep payment logic separate from operational academic logic.
- 1 Prepared the platform for subscription-based SaaS packaging.

M4: Provisioning foundation

- 1 Added pending organization and provisioning workflows for platform owners.
- 1 Added provisioning queue concepts and retry/run operations.
- 1 Created a controlled path for turning a pending SaaS organization into an operational school context.

M5: Platform access, permissions, and owner controls

- 1 Strengthened platform identity handling.
- 1 Added platform owner/developer concepts and owner management controls.
- 1 Improved permission boundaries for platform and tenant users.
- 1 Added tests around platform access and role permissions.

M7: Subscription Management And Entitlements

- 1 Added normalized entitlement definitions and plan entitlement values resolved from one confirmed active provider subscription.
- 1 Added a customer Subscription Management portal with lifecycle state and centrally controlled allowed actions.
- 1 Added paid branch-quantity increases and scheduled reductions with operational branch-capacity enforcement.
- 1 Added plan upgrades and scheduled downgrades using Paddle-authoritative previews and proration behavior.
- 1 Added scheduled cancellation and reversal while preserving paid access through the confirmed effective period.
- 1 Added provider-sourced billing history and protected, freshly resolved invoice downloads.
- 1 Added webhook idempotency, strict provider/local relationship validation, manual-review fail-closed paths, diagnostics, and guarded reconciliation tooling.

Paddle And Payment Architecture Summary

The payment architecture is organized under the `saas/` package.

Key modules:

- 1 `saas/pricing_service.py`: plan/pricing behavior.
- 1 `saas/payment_service.py`: payment-related service logic.
- 1 `saas/paddle_client.py`: Paddle integration boundary.
- 1 `saas/billing_service.py`: billing status and related account billing behavior.
- 1 `saas/currency_service.py`: currency-related helpers.
- 1 `saas/router.py`: SaaS and SaaS admin routes.
- 1 `saas/entitlement_service.py`: commercial access and paid branch-capacity authority.
- 1 `saas/subscription_portal_service.py`: customer portal composition.
- 1 `saas/subscription_change_service.py`: quantity change previews, submissions, schedules, and webhook reconciliation.
- 1 `saas/subscription_plan_change_service.py`: plan upgrade/downgrade lifecycle.
- 1 `saas/subscription_cancellation_service.py`: cancellation and reversal lifecycle.
- 1 `saas/subscription_lifecycle_service.py`: lifecycle resolver and allowed-action policy.
- 1 `saas/billing_history_service.py`: Paddle transaction history and invoice access.
- 1 `saas/payment_lifecycle_reconciliation_service.py`: guarded repair from attributable authoritative evidence.

Architecture rules:

- 1 Keep Paddle-specific details behind service/client boundaries.

- 1 Do not mix payment logic into academic operations.
- 1 Treat payment status, onboarding status, and provisioning status as related but separate concepts.
- 1 Use platform owner admin views for payment/provisioning oversight.
- 1 Avoid changing live SaaS payment flows unless the task explicitly requires it.
- 1 Keep Paddle credentials and endpoints in environment variables.
- 1 Keep Paddle price mappings environment-specific and store the runtime mapping in `subscription_plan_prices.provider_price_id`.
- 1 Use `scripts/sync_paddle_price_ids.py` with sandbox or production mapping JSON to configure initial checkout price IDs; never hardcode live Paddle IDs in migrations or source.
- 1 If an initial checkout price mapping is missing, fail closed before calling Paddle and show customers a support-oriented Secure Payment message while retaining internal plan/interval/currency diagnostics.
- 1 Paddle transaction checkout uses the public `/saas/payment` launcher as the payment-link page. Set `PADDLE_CHECKOUT_BASE_URL` to `https://app.tisplatform.com/saas/payment`, configure `PADDLE_CLIENT_TOKEN` and `PADDLE_ENVIRONMENT` for Paddle.js, and never expose `PADDLE_API_KEY` to frontend code.
- 1 Paddle is authoritative for prices, previews, proration, transactions, scheduled changes, and invoice documents. TIS stores workflow/audit state but does not invent monetary outcomes.
- 1 Subscription changes send the complete retained recurring item set. Immediate changes prevent provider mutation on payment failure; scheduled changes do not change local entitlements before verified effective evidence.
- 1 Webhooks are signature-verified and idempotent. Ambiguous, mismatched, or incomplete provider evidence fails closed to manual review.

Tenant Provisioning Summary

Tenant provisioning turns a pending SaaS organization into an operational TIS school context. Provisioning should create or connect the required organization records, branches, users, and initial tenant setup while preserving auditability and data isolation.

Key provisioning concepts:

- 1 Pending organization: SaaS onboarding entity not yet fully operational.
- 1 Provisioning job: controlled action to create or update operational tenant structures.
- 1 Platform owner review: human oversight before or during provisioning.
- 1 Retry behavior: failed provisioning work should be recoverable without corrupting tenant data.

Provisioning rules:

- 1 Do not directly create operational tenant data from public signup without the approved provisioning path.
- 1 Keep provisioning idempotent where possible.
- 1 Log or surface errors clearly.
- 1 Do not merge tenant data across school groups.

Landing Page Strategy

The public landing page exists to explain the product, build trust, capture demand, and route interested schools into SaaS signup or demo request flows.

Source of truth:

- 1 The public website implementation is `tis-landing-website/`.
- 1 Marketing content references live in `docs/marketing/`.

Landing priorities:

- 1 Explain the problem of scattered academic operations.
- 1 Present TIS as a connected academic operations platform.
- 1 Show credible platform capabilities.
- 1 Support early access, demo requests, and signup pathways.
- 1 Keep the landing page separate from operational app templates.

Landing rule:

- 1 Do not change landing page design, copy, or architecture during operational backend tasks unless explicitly approved.

Customer Experience Roadmap

Near-term customer experience:

- 1 Clear signup and login path.
- 1 Smooth onboarding for organization, contacts, branches, and academic setup.
- 1 Transparent billing/plan status.
- 1 Clear provisioning status after checkout or owner review.
- 1 Reliable operational login once provisioned.

Medium-term customer experience:

- 1 Better account self-service.
- 1 Clearer subscription lifecycle.
- 1 More guided onboarding and implementation support.
- 1 Improved platform owner visibility into pending organizations and payment status.

Long-term customer experience:

- 1 AI-assisted academic planning and decision support.
- 1 Subscription-gated advanced analytics.
- 1 Intelligent recommendations based on verified tenant data.
- 1 Richer executive visibility across branches and academic years.

Knowledge Management System

TIS uses a Knowledge Management System (KMS) to preserve source-of-truth documentation, current project state, decision history, module history, and generated snapshots.

KMS source documents:

- 1 docs/AI_PROJECT_CONTEXT.md: first-read compact onboarding context for future Codex and ChatGPT conversations.
- 1 docs/TIS_MASTER_CONTEXT.md: durable product, architecture, workflow, roadmap, and critical rules.
- 1 docs/PROJECT_STATE.md: living project state.
- 1 docs/DOCUMENTATION_UPDATE_POLICY.md: mandatory KMS and Knowledge Impact Assessment rules.
- 1 docs/CHANGE_HISTORY.md: chronological summary of meaningful changes.
- 1 docs/adr/: Architecture Decision Records.
- 1 docs/history/: module-specific history.
- 1 docs/engineering/: engineering handbook with module map, repository architecture, user/system flows, and developer onboarding.

PDF philosophy:

- 1 Markdown files are the source of truth.
- 1 The PDF booklet is a generated snapshot.
- 1 The PDF must never be edited manually.
- 1 The PDF must be regenerated when included Markdown source docs change.
- 1 The PDF should later be served through owner-only protected routes.

Generated KMS artifacts:

- 1 static/docs/TIS_Project_Reference_Booklet.pdf
- 1 static/docs/docs_manifest.json

Automatic enforcement:

- 1 root AGENTS.md defines mandatory Codex KMS behavior,
- 1 .kms-impact.yml records the task-level machine-readable KIA,
- 1 scripts/kms.py sync validates KIA before writing, regenerates PDF/manifest artifacts, and runs post-generation freshness checks,
- 1 scripts/kms.py check provides the single read-only local and CI validation command,
- 1 scripts/check_kms_impact.py compares the declaration with changed files and major-change rules,
- 1 scripts/generate_docs_pdf.py --check validates source coverage and snapshot freshness without writing,
- 1 GitHub Actions require validation for pull requests, dev, and master deployment.

Automation never rewrites Markdown. It detects omissions and blocks stale work so reviewed human/AI updates remain authoritative.

Owner-only app access:

- 1 /platform/knowledge: read-only Platform Owner Knowledge Center.
- 1 /platform/knowledge/booklet: protected inline PDF view.
- 1 /platform/knowledge/booklet/download: protected PDF download.

The Knowledge Center is protected by the existing Platform Owner access pattern. It is not public, not a landing page, and does not regenerate or rewrite source docs.

Engineering handbook:

- 1 docs/engineering/TIS_MODULE_MAP.md maps product/system modules and guardrails.
- 1 docs/engineering/REPOSITORY_ARCHITECTURE.md explains repository ownership and risky files.
- 1 docs/engineering/USER_AND_SYSTEM_FLOWS.md documents end-to-end customer, SaaS, payment, provisioning, operational, platform owner, KMS, and developer flows.
- 1 docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md explains data areas, ownership boundaries, and tenant isolation rules.
- 1 docs/engineering/DEVELOPMENT_STANDARDS.md defines non-negotiable engineering rules.
- 1 docs/engineering/UI_UX_DESIGN PHILOSOPHY.md defines design direction for operational, platform, SaaS, Knowledge Center, and landing surfaces.
- 1 docs/engineering/PRODUCT_ROADMAP.md records completed, current, next, and future roadmap.
- 1 docs/engineering/REJECTED_DECISIONS.md records significant rejected alternatives.
- 1 docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md defines future visual documentation standards.
- 1 docs/engineering/AI_OPTIMIZATION_GUIDE.md guides future AI assistants.
- 1 docs/engineering/PROJECT_GOVERNANCE.md defines ownership, approvals, quality gates, documentation gates, and traceability.
- 1 docs/engineering/KNOWLEDGE_LIFECYCLE.md defines documentation, engineering, approval, review, release, and maintenance lifecycle.
- 1 docs/engineering/DOCUMENTATION_AUTOMATION.md defines current automation and future automation rules.
- 1 docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md formalizes KIA.
- 1 docs/engineering/SELF_EVOLVING_WORKFLOW.md defines the official task-to-deployment workflow.
- 1 docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md explains KMS propagation.
- 1 docs/engineering/AI_CODING_WORKFLOW.md defines future AI coding discipline.
- 1 docs/engineering/FUTURE_AUTOMATION_ROADMAP.md records future automation opportunities.

KMS v1.0 status:

KMS v3.0 Phase 3D completes the KMS v1.0 lifecycle foundation. Future work should improve automation, search, visuals, route inventories, test strategy docs, deployment runbooks, and deeper module guides only through approved phases.

Development Workflow

Default workflow for approved implementation tasks:

- 1 Inspect the relevant code and docs before editing.
- 2 Keep changes scoped to the approved task.
- 3 Preserve tenant isolation, permission checks, SaaS flows, and landing page boundaries.
- 4 Update tests or add focused tests when behavior changes.
- 5 Complete the Knowledge Impact Assessment and update `.kms-impact.yml`.
- 6 Update relevant Markdown docs.
- 7 Update change history, ADRs, module history, and AI project context when needed.

- 8 Run `scripts/kms.py sync` if included docs changed.
- 9 Run `scripts/kms.py check` for final read-only enforcement.
- 10 Run reasonable implementation validation.
- 11 Report code changes, docs changes, KIA, validation, assumptions, and known issues.

Knowledge Impact Assessment Rule

Every approved implementation must:

- 1 Assess knowledge impact.
- 2 Update relevant Markdown docs.
- 3 Update `docs/CHANGE_HISTORY.md` for meaningful changes.
- 4 Create or update ADRs when major decisions change.
- 5 Update module history when a module's documented state changes.
- 6 Update `docs/AI_PROJECT_CONTEXT.md` when high-level AI onboarding context changes.
- 7 Regenerate the PDF booklet if included docs changed.
- 8 Mention KIA details in the final report.

A task is not complete until the KIA is assessed.

Required final report KIA template:

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

The generated booklet output is:

- 1 `static/docs/TIS_Project_Reference_Booklet.pdf`

Critical Rules Codex Must Follow

- 1 Do not touch SaaS flows unless the task explicitly requires it.
- 1 Do not touch operational logic unless the approved task requires it.
- 1 Do not touch database migrations or `tis.db` unless explicitly approved.
- 1 Do not weaken tenant isolation.
- 1 Do not bypass route permissions or platform owner checks.
- 1 Do not merge platform user, SaaS account, and tenant user concepts.
- 1 Do not change the landing page design or legacy landing files unless explicitly approved.
- 1 Do not add KMS regenerate behavior unless explicitly approved.
- 1 Do not expose the KMS PDF through direct public static links in the app UI.
- 1 Do not push or commit unless explicitly requested.

- 1 Treat production memory as a hard budget. Do not add unbounded full-dataset caches, duplicate production template renders, startup-heavy work, or normal-request diagnostic logging that can trigger Render restarts and 502s.
- 1 Prefer conservative, dependency-light automation.
- 1 Use `reportlab` for the documentation PDF generator.
- 1 Do not require LaTeX, Playwright, Chromium, external network calls, or system font dependencies for PDF generation.
- 1 Always include a Knowledge Impact Assessment in implementation final reports.
- 1 Always keep `.kms-impact.yml` aligned with the task and actual Git diff.
- 1 Never put customer, personal, production, credential, secret, environment, transaction, invoice, webhook payload, or database-row data into KMS documentation.

docs/PROJECT_STATE.md

TIS Project State

Last Updated

Last updated: 2026-07-21

Update this file after every meaningful milestone, active development change, roadmap shift, known issue change, or documentation/KMS change.

Current Branch Strategy

Current working branch assumption: dev.

Branch strategy:

- 1 Development work should happen on dev unless the owner explicitly requests another branch.
- 1 Production/live branch is assumed to be separate from active development.
- 1 Confirm production branch before any deployment, merge, or production-sensitive change.
- 1 Do not push, merge, or commit unless explicitly requested.
- 1 Preserve unrelated local changes.

Production / Live Branch Assumption

The live production branch is assumed to be the branch deployed to the public app environment, while dev is the active development branch. This assumption must be confirmed before deployment.

Production domains:

- 1 Public website: <https://tisplatform.com>
- 1 Application portal: <https://app.tisplatform.com>

Completed Milestones

M1: Identity and SaaS foundation

- 1 Core SaaS signup/login/account concepts established.
- 1 Platform, tenant, and SaaS account identities separated.
- 1 Identity and SaaS phase tests present.

M2: Onboarding foundation

- 1 SaaS onboarding flow covers organization, contacts, branches, academic setup, and review.
- 1 Pending organization concept supports pre-provisioning state.

M3: Billing and plan foundation

- 1 Plan catalog, checkout, billing status, checkout return, and checkout cancel flows exist.
- 1 Payment and billing code is isolated under saas/ service modules.

- 1 Initial Paddle checkout price IDs are configured through a script-based mapping sync into `subscription_plan_prices.provider_price_id`.
- 1 Sandbox and production Paddle price mappings must remain separate; real mapping files are ignored and credentials stay in environment variables.

M4: Provisioning foundation

- 1 Platform owner provisioning views and actions exist.
- 1 Pending organization review and provisioning queue behavior exist.
- 1 Provisioning retry/run operations are present.

M5: Platform access and owner controls

- 1 Platform owner and platform developer identities exist.
- 1 Platform console and owner/developer management controls exist.
- 1 Permission registry and platform access tests support this boundary.

SaaS account setup stabilization:

- 1 Phase 1 TIS Account email verification recovery is accepted.
- 1 Valid verification links now redirect to TIS Account login with a professional success notice.
- 1 Expired or invalid verification links now show a recovery page with a resend option.
- 1 Resend verification safely handles unverified, already verified, and unknown-email cases.
- 1 Unverified password-based accounts are blocked from starting or continuing school workspace setup.
- 1 New verification-flow wording uses "TIS Account" and "school workspace setup".
- 1 Payment, billing, provisioning, database schema, migrations, operational modules, and the landing website were not changed.
- 1 Google/Microsoft login remains future work and was not implemented.

SaaS customer-facing wording and branding:

- 1 Phase 2 TIS Account customer-facing wording cleanup is accepted.
- 1 Customer account/setup pages now use professional labels such as TIS Account, Account Dashboard, School Workspace Setup, Organization Profile, Branch Setup, Academic Setup, Subscription Setup, Secure Payment, and Workspace Activation.
- 1 The shared customer account shell uses the official full-color horizontal TIS logo on the light account background.
- 1 Transactional TIS Account emails use an existing official dark-blue TIS wordmark asset.
- 1 Customer views label internal statuses through customer-safe wording instead of exposing raw tenant, provisioning, checkout, provider, plan, school group, or attempt identifiers.
- 1 Internal `/saas` route/module/model names and stored statuses were not renamed.
- 1 Payment, billing, provisioning behavior, database schema, migrations, operational modules, and the landing website were not changed.
- 1 Google/Microsoft login remains future work and was not implemented.

SaaS guided setup framework:

- 1 Phase 3A shared TIS Account guided setup framework is accepted.
- 1 The customer account shell now supports an official-logo guided setup console with an 8-step journey stepper, status banner, one primary next action, main content area, and help/guidance area.
- 1 The account page now uses the framework and removes the old dense dashboard statistics from the customer account landing page.
- 1 Journey state is derived from existing account, onboarding, billing, payment, and activation data without changing stored statuses.
- 1 Onboarding forms, subscription/payment pages, billing/status pages, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the landing website, internal /saas route names, and Google/Microsoft login were not changed.
- 1 Phase 3B redesigned the five School Workspace Setup onboarding pages on top of the shared setup shell.
- 1 Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup now use consistent guided wizard sections, one shared-shell primary CTA, and secondary Back/Save Draft actions.
- 1 Phase 3B preserved backend logic, form actions, field names, validation behavior, draft behavior, onboarding progression, payment/billing/provisioning behavior, database schema, migrations, operational modules, the landing website, internal /saas route names, and OAuth behavior.
- 1 Phase 3C redesigned Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages using the same shared setup shell and guided style.
- 1 Phase 3C added customer-safe browser-return/payment confirmation guidance and explicit TIS Platform access-after-activation messaging without changing payment, billing, provisioning, webhook, checkout start/launch, database, migration, operational, landing, route, stored-status, admin, or OAuth behavior.

Documentation/KMS milestones:

- 1 Phase 1 documentation foundation completed and pushed to dev.
- 1 Phase 2A and Phase 2B KMS foundation approved for implementation.
- 1 Phase 2C Platform Owner Knowledge Center completed and pushed to dev.
- 1 KMS v3.0 Phase 3A Engineering Handbook approved for implementation.
- 1 KMS v3.0 Phase 3B approved for implementation.
- 1 KMS v3.0 Phase 3C approved for implementation.
- 1 KMS v3.0 Phase 3D final phase approved for implementation.
- 1 Automatic KMS enforcement implemented with repository instructions, machine-readable KIA, major-change detection, read-only artifact checks, CI validation, and a deployment prerequisite.
- 1 Phase 6 unified KMS command implemented with `scripts/kms.py sync` and `scripts/kms.py check`.

M7 subscription-management milestones:

- 1 Phase 1 entitlement foundation completed.
- 1 Phase 2 customer Subscription Management portal completed.
- 1 Phase 3 active paid branch-quantity management completed.
- 1 Phase 4 upgrades and scheduled downgrades completed with provider-authoritative previews/proration.
- 1 Phase 5 cancellation/reversal and centralized lifecycle/action policy completed.
- 1 Phase 6 provider billing history and protected invoice management completed.

- 1 Webhook and reconciliation safeguards were added across the M7 lifecycle.

Current Priority

Current priority: enforce automatic KMS synchronization and keep the engineering handbook aligned with completed M7 subscription behavior.

Current enforcement scope:

- 1 Codex reads root `AGENTS.md` and authoritative KMS context.
- 1 Every task updates `.kms-impact.yml`.
- 1 Major-change paths are conservatively classified by `scripts/check_kms_impact.py`.
- 1 Local KMS synchronization runs through `scripts/kms.py sync`.
- 1 Generated artifacts and KIA are validated read-only through `scripts/kms.py check`.
- 1 Pull requests and dev pushes run KMS enforcement.
- 1 `master` deployment waits for the KMS gate.
- 1 Automation validates and blocks; it does not rewrite Markdown.

Phase 2A and Phase 2B scope:

- 1 Create documentation update policy.
- 1 Create change history.
- 1 Create ADR foundation and initial accepted ADRs.
- 1 Create module history foundation.
- 1 Create AI project context.
- 1 Update master context, project state, and documentation index.
- 1 Update PDF generator to include KMS docs and manifest metadata.
- 1 Regenerate `static/docs/TIS_Project_Reference_Booklet.pdf`.

Phase 2C completed scope:

- 1 Added read-only `knowledge_service.py` as the single KMS app access layer.
- 1 Added owner-protected `/platform/knowledge` page.
- 1 Added owner-protected PDF view/download routes.
- 1 Added an owner-only Platform Console card.
- 1 Added platform knowledge module history.
- 1 Regenerated the PDF and manifest after documentation updates.

Still out of scope:

- 1 Regenerate button.
- 1 Additional app routes beyond the approved read-only Knowledge Center routes.
- 1 `ui_shell.py` and `authorization.py` changes unless separately approved.
- 1 SaaS flows.

- 1 Operational logic.
- 1 Database, migrations, or `tis.db`.
- 1 Landing page implementation.

KMS v3.0 Phase 3A scope:

- 1 Add complete TIS module map.
- 1 Add repository architecture map.
- 1 Add end-to-end user/system workflows.
- 1 Add clear AI/human developer onboarding structure.
- 1 Update generator to include engineering docs.
- 1 Regenerate the PDF and manifest.

KMS v3.0 Phase 3B scope:

- 1 Add database architecture overview.
- 1 Add development standards and non-negotiable rules.
- 1 Add UI/UX and design philosophy.
- 1 Add product roadmap.
- 1 Strengthen AI/human developer onboarding guidance.
- 1 Update generator to include the new engineering docs.
- 1 Regenerate the PDF and manifest.

KMS v3.0 Phase 3C scope:

- 1 Add rejected architectural decisions.
- 1 Add visual documentation framework.
- 1 Add AI optimization guide.
- 1 Add project governance and decision traceability.
- 1 Update generator to include the new engineering docs.
- 1 Regenerate the PDF and manifest.

KMS v3.0 Phase 3D final scope:

- 1 Add knowledge lifecycle documentation.
- 1 Add documentation automation guide.
- 1 Add formal KIA standard.
- 1 Add self-evolving workflow.
- 1 Add documentation dependency map.
- 1 Add AI coding workflow.
- 1 Add future automation roadmap.
- 1 Regenerate the PDF and manifest.

Current Known Issues

Known issues and watch points:

- 1 KMS policy depends on future developers and AI agents consistently completing the Knowledge Impact Assessment.
- 1 Generated PDF can become stale during local work, but CI now blocks stale artifacts from integration/deployment.
- 1 The owner-only Knowledge Center is implemented as read-only; there is no regenerate button yet.
- 1 Public static storage is not sufficient access control for sensitive docs; Phase 2C should serve docs through protected owner-only routes.
- 1 Render deployment constraints should continue to guide dependency choices.
- 1 Production memory must be treated as a hard constraint. The 2026-06-27 Render restart/502 investigation found two avoidable memory risks: observation diagnostics doing extra production template renders and global location lookup parsing a 47 MB dataset into a complete in-memory index for simple picker requests. Local stabilization changes now gate observation diagnostics and use scoped location loading; future work must follow the Production Memory and Render Stability standards.
- 1 Broad filesystem scans may warn about `tis_scope_test_5i3yf0h5/` access denial.
- 1 Google/Microsoft login is still future work; password-based accounts must remain email-verified before school workspace setup.
- 1 GitHub repository settings must mark KMS Enforcement / `kms-check` as required on protected branches; this cannot be configured by repository file changes alone.

Next Planned Work

Next planned work:

- 1 Review the KMS enforcement rules against real pull requests and tune only demonstrated false positives.
- 1 Keep M7 documentation current as subscription fixes evolve.
- 1 Later consider an explicit owner-only regenerate workflow.
- 1 Review, commit, and deploy the production memory stabilization changes when approved, then monitor Render memory, restart count, and route-level 502s after deployment.

Landing Page Baseline Situation

The public landing page source of truth is:

- 1 `tis-landing-website/`

Marketing docs:

- 1 `docs/marketing/landing_page_source_of_truth.md`
- 1 `docs/marketing/tis_landing_page_master_content.md`

Relevant ADRs:

- 1 `docs/adr/0001-separate-nextjs-landing-website.md`
- 1 `docs/adr/0007-landing-page-visual-system-strategy.md`

Legacy FastAPI landing files are not the current public website source of truth:

```
1 templates/landing.html
1 static/landing/landing.css
```

Do not modify landing page design, landing copy, or legacy landing files unless explicitly approved.

Knowledge Update Policy

Every approved implementation must complete the KIA:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until KIA is assessed and `.kms-impact.yml` matches the actual task diff. If included docs change, regenerate:

```
1 static/docs/TIS_Project_Reference_Booklet.pdf
```

Then run `.\.venv\Scripts\python.exe scripts\kms.py` check for final read-only validation. When documentation changes, `.\.venv\Scripts\python.exe scripts\kms.py sync` performs generation and post-generation validation together.

Scope Guardrails

- 1 Do not touch SaaS flows unless explicitly approved.
- 1 Do not touch operational logic unless required by the approved task.
- 1 Do not touch database migrations or `tis.db` unless explicitly approved.
- 1 Do not change the landing page unless explicitly approved.
- 1 Do not add a KMS regenerate button until separately approved.
- 1 Do not let automation rewrite authoritative Markdown.
- 1 Do not place customer, personal, production, billing-record, transaction, invoice, webhook payload, credential, secret, environment, or database-row data in KMS docs.
- 1 Do not commit or push unless explicitly requested.

docs/CHANGE_HISTORY.md

TIS Change History

This file is the chronological summary of meaningful TIS changes. It does not replace module history under docs/history/; it gives reviewers, developers, Codex, and ChatGPT a fast timeline of what changed and why.

Newest entries should be added first.

Entry Template

YYYY-MM-DD - Short Change Title

Area/module:
Previous state:
New state:
Reason:
Files changed:
Documentation updated:
PDF regenerated:
AI project context updated:
Reviewer/approval notes:

2026-07-21 - Added Unified Phase 6 KMS Commands

Area/module: KMS developer workflow, local automation, CI, and deployment validation

Previous state: Developers separately ran the PDF generator, artifact freshness check, and KIA impact checker. The repository had all required primitives but no single synchronization command or canonical read-only command shared by local work and CI.

New state: scripts/kms.py sync validates the task KIA before writing, regenerates the PDF and manifest through the existing generator, runs complete post-generation validation, and prints a concise summary. scripts/kms.py check delegates to the existing read-only enforcement logic. GitHub pull-request, dev, and deployment gates use the unified check command.

Reason: One reliable command reduces missed mechanical steps while preserving reviewed Markdown as the source of truth and keeping enforcement strict.

Files changed:

- 1 KMS command orchestrator and reusable checker API
- 1 KMS automation tests
- 1 repository instructions and CI workflows
- 1 KMS workflow documentation and generated artifacts

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: The command never rewrites authoritative Markdown and does not change application behavior or production data.

2026-07-21 - Aligned Push Enforcement With KIA Task Boundaries

Area/module: GitHub Actions and KMS impact validation

Previous state: Pull-request enforcement validated the full feature branch against its base, while push enforcement validated only `github.event.before...github.sha`. A follow-up fix commit therefore evaluated a cumulative `.kms-impact.yml` against only the latest commit and incorrectly reported previously updated KMS files as unchanged.

New state: Pull requests validate the pull-request base SHA against the actual pull-request head SHA. Pushes to dev find the merge base between the repository default branch and the pushed head, then validate that complete task range. Both events apply the unchanged strict declaration and generated-artifact checks to the same logical implementation boundary.

Reason: KIA declarations describe approved implementation tasks, which may contain multiple commits. Event delivery boundaries must not redefine those tasks.

Files changed:

- 1 KMS impact checker
- 1 KMS enforcement workflow
- 1 KMS automation regression tests
- 1 KMS governance documentation and generated artifacts

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; product architecture, developer onboarding order, and application behavior are unchanged.

Reviewer/approval notes: Enforcement remains strict across the complete task diff. No application behavior or production data changed.

2026-07-21 - Made KMS Enforcement Cross-Platform Deterministic

Area/module: Repository KMS generation, freshness validation, and CI enforcement

Previous state: Markdown source hashes used raw checkout bytes, and dynamically discovered ADR and history sources used native `Path` ordering. A Windows checkout with CRLF line endings could generate a manifest that passed locally but failed on GitHub Linux, where the same committed text used LF and path ordering differed.

New state: Markdown is decoded as UTF-8, normalized to LF, and then hashed. Source paths are normalized to repository-relative POSIX paths, dynamic sources use a stable case-insensitive ordering with a deterministic tie-breaker, and source comparison still rejects missing, unexpected, duplicate, or reordered entries. Git diff inspection now includes deleted files.

Reason: KMS enforcement must evaluate committed content consistently across developer workstations and GitHub Actions without weakening freshness or source-coverage checks.

Files changed:

- 1 KMS generator and impact checker
- 1 Knowledge Center freshness hashing helper
- 1 KMS automation tests and line-ending attributes
- 1 generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; onboarding, architecture, product behavior, and current priorities are unchanged.

Reviewer/approval notes: Repository-governance correction only. No application behavior, production data, SaaS flows, database, or migrations changed.

2026-07-21 - Added Automatic KMS Synchronization Enforcement

Area/module: Repository governance, KMS automation, CI, deployment gate, and AI workflow

Previous state: KMS updates depended on developers and AI assistants remembering the written KIA policy.

PDF/manifest generation was manually triggered, the Knowledge Center detected stale hashes only when viewed, and no test, commit, pull-request, or deployment gate blocked stale or missing documentation.

New state: Root `AGENTS.md` makes KMS onboarding mandatory. `.kms-impact.yml` records task-level impact. `scripts/check_kms_impact.py` compares declarations with Git changes, conservatively classifies major paths, validates declared Markdown, and invokes generated-artifact checks. The PDF generator has read-only `--check` mode and manifest PDF hashes. GitHub Actions enforce checks on pull requests and dev, and master deployment depends on the same gate. Automation never rewrites Markdown.

Reason: Major TIS work must not be mergeable or deployable while engineering knowledge is stale, while reviewed Markdown must remain authoritative and free from runtime/customer data.

Files changed:

- 1 `AGENTS.md`
- 1 `.kms-impact.yml`
- 1 `.github/pull_request_template.md`
- 1 `.github/workflows/kms-enforcement.yml`
- 1 `.github/workflows/deploy-on-master.yml`
- 1 `scripts/check_kms_impact.py`
- 1 `scripts/generate_docs_pdf.py`
- 1 `tests/test_kms_automation.py`
- 1 relevant KMS Markdown and generated artifacts

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Repository-governance automation only. No application behavior, production/customer/tenant/billing data, runtime records, database, migrations, or business logic changed.

2026-07-20 - Backfilled Completed M7 Subscription Management

Area/module: SaaS entitlements, subscription portal, quantity/plan changes, cancellation, billing history, invoices, Paddle webhooks, and reconciliation

Previous state: Module history covered M7 Phases 1, 2, 3, and 6 only. Central project state, architecture maps, workflows, roadmap, change history, PDF, and manifest still described the pre-M7 billing foundation; Phases 4 and 5 and reconciliation protections were absent.

New state: KMS records the completed M7 entitlement foundation, read/write customer portal, paid branch quantity management, upgrades and scheduled downgrades, provider-authoritative proration, cancellation/reversal, centralized

lifecycle and allowed-action policy, provider billing history, protected invoice downloads, and fail-closed webhook/reconciliation safeguards.

Reason: The engineering handbook must describe current implemented subscription behavior before automatic enforcement becomes authoritative.

Files changed:

- 1 central KMS context/state files
- 1 subscription and payment ADRs
- 1 engineering module, repository, database, flow, and roadmap docs
- 1 subscription module history
- 1 generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Documentation backfill only; it records already committed M7 behavior and introduces no SaaS behavior change.

2026-06-30 - Paddle Initial Checkout Price Mapping Configuration

Area/module: SaaS subscriptions, Paddle initial checkout configuration, tests, and KMS documentation

Previous state: The checkout launch flow safely blocked when the selected subscription plan price did not have `subscription_plan_prices.provider_price_id` configured, but there was no structured mapping sync process and the customer-facing error could expose provider configuration wording.

New state: Added a script-based Paddle price ID sync process using structured sandbox/production mapping examples. The database remains the source of truth through `subscription_plan_prices.provider_price_id`, real mapping files are ignored, and missing provider price IDs now surface a customer-safe Secure Payment support message while internal diagnostics retain plan code, billing interval, and currency details.

Reason: Initial subscription checkout needs environment-specific Paddle provider price IDs without hardcoding live IDs in source or changing payment state behavior.

Files changed:

- 1 .gitignore
- 1 scripts/sync_paddle_price_ids.py
- 1 config/paddle/paddle_prices.sandbox.example.json
- 1 config/paddle/paddle_prices.production.example.json
- 1 saas/payment_service.py
- 1 saas/router.py
- 1 tests/test_paddle_price_sync.py
- 1 tests/test_saas_phase1.py
- 1 docs/AI_PROJECT_CONTEXT.md
- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/PROJECT_STATE.md

```
1 docs/CHANGE_HISTORY.md
1 docs/history/subscriptions/README.md
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Initial checkout configuration only. No proration, upgrade, downgrade, cancellation, payment state transition, webhook, provisioning behavior, database schema, migration, operational module, landing website, OAuth, internal route rename, live Paddle ID hardcoding, commit, or push was performed.

2026-06-27 - Accepted Subscription And Workspace Activation Guided Journey Phase 3C

Area/module: Subscription Selection, Secure Payment summary, Payment Return/Cancel, Subscription Status, Workspace Activation status, tests, and KMS documentation

Previous state: The subscription, secure payment, billing status, payment return/cancel, and workspace activation pages were functionally correct but still used page-local CTAs, dense status blocks, and inconsistent guidance. Browser return messaging existed but was not part of the shared guided setup experience.

New state: Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages now use the Phase 3A shared setup shell and Phase 3B guided page style. Each page has one shared-shell primary CTA, customer-safe status labels, concise supporting cards, clear browser-return guidance, and explicit messaging that TIS Platform access becomes available after Workspace Activation.

Reason: The accepted Phase 3C scope required making subscription/payment/activation pages feel like one guided customer journey while preserving payment, billing, provisioning, webhook, checkout start/launch, database, and operational behavior.

Files changed:

```
1  saas/router.py
1  templates/saas/plan_selection.html
1  templates/saas/checkout_summary.html
1  templates/saas/checkout_return.html
1  templates/saas/checkout_cancel.html
1  templates/saas/account_billing.html
1  templates/saas/billing_status.html
1  tests/test_saas_phase1.py
1  docs/AI_PROJECT_CONTEXT.md
1  docs/TIS_MASTER_CONTEXT.md
1  docs/PROJECT_STATE.md
1  docs/CHANGE_HISTORY.md
```



```
1 docs/history/saas-onboarding/README.md
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 3C customer-facing subscription/payment/status redesign only. No payment behavior change, billing behavior change, provisioning behavior change, webhook logic change, checkout start/launch behavior change, database schema change, migration, operational module change, Next.js landing website change, OAuth change, internal /saas route rename, stored-status change, admin view change, commit, or push was performed.

2026-06-27 - Accepted School Workspace Setup Guided Wizard Phase 3B

Area/module: School Workspace Setup onboarding templates, shared SaaS customer shell, onboarding setup context, tests, and KMS documentation

Previous state: The five onboarding pages used functional but dense form layouts with repeated progress notices, mixed card styles, and page-local primary actions. Branch setup felt like repeated blank blocks, organization logo upload was plain, and the review page felt like a basic summary rather than a confident handoff to Subscription Selection.

New state: Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup now use a consistent guided enterprise setup wizard style on top of the Phase 3A shared shell. Each page has one shared-shell primary CTA, secondary Back/Save Draft actions, grouped form sections, concise guidance, cleaner spacing, a more premium logo upload area, compact branch panels, and a stronger review summary.

Reason: The accepted Phase 3B scope required redesigning only the School Workspace Setup onboarding pages while preserving all business logic, routes, field names, validation, draft behavior, payment, billing, provisioning, database, and operational boundaries.

Files changed:

```
1 saas/router.py
1 templates/saas/base.html
1 templates/saas/onboarding_organization.html
1 templates/saas/onboarding_branches.html
1 templates/saas/onboarding_academic_setup.html
1 templates/saas/onboarding_contacts.html
1 templates/saas/onboarding_review.html
1 tests/test_saas_phase1.py
1 docs/AI_PROJECT_CONTEXT.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/CHANGE_HISTORY.md
1 docs/history/saas-onboarding/README.md
```

```
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 3B onboarding page redesign only. No backend business logic change, route rename, form field rename, validation change, onboarding progression change, draft behavior change, payment behavior change, billing behavior change, provisioning behavior change, database schema change, migration, operational module change, Next.js landing website change, OAuth change, commit, or push was performed.

2026-06-27 - Accepted TIS Account Guided Setup Framework Phase 3A

Area/module: TIS Account customer dashboard, shared SaaS customer shell, setup journey display helper, and KMS documentation

Previous state: The customer account page still behaved like a dense dashboard with statistics, session details, multiple competing actions, and page-specific journey fragments. The shared customer shell had a logo and onboarding-specific progress UI, but it did not yet provide a reusable 8-step guided setup framework.

New state: The shared customer shell supports a guided setup console for pages that pass setup context. The TIS Account page now presents an official-logo guided console with an 8-step journey stepper, current-step/status area, one primary next action, concise account/workspace context, and guidance that TIS Platform access becomes available after Workspace Activation. Journey state is calculated from existing account, onboarding, billing, payment, and activation data without changing stored statuses.

Reason: The accepted Phase 3A scope required only the shared framework and account page foundation for a professional TIS Account / School Workspace Setup experience, while leaving full onboarding and payment page redesigns for later phases.

Files changed:

```
1  saas/router.py
1  saas/service.py
1  templates/saas/base.html
1  templates/saas/account.html
1  tests/test_saas_phase1.py
1  docs/AI_PROJECT_CONTEXT.md
1  docs/TIS_MASTER_CONTEXT.md
1  docs/PROJECT_STATE.md
1  docs/CHANGE_HISTORY.md
1  docs/history/saas-onboarding/README.md
1  static/docs/TIS_Project_Reference_Booklet.pdf
1  static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 3A shared framework only. Onboarding forms, subscription/payment pages, billing/status pages, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login, internal /saas route names, admin views, commits, and pushes were not changed.

2026-06-27 - Accepted TIS Account Customer-Facing Wording And Logo Cleanup

Area/module: SaaS customer account pages, school workspace setup pages, billing/subscription status views, transactional account emails, and KMS documentation

Previous state: Customer-facing TIS Account and school workspace setup pages could display internal or technical language such as SaaS-oriented product copy, raw status labels, checkout/payment internals, provider identifiers, tenant/provisioning terms, or account setup labels that were less polished. The shared customer account shell did not consistently present an official TIS logo image across inherited customer forms/pages.

New state: Customer-facing account/setup pages use professional labels such as TIS Account, Account Dashboard, School Workspace Setup, Organization Profile, Branch Setup, Academic Setup, Subscription Setup, Secure Payment, and Workspace Activation. Customer views use display labels for internal statuses and hide customer-irrelevant provider transaction/subscription IDs, attempt UUIDs, checkout session internals, plan IDs, and school group IDs. The shared customer account shell uses the official full-color horizontal TIS logo, and transactional TIS Account emails use an existing official dark-blue TIS wordmark asset.

Reason: The accepted Phase 2 plan required a focused customer-facing wording cleanup and official logo usage pass before any larger account setup UI redesign.

Files changed:

```
1    saas/router.py
1    saas/service.py
1    saas/provisioning_service.py
1    email_templates.py
1    templates/saas/base.html
1    templates/saas/signup.html
1    templates/saas/login.html
1    templates/saas/account.html
1    templates/saas/account_billing.html
1    templates/saas/billing_status.html
1    templates/saas/onboarding_organization.html
1    templates/saas/onboarding_branches.html
1    templates/saas/onboarding_academic_setup.html
1    templates/saas/onboarding_contacts.html
1    templates/saas/onboarding_review.html
1    templates/saas/plan_selection.html
1    templates/saas/checkout_summary.html
```

```
1    templates/saas/checkout_return.html
1    templates/saas/checkout_cancel.html
1    templates/saas/profile.html
1    templates/saas/security.html
1    templates/saas/sessions.html
1    tests/test_saas_phase1.py
1    tests/test_saas_phase5.py
1    tests/test_email_templates.py
1    docs/AI_PROJECT_CONTEXT.md
1    docs/TIS_MASTER_CONTEXT.md
1    docs/PROJECT_STATE.md
1    docs/CHANGE_HISTORY.md
1    docs/history/saas-onboarding/README.md
1    static/docs/TIS_Project_Reference_Booklet.pdf
1    static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 2 implementation only. No Phase 3 UI redesign, Google/Microsoft login, internal route/module rename, payment behavior change, billing behavior change, provisioning behavior change, database schema change, migration change, operational module change, Next.js landing website change, commit, or push was performed.

2026-06-27 - Accepted TIS Account Email Verification Recovery

Area/module: SaaS onboarding, TIS Account email verification, verification resend recovery, and school workspace setup gate

Previous state: Valid verification links rendered a static verification page instead of continuing the customer toward account setup. Expired or invalid verification links could feel like a dead end. Resend verification existed but did not provide a fully professional recovery path for expired links, already verified accounts, and unknown emails. Password-based accounts that were still pending verification could sign in and reach account/setup routes.

New state: Valid verification links mark the account email verified/active and redirect to the TIS Account login page with a professional success notice. Expired or invalid links show a recovery page with a resend option. Resend verification safely handles unverified accounts, already verified accounts, and unknown-email cases without revealing account existence. Unverified password-based accounts are blocked from starting or continuing school workspace setup. New visible wording in this verification flow uses "TIS Account" and "school workspace setup".

Reason: Testing showed that the customer account setup journey could be blocked after email verification, especially when a verification link expired or the customer needed to recover/resend the link.

Files changed:

```
1    saas/router.py
```

```
1    saas/service.py
1    templates/saas/login.html
1    templates/saas/verify_email.html
1    templates/saas/verification_sent.html
1    email_templates.py
1    tests/test_saas_phase1.py
1    tests/test_saas_phase5.py
1    docs/AI_PROJECT_CONTEXT.md
1    docs/TIS_MASTER_CONTEXT.md
1    docs/PROJECT_STATE.md
1    docs/CHANGE_HISTORY.md
1    docs/history/saas-onboarding/README.md
1    static/docs/TIS_Project_Reference_Booklet.pdf
1    static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 1 verification recovery implementation accepted. Phase 2 wording cleanup, Phase 3 setup UI redesign, Google/Microsoft login, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, and the Next.js landing website were not changed. No commit or push was performed.

2026-06-27 - Added Production Memory Stability Guardrails

Area/module: Operational app stability, observations, global location lookup, Render deployment constraints, and engineering standards

Previous state: Production traffic could hit avoidable memory pressure. The observations page included diagnostic stage logging and extra template rendering in the normal request path, and the global location picker could parse a 47 MB reference dataset into a complete in-memory index for simple picker requests. KMS standards did not yet explicitly forbid unbounded caches, duplicate production template renders, or normal-request diagnostic warning spam.

New state: The local stabilization patch gates observation diagnostics behind TIS_OBSERVATION_DIAGNOSTICS, removes duplicate observation template pre-renders, and changes location lookup behavior to use streaming/scoped country loading for normal country, region, city, and validation requests. KMS now documents strict production memory and Render stability rules.

Reason: Render logs showed app restarts and user-facing 502s around normal app navigation after deployment. A 512 MB service can be enough for TIS only if the app avoids unnecessary full-dataset memory loads, duplicate rendering, and production debug noise.

Files changed:

```
1    location_service.py
1    routers/observations.py
1    docs/engineering/DEVELOPMENT_STANDARDS.md
```

```
1 docs/PROJECT_STATE.md
1 docs/AI_PROJECT_CONTEXT.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/CHANGE_HISTORY.md
1 docs/history/engineering-handbook/2026-06-27-production-memory-stability-guardrails.md
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Local changes only. No commit, push, migration, database change, SaaS route change, billing change, tenant logic change, or deployment was performed.

2026-06-26 - Completed KMS v3.0 Phase 3D Lifecycle Foundation

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3C documented rejected decisions, visual documentation framework, AI optimization, governance, and traceability. It still needed a complete self-evolving lifecycle standard that ties implementation, validation, KIA, documentation updates, generated artifacts, review, commit, push, and deployment together.

New state: TIS now has final Phase 3D docs for knowledge lifecycle, documentation automation, KIA standard, self-evolving workflow, documentation dependency map, AI coding workflow, and future automation roadmap. This completes the KMS v1.0 lifecycle foundation.

Reason: Ensure every future approved implementation naturally keeps KMS synchronized without relying on uncontrolled app-side rewriting of source docs.

Files changed:

```
1 docs/engineering/KNOWLEDGE_LIFECYCLE.md
1 docs/engineering/DOCUMENTATION_AUTOMATION.md
1 docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md
1 docs/engineering/SELF_EVOLVING_WORKFLOW.md
1 docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md
1 docs/engineering/AI_CODING_WORKFLOW.md
1 docs/engineering/FUTURE_AUTOMATION_ROADMAP.md
1 docs/engineering/README.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/AI_PROJECT_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/README.md
1 docs/CHANGE_HISTORY.md
1 docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3d-lifecycle-foundation.md
```

```
1    scripts/generate_docs_pdf.py
1    static/docs/TIS_Project_Reference_Booklet.pdf
1    static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3D final phase only. SaaS flows, landing page code, database models, migrations, `tis.db`, routes, commits, and pushes remain out of scope.

2026-06-26 - Added KMS v3.0 Phase 3C Governance And AI Traceability

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3B documented database architecture, development standards, UI/UX design philosophy, roadmap, and stronger onboarding guidance. It did not yet preserve rejected decisions, visual documentation standards, a definitive AI optimization guide, project governance, or explicit decision traceability.

New state: TIS now has Phase 3C engineering docs for rejected decisions, visual documentation framework, AI optimization, project governance, and decision traceability. The PDF generator includes these docs.

Reason: Future developers and AI assistants need to understand why TIS became what it is, not only what currently exists.

Files changed:

```
1    docs/engineering/REJECTED_DECISIONS.md
1    docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md
1    docs/engineering/AI_OPTIMIZATION_GUIDE.md
1    docs/engineering/PROJECT_GOVERNANCE.md
1    docs/engineering/README.md
1    docs/TIS_MASTER_CONTEXT.md
1    docs/AI_PROJECT_CONTEXT.md
1    docs/PROJECT_STATE.md
1    docs/README.md
1    docs/CHANGE_HISTORY.md
1    docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3c-governance-ai-traceability.md
1    scripts/generate_docs_pdf.py
1    static/docs/TIS_Project_Reference_Booklet.pdf
1    static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3C only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

2026-06-26 - Added KMS v3.0 Phase 3B Engineering Layers

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3A added module map, repository architecture, user/system flows, and onboarding structure. The handbook still needed database architecture, development standards, UI/UX philosophy, roadmap, and stronger human/AI guidance.

New state: TIS now has Phase 3B engineering docs for database architecture, development standards, UI/UX design philosophy, and product roadmap. Core KMS docs and AI onboarding guidance reference these layers, and the PDF generator includes them.

Reason: Make the generated booklet more useful for new senior developers, Codex conversations, ChatGPT conversations, and future technical reviewers.

Files changed:

```
1 docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
1 docs/engineering/DEVELOPMENT_STANDARDS.md
1 docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md
1 docs/engineering/PRODUCT_ROADMAP.md
1 docs/engineering/README.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/AI_PROJECT_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/README.md
1 docs/CHANGE_HISTORY.md
1 scripts/generate_docs_pdf.py
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3B only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

2026-06-26 - Added KMS v3.0 Engineering Handbook

Area/module: Knowledge Management System and engineering onboarding

Previous state: The generated booklet included KMS source documents, ADRs, module history, AI context, and the Knowledge Center foundation, but it did not fully onboard a new human developer or future Codex/ChatGPT conversation into TIS modules, repository architecture, and end-to-end flows.

New state: TIS now has an engineering handbook layer with a complete module map, repository architecture guide, user/system flow guide, and engineering onboarding index. The PDF generator includes these docs and emits documentation version 3.0.

Reason: Make the generated booklet a true TIS Engineering Handbook rather than only a documentation bundle.

Files changed:

```
1 docs/engineering/README.md
1 docs/engineering/TIS_MODULE_MAP.md
1 docs/engineering/REPOSITORY_ARCHITECTURE.md
1 docs/engineering/USER_AND_SYSTEM_FLOWS.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/AI_PROJECT_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/README.md
1 docs/CHANGE_HISTORY.md
1 scripts/generate_docs_pdf.py
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3A only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

2026-06-26 - Added Platform Owner Knowledge Center

Area/module: Platform Knowledge Center and KMS access

Previous state: TIS had KMS source docs, ADRs, module history, a generated PDF booklet, and a manifest, but no protected in-app owner page for KMS status or booklet access.

New state: TIS now has a read-only Platform Owner Knowledge Center with KMS health score, manifest metadata, freshness detection, source document status, coverage checks, latest change-history entries, ADR list, module history areas, KIA checklist, and protected PDF view/download routes.

Reason: Platform owners need an internal utility for verifying KMS health and accessing the generated PDF without exposing direct public static links.

Files changed:

```
1 knowledge_service.py
1 main.py
1 templates/platform_knowledge_center.html
1 templates/platform_console.html
1 scripts/generate_docs_pdf.py
```

```
1 docs/TIS_MASTER_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/README.md
1 docs/CHANGE_HISTORY.md
1 docs/AI_PROJECT_CONTEXT.md
1 docs/history/platform-knowledge/README.md
1 docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for Phase 2C only. Regenerate button, SaaS changes, database changes, migrations, landing page changes, commits, and pushes remain out of scope.

2026-06-26 - Established Knowledge Management System Foundation

Area/module: Documentation and project knowledge management

Previous state: TIS had Phase 1 documentation source files and a generated PDF booklet, but no formal change history, ADR system, module history foundation, KMS policy, manifest, or compact AI onboarding file.

New state: TIS now has a Knowledge Management System foundation with chronological change history, documentation update policy, ADR structure and initial accepted ADRs, module history folders, AI project context, updated source docs, and an expanded PDF generator.

Reason: Preserve project knowledge for future human developers, Codex conversations, ChatGPT conversations, project owners, platform owners, and technical reviewers.

Files changed:

```
1 docs/CHANGE_HISTORY.md
1 docs/DOCUMENTATION_UPDATE_POLICY.md
1 docs/AI_PROJECT_CONTEXT.md
1 docs/adr/README.md
1 docs/adr/0001-separate-nextjs-landing-website.md
1 docs/adr/0002-separate-saas-identity-and-operational-users.md
1 docs/adr/0003-paddle-payment-architecture.md
1 docs/adr/0004-webhook-only-payment-confirmation.md
1 docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md
1 docs/adr/0006-documentation-as-source-knowledge-management-system.md
1 docs/adr/0007-landing-page-visual-system-strategy.md
1 docs/history/README.md
```

```
1 docs/history/*/README.md
1 docs/history/provisioning/2026-06-26-kms-foundation.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/README.md
1 scripts/generate_docs_pdf.py
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for Phase 2A and Phase 2B only. Platform Owner Knowledge Center, app routes, SaaS flows, database, migrations, landing page implementation, commits, and pushes remain out of scope.

docs/adr/README.md

Architecture Decision Records

ADRs record major TIS architectural and product decisions. They explain why the system is shaped the way it is.

ADR Rules

- 1 Create an ADR for major decisions affecting architecture, identity, SaaS, payment, provisioning, deployment, documentation governance, or long-term product boundaries.
- 1 Do not use ADRs for ordinary bug fixes.
- 1 If a decision is replaced, mark the older ADR as Superseded and link the new ADR.
- 1 ADRs complement docs/TIS_MASTER_CONTEXT.md; they do not replace it.

Status Values

- 1 Proposed
- 1 Accepted
- 1 Superseded
- 1 Deprecated

ADR Index

- 1 0001-separate-nextjs-landing-website.md
- 1 0002-separate-saas-identity-and-operational-users.md
- 1 0003-paddle-payment-architecture.md
- 1 0004-webhook-only-payment-confirmation.md
- 1 0005-delayed-tenant-provisioning-after-verified-payment.md
- 1 0006-documentation-as-source-knowledge-management-system.md
- 1 0007-landing-page-visual-system-strategy.md

docs/engineering/README.md

TIS Engineering Handbook

This folder turns the TIS KMS into a practical engineering handbook. It is intended for new human developers, future Codex conversations, future ChatGPT conversations, project owners, platform owners, and reviewers.

Read First

Recommended onboarding order:

- 1 docs/AI_PROJECT_CONTEXT.md
- 2 docs/TIS_MASTER_CONTEXT.md
- 3 docs/PROJECT_STATE.md
- 4 docs/engineering/TIS_MODULE_MAP.md
- 5 docs/engineering/REPOSITORY_ARCHITECTURE.md
- 6 docs/engineering/USER_AND_SYSTEM_FLOWS.md
- 7 Relevant ADRs under docs/adr/
- 8 Relevant module history under docs/history/

Engineering Documents

- 1 TIS_MODULE_MAP.md: product and system module map with purpose, files, maturity, docs, risks, and guardrails.
- 1 REPOSITORY_ARCHITECTURE.md: repository structure and responsibilities.
- 1 USER_AND_SYSTEM_FLOWS.md: end-to-end public, SaaS, payment, provisioning, operational, platform owner, and KMS flows.
- 1 DATABASE_ARCHITECTURE_OVERVIEW.md: high-level data model relationships and isolation boundaries.
- 1 DEVELOPMENT_STANDARDS.md: non-negotiable engineering rules for humans and AI assistants.
- 1 UI_UX_DESIGN_PHILOSOPHY.md: design principles for operational app, platform owner tools, SaaS onboarding, Knowledge Center, and landing website.
- 1 PRODUCT_ROADMAP.md: completed, current, next, and future roadmap.
- 1 REJECTED_DECISIONS.md: significant rejected architectural/product alternatives and why they were declined.
- 1 VISUAL_DOCUMENTATION_GUIDE.md: future screenshot and diagram standards.
- 1 AI_OPTIMIZATION_GUIDE.md: definitive onboarding and operating guide for future AI assistants.
- 1 PROJECT_GOVERNANCE.md: ownership, approvals, quality gates, documentation gates, and decision traceability.
- 1 KNOWLEDGE_LIFECYCLE.md: documentation, engineering, approval, review, release, and maintenance lifecycle.
- 1 DOCUMENTATION_AUTOMATION.md: current and future documentation automation rules.
- 1 KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md: formal KIA standard with examples, decision tree, and failure cases.
- 1 SELF_EVOLVING_WORKFLOW.md: official task-to-deployment workflow.
- 1 DOCUMENTATION_DEPENDENCY_MAP.md: relationships and propagation rules across KMS documents.

- 1 `AI_CODING_WORKFLOW.md`: required workflow for future AI coding assistants.
- 1 `FUTURE_AUTOMATION_ROADMAP.md`: possible future automation improvements.

Before Coding

Before changing code:

- 1 confirm the approved scope,
- 1 inspect affected files and tests,
- 1 read relevant ADRs,
- 1 read relevant module history,
- 1 preserve tenant isolation,
- 1 preserve identity boundaries,
- 1 avoid touching SaaS, landing, database, or migrations unless explicitly approved.

After Coding

Every implementation must complete the Knowledge Impact Assessment:

Knowledge impact: Yes/No

Docs updated:

Change history updated: Yes/No

ADR needed: Yes/No

Module history updated: Yes/No

PDF regenerated: Yes/No

AI project context updated: Yes/No

Reason if not updated:

If included source docs changed, regenerate:

```
.\.venv\Scripts\python.exe scripts\generate_docs_pdf.py
```

docs/engineering/TIS_MODULE_MAP.md

TIS Module Map

This map describes the known TIS modules, where they live, their maturity, related docs/ADRs, and the guardrails developers must respect.

Platform Owner

Purpose: Own global TIS platform operations, owner/co-owner controls, platform developer accounts, cross-organization oversight, and protected KMS access.

Main files/folders:

```
1    auth.py
1    main.py
1    templates/platform_console.html
1    templates/platform_knowledge_center.html
1    knowledge_service.py
1    permission_registry.py
```

Maturity/status: Implemented for owner identity, platform console, developer/co-owner controls, and read-only Knowledge Center.

Related docs/ADRs:

```
1    docs/TIS_MASTER_CONTEXT.md
1    docs/history/platform-knowledge/
```

Risks/guardrails:

```
1    Do not treat platform developers as owners.
1    Do not expose owner utilities to tenant users.
1    Reuse auth.is_platform_owner(...) and existing owner access helpers.
```

Platform Console

Purpose: Allow platform identities to inspect organizations, switch context, and manage platform owner/developer controls.

Main files/folders:

```
1    main.py
1    templates/platform_console.html
1    ui_shell.py
```

Maturity/status: Implemented and active.

Related docs/ADRs:

```
1    docs/PROJECT_STATE.md
```

Risks/guardrails:

- 1 Context switching must not weaken tenant isolation.
- 1 Owner-only management controls must remain owner-only.

Knowledge Center

Purpose: Show KMS health, source coverage, PDF freshness, ADRs, module history, change history, and KIA policy to platform owners.

Main files/folders:

- 1 knowledge_service.py
- 1 main.py
- 1 templates/platform_knowledge_center.html
- 1 static/docs/docs_manifest.json
- 1 static/docs/TIS_Project_Reference_Booklet.pdf

Maturity/status: Read-only Phase 2C implementation complete. Repository-level impact and freshness enforcement is active; no app regenerate button exists.

Related docs/ADRs:

- 1 docs/adr/0006-documentation-as-source-knowledge-management-system.md
- 1 docs/history/platform-knowledge/

Risks/guardrails:

- 1 Do not link directly to /static/docs/... from UI.
- 1 Do not let the app rewrite Markdown source docs.
- 1 Do not add regenerate behavior without approval.

SaaS Account System

Purpose: Handle public SaaS signup/login/account profile, sessions, security, billing visibility, and onboarding status.

Main files/folders:

- 1 saas/router.py
- 1 saas/service.py
- 1 saas/models.py
- 1 templates/saas/

Maturity/status: M1-M5 foundation implemented.

Related docs/ADRs:

- 1 docs/adr/0002-separate-saas-identity-and-operational-users.md
- 1 docs/history/saas-onboarding/

Risks/guardrails:

- 1 Do not merge SaaS account identity with operational tenant users.
- 1 Do not bypass verification/security/session boundaries.

SaaS Onboarding

Purpose: Collect organization, contact, branch, academic setup, and review data before provisioning.

Main files/folders:

- 1 `saas/router.py`
- 1 `saas/service.py`
- 1 `templates/saas/onboarding_*.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- 1 `docs/history/saas-onboarding/`
- 1 `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- 1 Pending onboarding data is not the same as operational tenant data.
- 1 Do not create live tenant records directly from public forms.

Pending Organizations

Purpose: Represent SaaS organizations waiting for review, payment readiness, or provisioning.

Main files/folders:

- 1 `saas/models.py`
- 1 `saas/service.py`
- 1 `saas/router.py`
- 1 `templates/saas/admin_pending_organizations.html`
- 1 `templates/saas/admin_pending_organization_detail.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- 1 `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`

Risks/guardrails:

- 1 Keep pending state separate from provisioned tenant state.
- 1 Preserve platform owner review and auditability.

Plans And Pricing

Purpose: Define SaaS plan catalog and pricing behavior.

Main files/folders:

- 1 saas/pricing_service.py
- 1 saas/router.py
- 1 saas/entitlement_service.py
- 1 saas/subscription_portal_service.py
- 1 saas/subscription_change_service.py
- 1 saas/subscription_plan_change_service.py
- 1 saas/subscription_cancellation_service.py
- 1 saas/subscription_lifecycle_service.py
- 1 templates/saas/plan_catalog.html
- 1 templates/saas/plan_selection.html
- 1 templates/saas/subscription.html

Maturity/status: M7 implemented: entitlement catalog, customer portal, quantity and plan management, scheduled changes, cancellation/reversal, and centralized lifecycle/action policy.

Related docs/ADRs:

- 1 docs/history/subscriptions/
- 1 docs/adr/0003-paddle-payment-architecture.md

Risks/guardrails:

- 1 Plan changes must update docs and change history.
- 1 Do not hard-code provider behavior outside service boundaries.

Paddle / Payment

Purpose: Integrate subscription checkout, payment state, billing status, and provider events.

Main files/folders:

- 1 saas/paddle_client.py
- 1 saas/payment_service.py
- 1 saas/billing_service.py
- 1 saas/currency_service.py
- 1 saas/router.py
- 1 saas/billing_history_service.py
- 1 saas/payment_lifecycle_reconciliation_service.py

Maturity/status: Implemented through M7, including provider-authoritative previews/proration, billing history, invoice access, webhook idempotency, fail-closed reconciliation, diagnostics, and guarded repair tooling.

Related docs/ADRs:

- 1 docs/adr/0003-paddle-payment-architecture.md
- 1 docs/adr/0004-webhook-only-payment-confirmation.md
- 1 docs/history/subscriptions/

Risks/guardrails:

- 1 Webhook-confirmed payment state is authoritative.
- 1 Checkout return alone must not trigger verified payment/provisioning.
- 1 TIS must not calculate replacement financial outcomes when Paddle preview/transaction data is authoritative.
- 1 Scheduled provider changes must not become local entitlement truth before verified effective evidence.
- 1 Invoice URLs must be freshly resolved and must not be stored.

Provisioning

Purpose: Convert verified/approved pending organizations into operational tenant structures.

Main files/folders:

- 1 `saas/provisioning_service.py`
- 1 `saas/router.py`
- 1 `templates/saas/admin_provisioning.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- 1 `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`
- 1 `docs/history/provisioning/`

Risks/guardrails:

- 1 Keep provisioning recoverable and reviewable.
- 1 Do not merge tenants or create records in the wrong school group.

Operational Login

Purpose: Authenticate operational users and route them into platform or tenant context.

Main files/folders:

- 1 `main.py`
- 1 `auth.py`
- 1 `templates/login.html` if present through root templates

Maturity/status: Implemented.

Related docs/ADRs:

- 1 `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- 1 Platform users and tenant users follow different context behavior.
- 1 Preserve permission and active-user checks.

Dashboard

Purpose: Give tenant users operational visibility into staffing, planning, reports, and current school context.

Main files/folders:

- 1 `main.py`
- 1 `templates/dashboard.html`
- 1 related report/export helpers

Maturity/status: Implemented and evolving.

Related docs/ADRs:

- 1 `docs/history/workforce-planning/`

Risks/guardrails:

- 1 Dashboard data must remain scoped to tenant/branch/year context.

Organizations / School Groups

Purpose: Represent tenant organizations and top-level school group context.

Main files/folders:

- 1 `models.py`
- 1 `main.py`
- 1 `templates/system_configuration_schools.html`

Maturity/status: Implemented.

Related docs/ADRs:

- 1 `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- 1 Tenant isolation starts at school group boundaries.

Branches

Purpose: Represent campuses/branches inside a school group.

Main files/folders:

- 1 `models.py`
- 1 `main.py`
- 1 `templates/system_configuration_branches.html`

Maturity/status: Implemented.

Related docs/ADRs:

- 1 `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- 1 Branch scope affects teachers, users, planning, timetable, calendar, branding, and reports.

Academic Years

Purpose: Scope operational data by academic year and active-year context.

Main files/folders:

- 1 `models.py`
- 1 `main.py`
- 1 `templates/system_configuration_years.html`

Maturity/status: Implemented.

Related docs/ADRs:

- 1 `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- 1 Do not mix data across academic years.

Users And Roles

Purpose: Manage tenant users, platform users, roles, profile data, active status, and branch/year context.

Main files/folders:

- 1 `auth.py`
- 1 `routers/users.py`
- 1 `models.py`
- 1 `templates/users.html`
- 1 `templates/edit_user.html`

Maturity/status: Implemented with platform identity refinements.

Related docs/ADRs:

- 1 `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- 1 Do not assign owner controls to developers or tenant users.
- 1 Preserve role normalization.

Permissions

Purpose: Control route/module actions by permission key and role package.

Main files/folders:

- 1 `authorization.py`
- 1 `permission_registry.py`

```
1    role_permission_service.py
1    auth.py
1    templates/system_configuration_role_permissions.html
```

Maturity/status: Implemented and critical.

Related docs/ADRs:

```
1    docs/TIS_MASTER_CONTEXT.md
```

Risks/guardrails:

```
1    Do not bypass authorization.enforce_route_permission.
1    Do not rely only on UI hiding for protected actions.
```

Teachers

Purpose: Manage teacher records, qualifications, capacity, workloads, and profile-related academic staffing data.

Main files/folders:

```
1    routers/teachers.py
1    teacher_qualifications.py
1    teacher_capacity.py
1    models.py
1    templates/teachers.html
1    templates/edit_teacher.html
```

Maturity/status: Implemented and central to operational value.

Related docs/ADRs:

```
1    docs/history/workforce-planning/
```

Risks/guardrails:

```
1    Teacher data must remain tenant/branch/year scoped.
```

Subjects

Purpose: Manage subject catalog, colors, qualifications, and planning/timetable relationships.

Main files/folders:

```
1    routers/subjects.py
1    subject_colors.py
1    models.py
1    templates/subjects.html
1    templates/edit_subject.html
```

Maturity/status: Implemented.

Related docs/ADRs:

1 docs/history/workforce-planning/

Risks/guardrails:

1 Subject changes can affect planning, timetable, teacher qualifications, and reports.

Sections

Purpose: Represent class/section structures used by planning and timetabling.

Main files/folders:

1 routers/planning.py

1 models.py

1 templates/planning.html

Maturity/status: Implemented through planning workflows.

Related docs/ADRs:

1 docs/history/workforce-planning/

Risks/guardrails:

1 Section changes can affect timetable allocation and workload reporting.

Workforce Planning

Purpose: Plan assignments, homeroom ownership, workloads, subject coverage, and staffing needs.

Main files/folders:

1 routers/planning.py

1 teacher_capacity.py

1 homeroom_defaults.py

1 templates/planning.html

Maturity/status: Implemented and evolving.

Related docs/ADRs:

1 docs/history/workforce-planning/

Risks/guardrails:

1 Planning logic must preserve branch/year scope and avoid accidental cross-year copies.

Timetable

Purpose: Place planned lessons into weekly timetable grids and exports.

Main files/folders:

1 routers/timetable.py

```
1    timetable_logic.py
1    templates/timetable.html
1    templates/system_configuration_timetable.html
```

Maturity/status: Implemented and evolving.

Related docs/ADRs:

```
1    docs/history/workforce-planning/
```

Risks/guardrails:

```
1    Timetable rules interact with planning, sections, subjects, and teacher capacity.
```

Academic Calendar

Purpose: Manage academic events, responsibilities, dates, exports, and branch/year scoped calendar views.

Main files/folders:

```
1    routers/academic_calendar.py
1    templates/academic_calendar.html
1    templates/system_configuration_calendar.html
```

Maturity/status: Implemented.

Related docs/ADRs:

```
1    docs/history/academic-calendar/
```

Risks/guardrails:

```
1    Calendar permissions and branch/year scoping must remain intact.
```

Observations

Purpose: Support teacher observations, feedback, evidence, scoring, history, and supervision records.

Main files/folders:

```
1    routers/observations.py
1    templates/observations.html
1    templates/observation_form.html
1    templates/observation_detail.html
1    templates/observation_history.html
```

Maturity/status: Implemented.

Related docs/ADRs:

```
1    docs/TIS_MASTER_CONTEXT.md
```

Risks/guardrails:

```
1    Observation records are sensitive and must remain tenant scoped.
```


Reports / Dashboards

Purpose: Expose operational summaries, exports, allocation reports, and decision visibility.

Main files/folders:

- 1 `main.py`
- 1 `report/export helper code`
- 1 `templates/dashboard.html`

Maturity/status: Implemented and evolving.

Related docs/ADRs:

- 1 `docs/history/workforce-planning/`

Risks/guardrails:

- 1 Report data must be permission-checked and tenant scoped.

Branding / Design Settings

Purpose: Manage organization logos, design settings, visual shell behavior, and platform visual controls.

Main files/folders:

- 1 `branding_storage.py`
- 1 `design_tokens.py`
- 1 `visual_design.py`
- 1 `ui_shell.py`
- 1 `static/css/branding.css`
- 1 `static/css/design-studio.css`
- 1 `templates/system_configuration_logos.html`
- 1 `templates/system_configuration_design.html`

Maturity/status: Implemented.

Related docs/ADRs:

- 1 `docs/adr/0007-landing-page-visual-system-strategy.md`

Risks/guardrails:

- 1 Do not confuse operational app branding with public landing website design.
- 1 Protect uploaded/owned assets.

Landing Website

Purpose: Public marketing and conversion surface for TIS.

Main files/folders:

- 1 tis-landing-website/
- 1 docs/marketing/

Maturity/status: Implemented as separate Next.js app.

Related docs/ADRs:

- 1 docs/adr/0001-separate-nextjs-landing-website.md
- 1 docs/adr/0007-landing-page-visual-system-strategy.md
- 1 docs/marketing/landing_page_source_of_truth.md

Risks/guardrails:

- 1 Do not modify landing code during operational/backend tasks unless explicitly approved.
- 1 Legacy FastAPI landing files are not the public source of truth.

AI Future Roadmap

Purpose: Future subscription-gated AI-assisted planning, analytics, assessment generation, recommendations, and decision support.

Main files/folders:

- 1 No dedicated AI production module yet.
- 1 Future work should be documented before implementation.

Maturity/status: Roadmap / future.

Related docs/ADRs:

- 1 docs/TIS_MASTER_CONTEXT.md
- 1 future ADRs required before major AI architecture decisions.

Risks/guardrails:

- 1 AI features must use verified tenant data and preserve privacy, permissions, and subscription boundaries.

TIS Repository Architecture

This document explains the main repository areas, what each owns, and what must not be changed casually.

Root FastAPI Application

`main.py`

Responsibility: Primary FastAPI app, route registration, many app-level workflows, middleware, startup checks, platform console routes, dashboards, exports, system configuration, scope switching, and operational pages.

Do not change casually:

- 1 authentication/session flow,
- 1 platform owner access checks,
- 1 tenant scope handling,
- 1 route behavior shared by many modules,
- 1 startup/schema repair behavior.

`auth.py`

Responsibility: Password hashing/verification, session cookies, current-user lookup, role normalization, platform identity helpers, permission helpers, email verification helpers, and tenant/platform identity boundaries.

Do not change casually:

- 1 `is_platform_owner`, `is_platform_user`, `is_platform_developer`,
- 1 role normalization,
- 1 session cookie behavior,
- 1 permission helper semantics.

`authorization.py`

Responsibility: Protected route rules, permission enforcement middleware integration, access-denied responses.

Do not change casually:

- 1 route permission mapping,
- 1 public path patterns,
- 1 permission matching semantics.

`database.py`

Responsibility: Database engine/session setup.

Do not change casually:

- 1 database URL handling,
- 1 session creation behavior,

- 1 engine configuration.

`models.py`

Responsibility: Primary operational SQLAlchemy models for users, school groups, branches, teachers, planning, timetable, observations, configuration, and related app data.

Do not change casually:

- 1 tenant ownership fields,
- 1 platform user fields,
- 1 relationships used by existing workflows,
- 1 schema without matching migration/repair strategy.

`db_migrations.py`

Responsibility: Local schema migration and repair logic used by the app.

Do not change casually:

- 1 migration ordering,
- 1 destructive schema operations,
- 1 production-sensitive repair behavior.

`permission_registry.py`

Responsibility: Permission groups, labels, default role permissions, system owner permissions, developer-assignable permission boundaries.

Do not change casually:

- 1 owner/developer permissions,
- 1 defaults for managed roles,
- 1 permission keys referenced by routes/templates/tests.

`role_permission_service.py`

Responsibility: Persistence and service helpers for role permission rows.

Do not change casually:

- 1 global vs school-scoped permission behavior,
- 1 role permission constraints.

`ui_shell.py`

Responsibility: Shared application shell, navigation, page metadata, scoped organization/year context display, logos, visual design CSS, and permission-based nav generation.

Do not change casually:

- 1 navigation visibility,
- 1 platform-vs-tenant shell behavior,

- 1 design-studio gating,
- 1 scope display.

Feature Routers: routers/

Responsibility: Modular operational route handlers:

- 1 routers/users.py
- 1 routers/teachers.py
- 1 routers/subjects.py
- 1 routers/planning.py
- 1 routers/timetable.py
- 1 routers/academic_calendar.py
- 1 routers/observations.py

Do not change casually:

- 1 tenant/branch/year scoping,
- 1 permission checks,
- 1 import/export behavior,
- 1 bulk operations.

SaaS Package: saas/

Responsibility: Public SaaS account flows, onboarding, plans, billing, Paddle integration, pending organizations, and provisioning.

Key files:

- 1 saas/router.py
- 1 saas/service.py
- 1 saas/models.py
- 1 saas/pricing_service.py
- 1 saas/payment_service.py
- 1 saas/paddle_client.py
- 1 saas/billing_service.py
- 1 saas/provisioning_service.py
- 1 saas/currency_service.py
- 1 saas/oauth.py
- 1 saas/entitlement_service.py
- 1 saas/subscription_portal_service.py
- 1 saas/subscription_change_service.py
- 1 saas/subscription_plan_change_service.py

- 1 `saas/subscription_cancellation_service.py`
- 1 `saas/subscription_lifecycle_service.py`
- 1 `saas/billing_history_service.py`
- 1 `saas/payment_lifecycle_reconciliation_service.py`

Do not change casually:

- 1 identity separation,
- 1 payment confirmation rules,
- 1 provisioning readiness,
- 1 Paddle/webhook boundaries,
- 1 public signup/onboarding state transitions.

Templates: `templates/`

Responsibility: Jinja templates for operational app, platform console, Knowledge Center, SaaS pages, system configuration, and feature workflows.

Do not change casually:

- 1 form action routes,
- 1 hidden scope fields,
- 1 permission-dependent controls,
- 1 app shell extension patterns.

Static Assets: `static/`

Responsibility: Operational CSS/JS, images, branding assets, generated documentation PDF and manifest.

Important:

- 1 `static/docs/TIS_Project_Reference_Booklet.pdf` is a generated snapshot.
- 1 `static/docs/docs_manifest.json` is generated metadata.

Do not change casually:

- 1 generated docs manually,
- 1 shared CSS without checking all templates,
- 1 protected document access assumptions.

Tests: `tests/`

Responsibility: Regression coverage for SaaS phases, tenant isolation, platform access, permissions, email, branding, and critical workflows.

Do not change casually:

- 1 tests should follow behavior, not hide regressions.
- 1 update tests when behavior intentionally changes.

Docs: docs/

Responsibility: KMS source of truth, engineering handbook, ADRs, change history, module history, AI context, marketing docs.

Do not change casually:

- 1 historical records should preserve old/new state.
- 1 update docs through the KIA process.

Scripts: scripts/

Responsibility: Maintenance, diagnostics, and governance scripts. Important examples:

- 1 `scripts/generate_docs_pdf.py`: generate or read-only validate KMS PDF/manifest artifacts.
- 1 `scripts/check_kms_impact.py`: compare KIA declarations, Git changes, major-path classification, and artifact freshness.
- 1 `scripts/kms.py`: supported Phase 6 command surface for one-step synchronization and complete read-only validation; delegates to the generator and checker.
- 1 `scripts/sync_paddle_price_ids.py`: environment-specific initial checkout price mapping.
- 1 `scripts/diagnose_paddle_plan_preview.py` and `scripts/diagnose_payment_lifecycle.py`: safe subscription/payment diagnostics.
- 1 `scripts/reconcile_finalized_payment_lifecycle.py`: guarded sandbox reconciliation from attributable provider evidence.

Do not change casually:

- 1 PDF generator dependency assumptions,
- 1 source list behavior,
- 1 manifest metadata.

Repository Governance

Responsibility: Make KMS impact visible and enforceable without rewriting documentation.

Key files:

- 1 `AGENTS.md`
- 1 `.kms-impact.yml`
- 1 `.github/pull_request_template.md`
- 1 `.github/workflows/kms-enforcement.yml`
- 1 `.github/workflows/deploy-on-master.yml`

Do not change casually:

- 1 major-path classification,
- 1 explicit no-impact override requirements,
- 1 authoritative Markdown/path rules,
- 1 deployment dependency on KMS validation,

- 1 prohibition on customer/runtime/secrets data in documentation.

Next.js Landing Website: `tis-landing-website/`

Responsibility: Public marketing website at <https://tisplatform.com>.

Do not change casually:

- 1 landing implementation during backend tasks,
- 1 visual system without approval,
- 1 public assets/copy without checking marketing docs and ADRs.

TIS User And System Flows

This document describes the major end-to-end flows a developer must understand before changing TIS.

Public Customer Flow

Flow:

- 1 Public visitor opens <https://tisplatform.com>.
- 2 Visitor clicks a signup/get-started path.
- 3 Visitor reaches SaaS signup at </saas/signup>.
- 4 SaaS account is created.
- 5 Email verification is completed when required.
- 6 User signs into SaaS account through </saas/login>.
- 7 User enters </saas/account>.
- 8 User completes organization onboarding:
 - 1 organization details,
 - 1 contacts,
 - 1 branches,
 - 1 academic setup,
 - 1 review.
- 1 User selects a plan.
- 2 User enters checkout.
- 3 Paddle handles payment.
- 4 Return/cancel page informs the user of checkout navigation result.
- 5 Paddle webhook confirms payment.
- 6 Local payment/billing state is updated.
- 7 Pending organization becomes ready for provisioning.
- 8 Platform owner reviews/runs provisioning.
- 9 Operational tenant structures are created.
- 10 Operational login becomes available through </login>.

Guardrails:

- 1 Checkout return is not authoritative payment confirmation.
- 1 Public signup must not directly create operational tenant data.
- 1 Provisioning should occur only through the approved flow.

SaaS Identity Flow

Flow:

- 1 User signs up through /saas/signup.
- 2 SaaS account/session records are created.
- 3 User signs in through /saas/login.
- 4 Account dashboard is available at /saas/account.
- 5 User can access profile, sessions, security, billing status, and onboarding state.

Important distinction:

- 1 SaaS account identity is not the same as operational tenant user identity.
- 1 Platform identity is also separate.

Guardrails:

- 1 Do not merge SaaS accounts with operational users unless an approved provisioning flow creates the needed operational records.
- 1 Keep SaaS authentication and operational authentication boundaries clear.

Payment Flow

Flow:

- 1 User selects a plan.
- 2 App creates or references a checkout session.
- 3 User goes to Paddle checkout.
- 4 User returns through checkout return or cancel pages.
- 5 Paddle sends webhook events.
- 6 Webhook-confirmed payment updates local payment/billing state.
- 7 Verified payment can make a pending organization ready for provisioning.

Guardrails:

- 1 Webhook confirmation is authoritative.
- 1 Do not use return-page navigation as proof of payment.
- 1 Keep Paddle-specific details inside saas/ payment/client service boundaries.

Active Subscription Management Flow

- 1 Authorized billing administrator opens /saas/subscription.
- 2 TIS resolves one confirmed active subscription, entitlements, lifecycle state, paid/active branch capacity, and allowed actions.
- 3 Quantity or plan changes are previewed through Paddle; TIS displays provider-returned totals and never recalculates proration.

- 4 Immediate increases/upgrades use provider payment-failure prevention and remain locally pending until authoritative confirmation.
- 5 Reductions/downgrades are scheduled for the next billing boundary and retain current local access until verified effective evidence.
- 6 Scheduled plan or quantity changes may be canceled or replaced before their effective boundary when provider state agrees.
- 7 Cancellation is scheduled at period end; reversal removes the provider-scheduled cancellation after reauthorization and validation.
- 8 The centralized lifecycle resolver exposes only actions valid for current provider/local state.
- 9 Billing history is read from Paddle transactions. Invoice download reauthorizes the user and requests a fresh provider URL.

Guardrails:

- 1 provider and local ownership must match,
- 1 active branch usage cannot exceed a requested reduced capacity,
- 1 webhook processing is idempotent,
- 1 ambiguous outcomes enter manual review,
- 1 return pages and local requests are not payment confirmation.

Provisioning Flow

Flow:

- 1 Pending organization has completed required onboarding.
- 2 Payment is verified or owner-approved readiness is satisfied.
- 3 Provisioning job is queued or run.
- 4 Operational records are created or connected:
 - 1 school group,
 - 1 branch,
 - 1 academic year,
 - 1 initial operational user,
 - 1 permissions/role context,
 - 1 required setup defaults.
- 1 Provisioning status is updated.
- 2 Activation/access email may be sent.
- 3 User enters operational portal through /login.

Guardrails:

- 1 Keep provisioning idempotent where possible.
- 1 Do not mix school groups.

- 1 Do not skip platform owner visibility.

Operational Login Flow

Flow:

- 1 User opens /login.
- 2 Credentials are verified.
- 3 Active-user status is checked.
- 4 Platform users route toward platform context.
- 5 Tenant users receive branch/year scope.
- 6 Session and scope cookies are set.
- 7 User lands on /platform or /dashboard.
- 8 Middleware enforces route permissions.

Guardrails:

- 1 Preserve platform vs tenant branching.
- 1 Preserve idle timeout behavior for tenant users.
- 1 Do not bypass permission middleware.

Platform Owner Flow

Flow:

- 1 Platform owner logs in through /login.
- 2 Owner lands in platform context.
- 3 Owner uses /platform for organization context and owner/developer controls.
- 4 Owner uses SaaS admin pages for pending organizations, payments, and provisioning.
- 5 Owner uses /platform/knowledge to review KMS health.
- 6 Owner views/downloads the PDF through protected routes:
 - 1 /platform/knowledge/booklet
 - 1 /platform/knowledge/booklet/download

Guardrails:

- 1 Platform developers are not owners.
- 1 Owner-only pages must use existing owner access helpers.
- 1 Do not expose KMS PDF through direct static links.

Knowledge Management Flow

Flow:

- 1 Developer or Codex reads docs/AI_PROJECT_CONTEXT.md.

- 2 Developer reads master context, project state, engineering docs, relevant ADRs, and module history.
- 3 Approved change is implemented.
- 4 `.kms-impact.yml` and the human-readable Knowledge Impact Assessment are completed.
- 5 Relevant docs are updated:
 - 1 master context,
 - 1 project state,
 - 1 change history,
 - 1 ADRs if needed,
 - 1 module history if needed,
 - 1 AI project context if needed,
 - 1 engineering docs if architecture/module/flow understanding changed.
- 1 PDF generator runs.
- 2 PDF snapshot and manifest are regenerated.
- 3 Knowledge Center checks manifest freshness and health.
- 4 CI compares the declaration with changed files and blocks stale pull requests, dev integration, or master deployment.

Guardrails:

- 1 Markdown remains source of truth.
- 1 PDF is generated and must not be edited manually.
- 1 App must not silently rewrite source docs.
- 1 Regenerate button is not implemented yet.

Human / AI Developer Onboarding Flow

Flow:

- 1 Read docs/AI_PROJECT_CONTEXT.md.
- 2 Read docs/README.md.
- 3 Read docs/TIS_MASTER_CONTEXT.md.
- 4 Read docs/PROJECT_STATE.md.
- 5 Read docs/engineering/TIS_MODULE_MAP.md.
- 6 Read docs/engineering/REPOSITORY_ARCHITECTURE.md.
- 7 Read docs/engineering/USER_AND_SYSTEM_FLOWS.md.
- 8 Read relevant ADRs and module history.
- 9 Inspect code with `rg` before editing.
- 10 Make scoped changes only.
- 11 Update KMS docs and regenerate the PDF if needed.

12 Report the KIA.

Before coding, inspect:

- 1 affected routes,
- 1 models and scope fields,
- 1 permission rules,
- 1 templates/forms,
- 1 tests for the touched module,
- 1 related docs/ADRs/history.

After coding, update:

- 1 docs/CHANGE_HISTORY.md for meaningful changes,
- 1 module history for area-specific changes,
- 1 ADRs for major decisions,
- 1 engineering docs when module maps, architecture, or flows change,
- 1 AI context when onboarding truth changes,
- 1 project state when priority/status changes.

docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md

TIS Database Architecture Overview

This document explains the conceptual TIS data model. It is intentionally high level and does not list every field. Use `models.py`, `saas/models.py`, and tests for exact implementation details.

Core Boundary Principle

TIS has three identity/data worlds that must never be casually mixed:

- 1 Platform data: platform owners, co-owners, developers, platform permissions, and cross-organization oversight.
- 1 SaaS account data: public signup, onboarding, billing, pending organizations, and payment/provisioning readiness.
- 1 Operational tenant data: provisioned school groups, branches, academic years, users, teachers, subjects, planning, timetable, calendar, and observations.

The safest mental model:

Platform Owner oversees many organizations.

SaaS account prepares an organization for subscription/provisioning.

Operational tenant data belongs to one provisioned school group/branch/year context.

Platform Identities

Represents: Platform Owner, Co-Owner, and Platform Developer identities.

Ownership boundary: Platform identities can operate outside ordinary tenant scope only through approved platform workflows.

Must never be mixed:

- 1 Platform Developer must not become Platform Owner through permission drift.
- 1 Platform users must not be treated as normal tenant users unless an explicit context workflow establishes scope.

Related files:

- 1 `auth.py`
- 1 `models.py`
- 1 `permission_registry.py`
- 1 `main.py`

SaaS Accounts

Represents: Public SaaS signup/login/account identities used before or alongside operational tenant access.

Ownership boundary: SaaS accounts own onboarding and billing/account state, not operational school records directly.

Must never be mixed:

- 1 SaaS account identity is not operational user identity.
- 1 SaaS account creation must not directly create live tenant data.

Related files:

- 1 `saas/models.py`
- 1 `saas/service.py`
- 1 `saas/router.py`
- 1 `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Pending Organizations

Represents: An organization moving through SaaS onboarding before operational provisioning.

Ownership boundary: Pending organization data belongs to the SaaS onboarding/provisioning pipeline.

Must never be mixed:

- 1 Pending organization data is not the live school group until provisioning creates or connects operational records.

Related files:

- 1 `saas/models.py`
- 1 `saas/service.py`
- 1 `saas/provisioning_service.py`

Payment Records

Represents: Payment, billing, checkout, and provider-confirmed subscription state.

Ownership boundary: Payment state belongs to SaaS billing/subscription logic and should not be embedded directly into academic modules.

Must never be mixed:

- 1 Checkout return navigation must not be treated as verified payment.
- 1 Payment/provider details must not leak into operational teacher/planning/calendar logic.

Related files:

- 1 `saas/payment_service.py`
- 1 `saas/billing_service.py`
- 1 `saas/paddle_client.py`
- 1 `saas/entitlement_service.py`
- 1 `saas/subscription_change_service.py`
- 1 `saas/subscription_plan_change_service.py`
- 1 `saas/subscription_cancellation_service.py`
- 1 `saas/subscription_lifecycle_service.py`
- 1 `saas/billing_history_service.py`
- 1 `docs/adr/0003-paddle-payment-architecture.md`
- 1 `docs/adr/0004-webhook-only-payment-confirmation.md`

Entitlement And Subscription-Change Records

EntitlementDefinition defines commercial capability keys and value types. PlanEntitlement associates reviewed values with subscription plans. Runtime entitlement resolution starts from the provisioned school group, paid operational contract, and one confirmed active PaymentSubscription; it does not trust onboarding selections, page values, or pending checkout attempts.

SubscriptionChangeRequest is durable workflow/audit state for branch quantity, plan transition, and cancellation actions. It records requested/provider-observed state and lifecycle outcomes, but Paddle remains authoritative for monetary previews, proration, scheduled changes, transactions, and invoice documents.

Guardrails:

- 1 PaymentSubscription.quantity is paid branch-capacity authority.
- 1 unresolved ownership, duplicate active relationships, provider mismatches, or incomplete evidence fail closed.
- 1 scheduled changes do not update effective local entitlements before verified provider/webhook evidence.
- 1 billing history is retrieved from Paddle and is not copied into a new local financial ledger.
- 1 invoice URLs are requested fresh and are not persisted.

Provisioning Jobs

Represents: Controlled work that turns a ready pending organization into operational school structures.

Ownership boundary: Provisioning bridges SaaS data and operational tenant data, but only through explicit platform-owner-visible workflows.

Must never be mixed:

- 1 Do not directly create operational tenant records from public signup or checkout return pages.
- 1 Do not create records under the wrong school group.

Related files:

- 1 `saas/provisioning_service.py`
- 1 `saas/router.py`
- 1 `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`

School Groups / Organizations

Represents: The top-level operational tenant boundary.

Ownership boundary: Most tenant operational data belongs to a school group directly or through branch/year relationships.

Must never be mixed:

- 1 Data from one school group must not appear in another school group's operational workflows.

Related files:

- 1 `models.py`
- 1 `main.py`

Branches

Represents: Campuses or branches inside a school group.

Ownership boundary: Branches scope users, teachers, planning, timetable, academic calendar, branding, and reports.

Must never be mixed:

- 1 Branch-scoped data must not silently cross campuses.
- 1 Platform context switching must remain explicit.

Related files:

- 1 `models.py`
- 1 `main.py`
- 1 `ui_shell.py`

Academic Years

Represents: The academic period used to scope operational records.

Ownership boundary: Planning, timetable, subjects, calendar, and related data may depend on active academic year.

Must never be mixed:

- 1 Do not copy or read current-year data into another year unless the workflow explicitly does so.

Related files:

- 1 `models.py`
- 1 `main.py`
- 1 `year_copy.py`

Operational Users

Represents: Users who work inside an operational school group/branch/year context.

Ownership boundary: Operational users belong to tenant structures and are governed by roles, permissions, branch scope, and active status.

Must never be mixed:

- 1 Operational users are not SaaS accounts by default.
- 1 Tenant users should not gain platform owner access.

Related files:

- 1 `models.py`
- 1 `auth.py`
- 1 `routers/users.py`

Roles And Permissions

Represents: Role packages, permission keys, platform developer permissions, and route/action authorization.

Ownership boundary: Permissions decide what a user can view or change in the current scope.

Must never be mixed:

- 1 UI hiding is not enough; protected actions need route/service checks.
- 1 Owner controls must remain outside developer-assignable permission drift.

Related files:

- 1 `permission_registry.py`
- 1 `role_permission_service.py`
- 1 `authorization.py`
- 1 `auth.py`

Teachers

Represents: Teacher records, qualifications, capacity, workload, and staffing-relevant academic data.

Ownership boundary: Teacher records are tenant/branch/year-sensitive operational data.

Must never be mixed:

- 1 Teacher data from one branch or school group must not appear in another tenant's planning/reporting.

Related files:

- 1 `routers/teachers.py`
- 1 `teacher_qualifications.py`
- 1 `teacher_capacity.py`
- 1 `models.py`

Subjects

Represents: Subject catalog, colors, requirements, qualification relationships, planning and timetable dependencies.

Ownership boundary: Subjects are academic configuration data inside tenant/year context.

Must never be mixed:

- 1 Subject changes can affect planning, timetable, teacher matching, and reports; update all affected docs/tests.

Related files:

- 1 `routers/subjects.py`
- 1 `subject_colors.py`
- 1 `models.py`

Sections / Classes

Represents: Class sections used by planning and timetabling.

Ownership boundary: Sections belong to operational branch/year structures.

Must never be mixed:

- 1 Section structures should not cross branch/year boundaries accidentally.

Related files:

- 1 routers/planning.py
- 1 models.py

Workforce Planning

Represents: Teacher assignments, homeroom ownership, capacity, workload, subject coverage, and staffing needs.

Ownership boundary: Planning is operational data tied to school group, branch, academic year, teachers, subjects, and sections.

Must never be mixed:

- 1 Planning changes can affect dashboards, reports, timetable, and staffing decisions.

Related files:

- 1 routers/planning.py
- 1 teacher_capacity.py
- 1 homeroom_defaults.py

Timetable

Represents: Weekly lesson placement and timetable settings.

Ownership boundary: Timetable data depends on planning, teacher, subject, section, branch, and year context.

Must never be mixed:

- 1 Timetable edits must preserve scheduling constraints and scope.

Related files:

- 1 routers/timetable.py
- 1 timetable_logic.py

Academic Calendar

Represents: Events, academic dates, responsibilities, exports, and calendar settings.

Ownership boundary: Calendar records are branch/year/tenant scoped.

Must never be mixed:

- 1 Calendar events for one tenant or branch must not appear in another.

Related files:

- 1 routers/academic_calendar.py
- 1 templates/academic_calendar.html

Observations

Represents: Teacher observation records, feedback, scoring, evidence, and history.

Ownership boundary: Observation data is sensitive tenant operational data.

Must never be mixed:

- 1 Observation records must remain tenant-scoped and permission-protected.

Related files:

- 1 routers/observations.py
- 1 templates/observation_*.html

Knowledge Management System Artifacts

Represents: Markdown docs, ADRs, module history, generated PDF, manifest, and Knowledge Center read-only status.

Ownership boundary: Markdown under docs/ is source of truth. Generated artifacts under static/docs/ are snapshots.

Must never be mixed:

- 1 The app must not silently rewrite Markdown source docs.
- 1 The PDF must not be edited manually.
- 1 Protected documentation access should go through owner-only routes.

Related files:

- 1 docs/
- 1 scripts/generate_docs_pdf.py
- 1 static/docs/
- 1 knowledge_service.py

Tenant Isolation Rules

- 1 Always know the current school group, branch, and academic year.
- 1 Do not assume platform users have tenant scope.
- 1 Do not assume SaaS accounts have operational records.
- 1 Keep pending SaaS organization state separate from provisioned operational data.
- 1 Preserve route permissions and service-level checks.
- 1 Tests touching cross-tenant data should be treated as high value.

TIS Development Standards

These standards are mandatory for future human developers, Codex conversations, ChatGPT conversations, and technical reviewers working on TIS.

Inspect Before Editing

Before changing files:

- 1 inspect the current code with `rg` and targeted file reads,
- 1 understand the affected route/service/template/model,
- 1 read relevant docs, ADRs, and module history,
- 1 check tests for the touched module,
- 1 check the current branch and working tree.

Do not code from memory when local context is available.

Plan Before Implementation

For non-trivial work:

- 1 identify the affected modules,
- 1 identify tenant/permission/identity boundaries,
- 1 identify documentation updates,
- 1 identify tests or smoke checks,
- 1 keep the implementation path small.

Planning does not need to be long, but the risk boundaries must be clear.

Keep Changes Small And Scoped

- 1 Implement only the approved request.
- 1 Avoid unrelated refactors.
- 1 Avoid formatting churn in unrelated files.
- 1 Preserve existing patterns unless there is a clear reason to change them.
- 1 Do not move code across modules as a side effect.

Preserve Tenant Isolation

Tenant isolation is non-negotiable.

Always protect:

- 1 school group boundaries,
- 1 branch boundaries,
- 1 academic year scope,

- 1 user role/permission scope,
- 1 observation and teacher data sensitivity.

Any change that touches tenant scope should be treated as high risk.

Preserve Login And Platform Owner Flows

Do not casually change:

- 1 /login,
- 1 session cookies,
- 1 platform owner detection,
- 1 platform developer permissions,
- 1 /platform,
- 1 /platform/knowledge,
- 1 scope switching,
- 1 access denied handling.

Platform developers are not platform owners unless existing owner logic explicitly treats them that way.

Protect SaaS, Payment, And Provisioning

Do not casually change:

- 1 /saas/signup,
- 1 /saas/login,
- 1 /saas/account,
- 1 SaaS onboarding steps,
- 1 Paddle checkout/client code,
- 1 payment confirmation logic,
- 1 billing state,
- 1 provisioning readiness,
- 1 provisioning jobs.

Webhook-confirmed payment is authoritative. Checkout return pages are not proof of payment.

Database And Migration Rules

- 1 Never modify `tis.db` unless explicitly approved.
- 1 Never modify migrations casually.
- 1 Do not change `models.py` without understanding migration/repair implications.
- 1 Do not add schema changes without tests and documentation.
- 1 Never use destructive database operations without explicit approval.

Landing Website Rule

The public landing website is separate:

- 1 implementation: tis-landing-website/,
- 1 marketing docs: docs/marketing/,
- 1 ADRs: 0001, 0007.

Do not modify landing page code during backend or operational app tasks unless explicitly approved.

Protected Documentation Rule

- 1 Do not expose generated PDFs through direct public UI links.
- 1 Use owner-protected routes for PDF access.
- 1 Markdown source docs are reviewed development artifacts.
- 1 The app must not silently rewrite Markdown source docs.

Production Memory And Render Stability

Render memory is a hard production budget. Treat a 512 MB service as constrained and design routes, services, and assets accordingly.

Strict rules:

- 1 Do not load, parse, or cache complete large datasets at startup or on a normal first request when a scoped lookup, streaming parser, pagination, or bounded cache can be used.
- 1 Do not keep both raw and transformed copies of large JSON, CSV, Excel, PDF, image, or uploaded payloads in memory unless explicitly justified and validated.
- 1 Do not run duplicate template renders in production request paths. Diagnostic pre-renders must be removed or gated behind an explicit environment flag.
- 1 Do not emit stage-by-stage debug logs at warning level during normal traffic. Production diagnostics must be opt-in and should use appropriate log levels.
- 1 Filter by tenant, school group, branch, academic year, and permission scope before materializing query results.
- 1 Keep caches bounded by size or scope. Global caches for user-facing lookup data must be justified by measured memory impact.
- 1 Large exports should stream or build bounded artifacts; avoid repeatedly constructing large in-memory workbooks, PDFs, or payloads inside loops.
- 1 New static assets, videos, generated files, and location/reference datasets must be reviewed for repository size, runtime memory, and deployment impact.
- 1 Any feature that adds broad data aggregation, new startup work, large reference files, or route-level diagnostics must include a memory/performance smoke check before deployment.

Minimum validation for memory-sensitive changes:

- 1 run targeted tests for the touched module,
- 1 run a compile/import check for changed Python modules,
- 1 measure peak memory locally when a route or service parses large data,

- 1 review production logs after deployment for restarts, OOM symptoms, and unexpected warning spam.

Commit And Push Rule

- 1 Do not commit unless explicitly requested.
- 1 Do not push unless explicitly requested.
- 1 Do not merge unless explicitly requested.
- 1 Final reports should describe changes and validation clearly.

KMS Update Rule

Every meaningful implementation must update KMS if affected:

- 1 docs/TIS_MASTER_CONTEXT.md,
- 1 docs/PROJECT_STATE.md,
- 1 docs/CHANGE_HISTORY.md,
- 1 docs/AI_PROJECT_CONTEXT.md,
- 1 ADRs if major decisions changed,
- 1 module history for module-specific before/after state,
- 1 engineering docs if module maps, architecture, data model, standards, design philosophy, roadmap, or flows changed.

Regenerate the PDF if included docs changed.

Knowledge Impact Assessment

Every final implementation report must include:

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

A task is not complete until KIA is assessed.

Validation Standards

Run the narrowest meaningful validation:

- 1 compile checks for Python scripts/modules,
- 1 targeted pytest tests when behavior changes,
- 1 route/template smoke checks when pages are touched,
- 1 PDF generation when docs change,
- 1 frontend checks when landing/frontend code changes.

If validation cannot be run, say why.

TIS UI/UX Design Philosophy

TIS should feel like a clean, premium academic SaaS platform. It should not feel like a generic admin dashboard, spreadsheet wrapper, or decorative marketing shell disconnected from real product value.

Overall Product Identity

Design principles:

- 1 professional,
- 1 light,
- 1 trustworthy,
- 1 academic,
- 1 calm,
- 1 data-aware,
- 1 premium without being flashy.

The product serves serious academic operations. UI should reduce confusion and support confident decisions.

Operational FastAPI App

The operational app should prioritize:

- 1 clarity,
- 1 role-based navigation,
- 1 data density,
- 1 fast scanning,
- 1 predictable controls,
- 1 strong permission boundaries,
- 1 branch/year context visibility.

Avoid:

- 1 generic admin clutter,
- 1 decorative cards that do not help workflows,
- 1 large marketing-style hero sections inside operational tools,
- 1 hiding important state behind vague labels,
- 1 layouts that make repeated operational use slow.

Operational pages should help users answer:

- 1 What am I looking at?
- 1 Which school/branch/year is active?
- 1 What needs action?

- 1 What can my role do here?
- 1 What changed after I saved?

Platform Owner Console

The Platform Owner Console should feel like a controlled operations console.

Priorities:

- 1 global visibility,
- 1 owner/developer separation,
- 1 clear organization switching,
- 1 safe access to sensitive tools,
- 1 explicit status labels,
- 1 no accidental tenant context confusion.

Avoid:

- 1 broad controls without owner-only checks,
- 1 exposing platform tools to developers by appearance alone,
- 1 unclear organization/branch state.

Knowledge Center

The Knowledge Center is an internal owner utility, not a marketing page.

Priorities:

- 1 KMS health,
- 1 source coverage,
- 1 freshness status,
- 1 protected PDF actions,
- 1 ADR/change/module visibility,
- 1 KIA policy reminder.

Avoid:

- 1 direct public static PDF links,
- 1 regenerate actions until approved,
- 1 app-side Markdown rewriting,
- 1 decorative presentation that hides status.

SaaS Onboarding Pages

SaaS onboarding should feel guided, calm, and customer-facing.

Priorities:

- 1 clear next step,

- 1 progress visibility,
- 1 plain customer language,
- 1 reassurance around setup and billing,
- 1 no internal engineering terms.

Customer-facing language should avoid:

- 1 tenant,
- 1 provisioning,
- 1 M1/M2/M3/M4/M5,
- 1 schema,
- 1 migration,
- 1 internal role mechanics.

Use customer language such as:

- 1 organization,
- 1 school,
- 1 branch or campus,
- 1 setup,
- 1 plan,
- 1 billing,
- 1 account,
- 1 getting your workspace ready.

Next.js Landing Website

The landing website should use premium storytelling and strong visual assets.

Priorities:

- 1 clear problem/solution narrative,
- 1 actual product credibility,
- 1 school operations language,
- 1 strong visual hierarchy,
- 1 polished screenshots or generated assets when appropriate,
- 1 conversion path into demo/signup.

Avoid:

- 1 weak fake 3D visuals,
- 1 generic SaaS gradients without product specificity,
- 1 vague claims unsupported by product reality,
- 1 internal terms like tenant/provisioning/milestones,

- 1 cluttered feature walls.

The landing page source of truth is `tis - landing-website/`, not legacy FastAPI landing files.

Visual System Direction

Future design system should support:

- 1 consistent cards, tables, filters, status badges, and action buttons,
- 1 clear empty/loading/error states,
- 1 accessible contrast,
- 1 stable layouts across modules,
- 1 consistent icon and label patterns,
- 1 role-aware navigation,
- 1 responsive behavior without losing operational density.

Tone And Copy

Internal app copy:

- 1 concise,
- 1 action-oriented,
- 1 operationally clear.

Customer-facing copy:

- 1 plain,
- 1 confident,
- 1 benefit-oriented,
- 1 free of internal implementation terms.

Platform owner copy:

- 1 precise,
- 1 status-driven,
- 1 explicit about access and risk.

docs/engineering/PRODUCT_ROADMAP.md

TIS Product Roadmap

This roadmap summarizes completed, current, next, and future product directions. It is not a release commitment; it is the current planning map for developers, owners, and reviewers.

Completed

SaaS Identity Foundation

Established SaaS account signup, login, account area, and the boundary between SaaS accounts, platform identities, and operational tenant users.

Pending Organizations Zone

Created pending organization structures and owner-facing review concepts for organizations moving through SaaS onboarding.

Plans And Billing Foundation

Added plan catalog, plan selection, billing status, checkout summary, checkout return/cancel, and payment/billing service boundaries.

Paddle Payment Collection

Added Paddle-oriented payment architecture with provider logic isolated behind service/client modules.

Tenant Provisioning Engine

Added provisioning workflows and jobs to create or connect operational school group/branch/year/user records from ready pending organizations.

KMS Foundation

Created source-of-truth Markdown docs, change history, ADRs, module history, AI project context, generated PDF booklet, and manifest.

Platform Owner Knowledge Center

Added protected owner-only Knowledge Center for KMS health, source freshness, manifest metadata, ADRs, module history, and protected PDF view/download.

KMS v3.0 Phase 3A

Added engineering handbook foundation with module map, repository architecture, user/system flows, and onboarding structure.

M7 Subscription Management

Completed entitlement foundations, customer Subscription Management portal, paid branch quantity management, upgrades and scheduled downgrades, provider-authoritative proration, cancellation/reversal, centralized lifecycle/action policy, Paddle billing history, protected invoice downloads, and webhook/reconciliation safeguards.

Automatic KMS Enforcement

Added root AI instructions, machine-readable task KIA, major-change detection, read-only PDF/manifest checks, pull-request and dev CI validation, and a required KMS gate before master deployment.

Current

KMS Enforcement Review

Review enforcement against real development tasks, keep classification conservative, and tune only demonstrated false positives without permitting silent no-impact declarations.

Landing / Customer Experience Preparation

The product is preparing for stronger public customer journey work from landing page storytelling through SaaS signup, onboarding, billing, and operational access.

Next

Premium Landing Page Redesign

Improve public website quality, product storytelling, visuals, and conversion clarity while preserving the separate Next.js source-of-truth.

Customer Journey From Landing To Signup

Clarify the path from public website to SaaS signup, plan selection, onboarding, checkout, and workspace readiness.

SaaS Onboarding Refinement

Improve onboarding language, progress, validation, customer reassurance, and platform owner visibility.

Paddle Live Configuration

Configure live Paddle behavior when the payment account and production readiness are approved.

Expand Entitlement Enforcement

Extend plan entitlement checks beyond the current approved pilot only when commercial rules for each module are reviewed.

Subscription Operations Hardening

Continue production validation, renewal/payment-failure handling, reconciliation observability, and owner support workflows without weakening Paddle authority or tenant isolation.

Future

AI Academic Assistant

Support academic leaders with intelligent recommendations based on verified tenant data.

AI Assessment Generation

Generate curriculum-aware assessments or question banks when curriculum/planning data is reliable enough.

AI Observation Improvement Plans

Assist supervisors with improvement plans, follow-up suggestions, and structured feedback from observation records.

AI Calendar Activity Planning

Suggest academic calendar activities, reminders, and coordination plans.

AI Dashboards And Academic Analytics

Provide richer academic insight, trends, risks, and multi-branch intelligence.

Multi-Branch Executive Dashboards

Give leadership cross-branch visibility into staffing, workload, observations, calendar, and planning health.

Curriculum And Planning Workflows

Expand from operational staffing/planning toward curriculum-aware academic planning if approved.

Reporting Improvements

Improve exports, dashboards, executive summaries, and decision-support reports.

Mobile Or Portal Expansion

Explore teacher/mobile/parent-style portals only if later approved and aligned with product strategy.

Roadmap Guardrails

- 1 Do not build AI features before data quality, permissions, and plan boundaries are clear.
- 1 Do not extend subscription lifecycle actions without verified Paddle behavior, fail-closed reconciliation, and updated KMS records.
- 1 Do not redesign landing/customer journey without preserving the Next.js boundary.
- 1 Do not expose internal terms to customers.
- 1 Update KMS docs and roadmap after meaningful roadmap changes.

docs/engineering/REJECTED_DECISIONS.md

TIS Rejected Architectural Decisions

This document records significant ideas that were considered and rejected. It helps future developers and AI assistants avoid reopening old decisions without new evidence.

Rejected decisions are not failures. They are part of the design record.

Single Application For Landing And Portal

Date: 2026-06-26

Context: TIS needs both a public marketing site and a protected operational app. The public site needs premium storytelling and conversion design; the app needs authenticated academic operations.

Proposed solution: Use the FastAPI/Jinja app for both the public landing website and the operational portal.

Why rejected: The two surfaces have different runtimes, audiences, design needs, deployment concerns, and risk profiles. Mixing them would make backend/operational tasks more likely to accidentally change the public website.

Chosen solution: Keep `tis-landing-website/` as a separate Next.js public landing website and keep the FastAPI app as the operational portal.

Long-term consequences: Developers must respect the landing/portal boundary. Landing changes require explicit approval and must use the Next.js source of truth.

Stripe Instead Of Paddle

Date: 2026-06-26

Context: TIS needs subscription payment collection, billing state, and future subscription lifecycle workflows.

Proposed solution: Use Stripe as the payment provider.

Why rejected: The current approved architecture is Paddle-oriented. Changing providers would reopen checkout, webhook, billing, tax/compliance, customer portal, and subscription lifecycle assumptions.

Chosen solution: Use Paddle behind service/client boundaries in `saas/`.

Long-term consequences: Provider-specific behavior must remain isolated. A future provider change would require a new ADR and careful migration plan.

Immediate Tenant Provisioning

Date: 2026-06-26

Context: Public signup and SaaS onboarding collect organization data before a school is ready for live operational access.

Proposed solution: Create operational school group, branch, user, and academic records immediately after signup or checkout return.

Why rejected: Immediate provisioning risks creating live tenant data before payment is verified, before onboarding is complete, or before platform owner review. Checkout return pages are not authoritative payment confirmation.

Chosen solution: Delay tenant provisioning until payment/readiness is verified and use platform-owner-visible provisioning jobs.

Long-term consequences: There are more states between signup and operational login, but tenant data is safer and provisioning can be reviewed/retried.

Public Documentation Access

Date: 2026-06-26

Context: The KMS generates a PDF under `static/docs/`, but the content includes internal architecture, roadmap, and operational guidance.

Proposed solution: Link directly to the generated PDF through public static paths.

Why rejected: Internal documentation should not be exposed as a public asset. Static file serving is not authorization-aware.

Chosen solution: Expose the PDF through protected Platform Owner routes only.

Long-term consequences: The UI must not link directly to `/static/docs/...` Future storage may move generated docs outside public static storage.

Documentation Without ADRs

Date: 2026-06-26

Context: TIS decisions span SaaS identity, payments, provisioning, landing architecture, and KMS governance.

Proposed solution: Keep only a master context and change history.

Why rejected: Chronological history does not explain major decision tradeoffs deeply enough. Future developers would repeatedly reopen settled architecture questions.

Chosen solution: Maintain ADRs under `docs/adr/` for major accepted decisions and this rejected-decision record for important alternatives.

Long-term consequences: Major decisions require explicit traceability. ADRs and rejected decisions must be updated when architecture changes.

Weak Generic Landing Page Design

Date: 2026-06-26

Context: The public landing website is the first impression for schools evaluating TIS.

Proposed solution: Use a generic SaaS layout with vague claims, weak fake 3D visuals, or internal implementation language.

Why rejected: TIS needs credibility with school leaders. Generic visuals and internal language reduce trust and do not communicate academic operations value.

Chosen solution: Use premium storytelling, strong visual assets, clear customer language, and the separate Next.js landing visual system.

Long-term consequences: Landing work should be deliberate, visually strong, and customer-facing. Avoid terms like tenant, provisioning, and internal milestone labels.

Mixing SaaS Identities With Operational Users

Date: 2026-06-26

Context: TIS has public SaaS accounts, operational tenant users, and platform identities.

Proposed solution: Use one identity concept for signup, billing, tenant access, and platform administration.

Why rejected: This would weaken security and make it easy to confuse pending accounts with live operational users or platform owners.

Chosen solution: Keep SaaS account identity, operational tenant identity, and platform identity distinct.

Long-term consequences: Developers must always ask which identity world a change belongs to. Provisioning bridges worlds only through approved flows.

App-Side Automatic Documentation Rewriting

Date: 2026-06-26

Context: The KMS requires docs to remain current after meaningful implementation work.

Proposed solution: Let the running app automatically rewrite Markdown docs when it detects staleness.

Why rejected: Source-of-truth docs must be reviewed development artifacts. Silent rewriting would damage reviewability and could introduce incorrect historical records.

Chosen solution: Developers or AI assistants update Markdown docs as part of approved implementation work. The app may detect freshness and expose status, but not rewrite source docs.

Long-term consequences: Knowledge Impact Assessment remains a required human/AI workflow step. Future regenerate actions may rebuild PDFs only from reviewed Markdown.

Uncontrolled Regenerate Button

Date: 2026-06-26

Context: The Knowledge Center can show whether the PDF snapshot is current or stale.

Proposed solution: Add an immediate button that regenerates docs and/or source files from the app.

Why rejected: Phase 2C is read-only. Runtime regeneration raises deployment persistence, audit, concurrency, and source-control questions.

Chosen solution: Do not add regenerate behavior until separately approved. Keep source docs updated through reviewed development work.

Long-term consequences: A future regenerate action must be explicit, owner-only, audited where appropriate, and must not rewrite Markdown source docs.

TIS Visual Documentation Guide

This guide defines how visual documentation should evolve. No screenshots are required yet, but future screenshots and diagrams should follow this structure.

Purpose

Visual documentation should help developers, reviewers, owners, and future AI assistants understand TIS screens, workflows, data relationships, and design expectations.

Visual docs should not replace Markdown source docs. They should support them.

Future Storage Location

Recommended future location:

```
docs/visuals/  
  screenshots/  
  diagrams/  
  flows/
```

Recommended generated/reference output:

```
static/docs/visuals/
```

Do not store sensitive production data in screenshots.

Screenshot Standards

Screenshots should:

- 1 use clean test/demo data,
- 1 avoid private student/teacher/customer information,
- 1 show the full relevant viewport or focused workflow area,
- 1 include enough context to identify branch/year/page state,
- 1 avoid browser chrome unless needed,
- 1 be updated when UI behavior or layout meaningfully changes.

Preferred formats:

- 1 PNG for UI screenshots,
- 1 SVG or PNG for diagrams,
- 1 PDF only for exported documentation snapshots.

Naming Convention

Use stable, descriptive names:

```
YYYY-MM-DD_module_screen_state.png  
YYYY-MM-DD_flow_name_step_number.png  
YYYY-MM-DD_architecture_diagram_name.svg
```

Examples:

2026-06-26_platform-console_owner-view.png
2026-06-26_knowledge-center_current-status.png
2026-06-26_saas-onboarding_step-organization.png

Update Policy

Update visual docs when:

- 1 a screen layout meaningfully changes,
- 1 workflow steps change,
- 1 a module gains a new major view,
- 1 navigation or role visibility changes,
- 1 a diagram no longer reflects architecture.

Visual doc updates should be mentioned in the Knowledge Impact Assessment.

Future Diagram Strategy

Future diagrams should cover:

- 1 identity separation,
- 1 SaaS onboarding to provisioning,
- 1 payment webhook confirmation,
- 1 tenant isolation,
- 1 operational data model,
- 1 KMS source-to-PDF-to-Knowledge-Center flow,
- 1 landing-to-signup customer journey.

Diagrams should be simple and versioned through Markdown references.

Planned Visual Areas

Platform Console

Future visuals:

- 1 owner account controls,
- 1 developer management,
- 1 organization/branch context switching.

Knowledge Center

Future visuals:

- 1 healthy status,
- 1 stale status,
- 1 source document table,
- 1 KIA panel.

Dashboard

Future visuals:

- 1 operational dashboard overview,
- 1 staffing/report widgets,
- 1 scope context display.

Teachers

Future visuals:

- 1 teacher list,
- 1 teacher edit/profile,
- 1 qualification/capacity views.

Workforce Planning

Future visuals:

- 1 planning table,
- 1 section ownership,
- 1 workload/capacity states.

Academic Calendar

Future visuals:

- 1 calendar view,
- 1 event editing,
- 1 exports or responsibility states.

SaaS Onboarding

Future visuals:

- 1 signup,
- 1 organization step,
- 1 contacts,
- 1 branches,
- 1 academic setup,
- 1 review.

Billing

Future visuals:

- 1 plan selection,
- 1 checkout summary,

- 1 billing status,
- 1 payment pending/verified states.

Landing Page

Future visuals:

- 1 hero,
- 1 problem/solution section,
- 1 product capability section,
- 1 signup/demo conversion paths.

Reports

Future visuals:

- 1 allocation report,
- 1 dashboards,
- 1 export states.

Guardrails

- 1 Do not include sensitive real data.
- 1 Do not let visual docs become the only source of truth.
- 1 Do not update landing visuals during backend tasks unless explicitly approved.
- 1 Keep visual docs aligned with KMS change history.

docs/engineering/AI_OPTIMIZATION_GUIDE.md

TIS AI Optimization Guide

This is the definitive onboarding guide for future AI assistants working on TIS.

Preferred Onboarding Order

For any new AI coding conversation:

- 1 Read docs/AI_PROJECT_CONTEXT.md.
- 2 Read docs/README.md.
- 3 Read docs/PROJECT_STATE.md.
- 4 Read docs/TIS_MASTER_CONTEXT.md.
- 5 Read docs/engineering/README.md.
- 6 Read task-specific engineering docs.
- 7 Read relevant ADRs.
- 8 Read relevant module history.
- 9 Inspect code with `rg` before editing.

Choosing Task-Specific Docs

Use this map:

- 1 SaaS signup/login/account: docs/history/saas-onboarding/, ADR 0002.
- 1 Billing/payment/Paddle: docs/history/subscriptions/, ADR 0003, ADR 0004.
- 1 Provisioning: docs/history/provisioning/, ADR 0005.
- 1 Landing page: docs/marketing/, ADR 0001, ADR 0007.
- 1 Platform owner/Knowledge Center: docs/history/platform-knowledge/.
- 1 Workforce planning/timetable/teachers/subjects: docs/history/workforce-planning/.
- 1 Calendar: docs/history/academic-calendar/.
- 1 Architecture/data model: docs/engineering/REPOSITORY_ARCHITECTURE.md, docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md.
- 1 Design/UI: docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md.
- 1 Standards: docs/engineering/DEVELOPMENT_STANDARDS.md.
- 1 Roadmap: docs/engineering/PRODUCT_ROADMAP.md.

How To Interpret ADRs

ADRs explain accepted long-term decisions.

When an ADR applies:

- 1 treat it as design authority,
- 1 do not reverse it casually,

- 1 if a change conflicts with it, propose a new ADR or mark it superseded only with approval,
- 1 link related docs/history.

How To Use Module History

Module history preserves deeper before/after context.

Before changing a module:

- 1 read the module history folder,
- 1 identify the previous documented state,
- 1 update it if the module's meaning changes,
- 1 do not replace chronological CHANGE_HISTORY.md with module history.

How To Complete KIA

Every final response must include:

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

If meaningful behavior, architecture, workflow, roadmap, design, data model, or product state changes, documentation probably changed too.

Avoiding Architectural Regressions

Do not:

- 1 merge SaaS account identity with operational users,
- 1 treat checkout return as verified payment,
- 1 provision tenants immediately from public signup,
- 1 expose internal docs publicly,
- 1 let platform developers act as platform owners,
- 1 modify landing code during backend tasks,
- 1 rewrite source docs from the running app,
- 1 change database/migrations without explicit approval.

Preserving Tenant Isolation

Always identify:

- 1 school group,
- 1 branch,
- 1 academic year,
- 1 current user,

- 1 user type,
- 1 role/permissions,
- 1 whether the user is platform, tenant, or SaaS account.

If scope is unclear, inspect code and tests before editing.

Distinguishing Operational vs SaaS Code

Operational app:

- 1 root FastAPI files,
- 1 routers/,
- 1 templates/ operational templates,
- 1 models.py,
- 1 branch/year/teacher/planning/calendar/observation workflows.

SaaS:

- 1 saas/,
- 1 templates/saas/,
- 1 signup/login/account/onboarding/billing/provisioning readiness.

Landing:

- 1 tis-landing-website/,
- 1 docs/marketing/.

KMS:

- 1 docs/,
- 1 scripts/generate_docs_pdf.py,
- 1 knowledge_service.py,
- 1 templates/platform_knowledge_center.html,
- 1 static/docs/.

Prompt Engineering Recommendations

Future prompts should specify:

- 1 implementation vs planning,
- 1 allowed files,
- 1 forbidden files,
- 1 whether docs must be updated,
- 1 whether PDF regeneration is required,
- 1 whether routes/app behavior can change,
- 1 whether database/migrations are in scope,

- 1 whether landing is in scope.

Good prompt pattern:

Implement [specific goal].
Allowed files: [...]
Do not modify: [...]
Update KMS docs and regenerate PDF if docs change.
Run: [...]
Report KIA.

AI Readiness Review

Current strengths:

- 1 AI context exists.
- 1 Engineering docs cover modules, repository, flows, database, standards, UI/UX, roadmap, rejected decisions, governance, and visual docs.
- 1 ADRs and module history preserve decision context.
- 1 PDF/manifest provide generated snapshot and freshness metadata.

Remaining gaps for future KMS improvements:

- 1 More screenshots and diagrams are needed.
- 1 More module-specific deep docs may be useful for timetable, observations, permissions, and dashboard/reporting.
- 1 Test strategy documentation could be expanded.
- 1 Deployment/runbook documentation could be expanded.
- 1 API/route inventory could be generated or manually documented later.
- 1 Data migration history could be summarized more explicitly.

Do not implement these gaps unless a future phase approves them.

docs/engineering/PROJECT_GOVERNANCE.md

TIS Project Governance

This document defines how TIS engineering decisions, approvals, documentation, and quality gates should be governed.

Ownership Model

Product/project ownership: Defines business direction, roadmap priorities, customer experience, and approval for major changes.

Platform Owner: Owns platform-level operations, platform accounts, provisioning oversight, and KMS visibility.

Engineering implementer: Human developer, Codex, or ChatGPT-assisted coding session responsible for scoped implementation and validation.

Reviewer: Reviews implementation, KMS updates, risk boundaries, and validation results.

Architectural Authority

Architectural authority comes from:

- 1 approved user direction,
- 1 ADRs,
- 1 docs/TIS_MASTER_CONTEXT.md,
- 1 engineering handbook docs,
- 1 module history,
- 1 existing code patterns.

If these conflict, pause and clarify before making irreversible changes.

Approval Workflow

Major work should follow:

- 1 clarify objective and scope,
- 2 identify allowed/forbidden files,
- 3 inspect code/docs,
- 4 implement scoped change,
- 5 update KMS,
- 6 run validation,
- 7 report KIA,
- 8 wait for review before broader phases.

Documentation Authority

Markdown under docs/ is the source of truth.

Generated artifacts:

- 1 PDF booklet,
- 1 manifest,
- 1 future screenshots/diagrams.

Generated artifacts support the docs; they do not replace them.

Coding Workflow

Default workflow:

- 1 inspect before editing,
- 1 prefer existing patterns,
- 1 keep changes small,
- 1 preserve tenant isolation,
- 1 preserve identity boundaries,
- 1 avoid unrelated rewrites,
- 1 run targeted validation,
- 1 update KMS when meaningful.

Review Expectations

Review should check:

- 1 behavior matches scope,
- 1 no forbidden files changed,
- 1 tenant isolation preserved,
- 1 permissions preserved,
- 1 SaaS/payment/provisioning untouched unless approved,
- 1 landing untouched unless approved,
- 1 docs/KIA complete,
- 1 validation credible.

Branch Strategy

Current assumption:

- 1 active development branch: dev,
- 1 production/live branch is separate and must be confirmed before deployment.

Rules:

- 1 do not push unless requested,
- 1 do not commit unless requested,
- 1 do not merge unless requested,

- 1 preserve unrelated local changes.

Release Philosophy

Prefer:

- 1 small reviewable increments,
- 1 documentation updated with implementation,
- 1 explicit milestone boundaries,
- 1 staged rollout for risky systems,
- 1 no hidden operational changes.

High-risk areas:

- 1 authentication,
- 1 platform owner access,
- 1 tenant isolation,
- 1 payment,
- 1 provisioning,
- 1 database/migrations,
- 1 landing conversion path.

Quality Gates

Before final report:

- 1 run relevant compile/tests/smoke checks,
- 1 verify generated docs if docs changed,
- 1 inspect working-tree scope,
- 1 mention validation gaps if any.

Documentation Gates

Every meaningful implementation must answer:

- 1 Did master context change?
- 1 Did project state change?
- 1 Did change history change?
- 1 Is an ADR needed?
- 1 Is module history needed?
- 1 Did AI project context need updating?
- 1 Did engineering docs need updating?
- 1 Was PDF regenerated?

KIA Requirement

Final reports must include:

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

Engineering Decision Traceability

Use this traceability model:

- 1 ADR: why a major decision was accepted.
- 1 Rejected decisions: why significant alternatives were not chosen.
- 1 CHANGE_HISTORY: chronological summary of meaningful changes.
- 1 Module history: deeper before/after context for one area.
- 1 Master context: durable current truth.
- 1 Project state: current status, priority, known issues, next work.
- 1 AI project context: compact first-read AI onboarding.
- 1 Engineering handbook: module, architecture, flow, standards, data, UI, roadmap, governance knowledge.
- 1 Generated PDF: snapshot for review/reference.
- 1 Manifest: generated metadata and source freshness basis.

When to update each:

- 1 Update ADRs for major decisions.
- 1 Update rejected decisions when a significant alternative is intentionally declined.
- 1 Update change history for meaningful changes.
- 1 Update module history for area-specific evolution.
- 1 Update master context for durable current truth.
- 1 Update project state for current status and next work.
- 1 Update AI context when first-read onboarding truth changes.
- 1 Update engineering docs when developer understanding changes.
- 1 Regenerate PDF/manifest when included docs change.

docs/engineering/KNOWLEDGE_LIFECYCLE.md

TIS Knowledge Lifecycle

This document defines how the TIS Knowledge Management System evolves with the software from planning through deployment and maintenance.

Core Principle

Documentation evolves with implementation. Markdown is the source of truth. The PDF and manifest are generated snapshots. The Knowledge Center is the owner-facing status surface.

No meaningful implementation is complete until the Knowledge Impact Assessment is complete.

Documentation Lifecycle

- 1 A task is proposed.
- 2 Relevant docs, ADRs, and module history are read.
- 3 The likely documentation impact is identified.
- 4 Implementation happens within approved scope.
- 5 KIA is completed.
- 6 Relevant Markdown docs are updated.
- 7 ADRs are added or updated if decisions changed.
- 8 Module history is updated if a module's documented state changed.
- 9 PDF and manifest are regenerated if included docs changed.
- 10 Final report describes the knowledge impact.

Engineering Lifecycle

- 1 Inspect before editing.
- 2 Plan the smallest safe implementation.
- 3 Preserve tenant, identity, permission, SaaS, payment, provisioning, and landing boundaries.
- 4 Implement scoped changes.
- 5 Validate with focused checks.
- 6 Update KMS.
- 7 Report KIA.
- 8 Wait for review, commit, push, or deployment approval.

Approval Lifecycle

- 1 Owner defines goal and constraints.
- 2 Implementer confirms scope.
- 3 Implementer makes only approved changes.

- 4 Reviewer checks behavior, scope, docs, validation, and KIA.
- 5 Commit/push/deployment happens only after explicit approval.

Review Lifecycle

Review should verify:

- 1 no forbidden files changed,
- 1 tenant isolation remains intact,
- 1 SaaS/payment/provisioning boundaries are preserved,
- 1 landing code remains untouched unless approved,
- 1 database/migrations are untouched unless approved,
- 1 KMS docs and generated artifacts are current,
- 1 KIA is complete.

Release Lifecycle

Before release:

- 1 Confirm branch and deployment target.
- 2 Confirm tests/checks.
- 3 Confirm docs/PDF/manifest are current.
- 4 Confirm change history and project state.
- 5 Confirm release notes or owner communication if needed.
- 6 Deploy only through approved process.

Maintenance Lifecycle

Maintenance includes:

- 1 periodic KMS freshness review,
- 1 periodic ADR/rejected-decision review,
- 1 module history cleanup for discoverability,
- 1 PDF readability review,
- 1 Knowledge Center health checks,
- 1 updating roadmap/current state after milestones.

Planning To Production

Planning creates the first knowledge obligation. Production closes it.

Expected path:

Idea / task
-> scope and constraints
-> docs and code inspection
-> implementation
-> validation

- > KIA
- > docs/history/ADR updates
- > PDF/manifest regeneration
- > review
- > commit/push
- > deployment
- > post-deployment state update if needed

Guardrails

- 1 The app must not silently rewrite Markdown docs.
- 1 Generated PDFs must not be edited manually.
- 1 Regeneration is artifact generation, not source editing.
- 1 KMS update work must remain reviewable.

TIS Documentation Automation

This document defines current and future automation expectations for the TIS KMS.

Current Automation

Current approved automation:

- 1 `scripts/generate_docs_pdf.py` reads approved Markdown sources.
- 1 It generates `static/docs/TIS_Project_Reference_Booklet.pdf`.
- 1 It generates `static/docs/docs_manifest.json`.
- 1 The manifest records documentation version, branch, commit SHA, source paths, mtimes, sizes, and hashes.
- 1 The manifest records the generated PDF hash and size.
- 1 Markdown source hashes are computed from UTF-8 text normalized to LF, so equivalent CRLF and LF checkouts produce the same hash.
- 1 Manifest source paths are repository-relative POSIX paths; dynamically discovered sources use a stable case-insensitive sort with an explicit tie-breaker.
- 1 `scripts/generate_docs_pdf.py --check` validates source coverage, source hashes, PDF identity, manifest metadata, and documentation version without writing files.
- 1 `.kms-impact.yml` records task-level Knowledge Impact in a small machine-readable schema.
- 1 `scripts/check_kms_impact.py` classifies likely major changes, validates the declaration against the Git diff, and runs generated-artifact freshness checks.
- 1 `scripts/kms.py sync` runs KIA preflight validation, regenerates the PDF and manifest, runs complete freshness validation, and prints a completion summary.
- 1 `scripts/kms.py check` is the single read-only validation entry point for local work and CI.
- 1 GitHub Actions enforce KMS checks on pull requests, dev pushes, and before master deployment.
- 1 The Knowledge Center reads the manifest and checks freshness.

Repository owners must configure the KMS Enforcement / `kms-check` status as a required branch-protection check for protected integration branches. Workflow code makes production deployment depend on KMS validation directly; GitHub branch protection is the external setting that makes failed pull requests unmergeable.

Current automation does not:

- 1 rewrite Markdown docs,
- 1 create ADRs,
- 1 create module history entries,
- 1 decide KIA outcomes,
- 1 generate documentation prose,
- 1 commit or push changes.

Automation blocks stale or undeclared work; humans and approved AI assistants remain responsible for reviewed Markdown updates.

Cross-platform normalization does not relax enforcement. Changed text still changes the source hash, and missing, unexpected, duplicate, or reordered source entries still fail validation.

Git Event Task Boundaries

KIA declarations apply to an implementation task, not to an individual GitHub delivery event. A task may contain an implementation commit followed by one or more review or CI correction commits.

- 1 Pull-request checks compare the pull-request base SHA with the actual pull-request head SHA.
- 1 Push checks on dev calculate the merge base between the repository default branch and the pushed head, then compare that merge base with the pushed head.
- 1 Push checks must not use `github.event.before` as the KIA base because it represents only the latest delivery and can split a multi-commit task.
- 1 Both event types use three-dot comparison and validate every changed KMS Markdown file against the cumulative declaration.

This preserves strict checks for undeclared, missing, or falsely declared documentation while allowing a follow-up commit to modify only the subset relevant to that correction.

Machine-Readable KIA Declaration

Every task updates `.kms-impact.yml`:

```
knowledge_impact: yes
summary: Describe the implemented engineering change.
affected_areas:
  - subscriptions
kms_files_updated:
  - docs/CHANGE_HISTORY.md
no_impact_reason:
major_change_override: no
```

Rules:

- 1 `yes` requires changed authoritative `docs/* .md` files.
- 1 `no` requires a specific reason and no declared KMS Markdown changes.
- 1 a detected major path with `no` also requires `major_change_override: yes`.
- 1 every changed authoritative Markdown file must be listed.
- 1 generated PDF and manifest must be current in both cases.

When Documentation Must Be Updated

Mandatory updates are required when a task changes:

- 1 architecture,
- 1 data model,
- 1 tenant isolation,
- 1 identity boundaries,
- 1 SaaS/account flows,
- 1 billing/payment/provisioning,
- 1 operational modules,

- 1 platform owner behavior,
- 1 UI/UX philosophy,
- 1 landing strategy,
- 1 roadmap,
- 1 governance,
- 1 development standards,
- 1 KMS behavior.

Optional updates may be skipped for:

- 1 typo-only code comments,
- 1 internal refactors with no behavior, architecture, flow, or ownership change,
- 1 generated files that do not affect source truth.

Even when docs are skipped, KIA must explain why.

Phase 6 Commands

Synchronize generated artifacts after reviewing and updating authoritative Markdown:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Run complete read-only validation:

```
.\.venv\Scripts\python.exe scripts\kms.py check
```

sync aborts before writing when KIA or authoritative Markdown validation fails. It writes only the generated PDF and manifest, then applies the same complete validation used by check. The lower-level generator and impact checker remain reusable implementation modules and diagnostic entry points.

Manifest Lifecycle

The manifest is generated every time the PDF is regenerated.

It should be treated as:

- 1 generated metadata,
- 1 source freshness basis,
- 1 Knowledge Center input,
- 1 not the source of truth.

If included Markdown docs change and the manifest is not regenerated, the Knowledge Center should show stale status.

PDF Lifecycle

The PDF is:

- 1 a generated snapshot,
- 1 owner reference material,
- 1 a reviewable artifact,

- 1 not manually edited.

Regenerate after included Markdown docs change.

AI Context Lifecycle

Update docs/AI_PROJECT_CONTEXT .md when:

- 1 first-read onboarding changes,
- 1 current priority changes,
- 1 critical rules change,
- 1 new major docs are added,
- 1 architecture/identity/flow truth changes.

Do not overload AI context with every detail. It is a compact entry point.

Handbook Lifecycle

Update engineering handbook docs when:

- 1 module boundaries change,
- 1 repository ownership changes,
- 1 user/system flows change,
- 1 database concepts change,
- 1 standards change,
- 1 design philosophy changes,
- 1 roadmap changes,
- 1 governance changes.

Module History Lifecycle

Update module history when:

- 1 a specific module's previous documented state should be preserved,
- 1 a feature area changes meaningfully,
- 1 before/after context matters for future implementers.

ADR Lifecycle

Create or update ADRs when:

- 1 major accepted decisions are made,
- 1 accepted decisions are superseded,
- 1 architecture or product boundaries change.

Use rejected decisions when important alternatives are intentionally declined.

Mandatory vs Optional Summary

Mandatory:

- 1 KIA in final report,
- 1 docs update for meaningful changes,
- 1 change history for meaningful changes,
- 1 PDF/manifest regeneration when included docs change.

Conditional:

- 1 ADR for major decisions,
- 1 module history for area-specific state changes,
- 1 AI context for first-read truth changes,
- 1 engineering docs for handbook-level knowledge changes.

Optional:

- 1 visual docs until screenshots/diagrams are approved,
- 1 local Git hooks; CI is authoritative because hooks are optional and bypassable.
- 1 future search, semantic indexing, and documentation analytics.

docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md

TIS Knowledge Impact Assessment Standard

The Knowledge Impact Assessment (KIA) is a required engineering standard. It confirms whether a task changed project knowledge and whether the KMS was kept current.

Required KIA Fields

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

The same decision must be recorded in `.kms-impact.yml` for machine validation. The repository declaration adds `summary`, `affected_areas`, `kms_files_updated`, `no_impact_reason`, and `major_change_override`.

Decision Tree

Ask:

- 1 Did behavior change?
- 2 Did architecture change?
- 3 Did a workflow change?
- 4 Did a module boundary change?
- 5 Did data ownership or tenant scope change?
- 6 Did SaaS/payment/provisioning change?
- 7 Did UI/UX or customer language change?
- 8 Did roadmap or current state change?
- 9 Did development standards/governance change?
- 10 Would a future developer or AI assistant need to know this?

If any answer is yes, knowledge impact is probably yes.

Checklist

For knowledge-impacting work:

- 1 update relevant source docs,
- 1 update docs/CHANGE_HISTORY.md,
- 1 update module history if module state changed,
- 1 update ADRs if major decisions changed,
- 1 update rejected decisions if an important alternative was declined,
- 1 update AI project context if first-read truth changed,

- 1 regenerate PDF/manifest,
- 1 report validation.
- 1 update `.kms-impact.yml` and run `scripts/check_kms_impact.py`.

Examples

Example: Docs Updated

Knowledge impact: Yes
Docs updated:
- docs/TIS_MASTER_CONTEXT.md
- docs/CHANGE_HISTORY.md
- docs/history/provisioning/...
Change history updated: Yes
ADR needed: No
Module history updated: Yes
PDF regenerated: Yes
AI project context updated: No
Reason if not updated: AI onboarding truth did not change.

Example: No Docs Needed

Knowledge impact: No
Docs updated: None
Change history updated: No
ADR needed: No
Module history updated: No
PDF regenerated: No
AI project context updated: No
Reason if not updated: Change was limited to a typo in an internal variable name and did not affect behavior, architecture, workflow, roadmap, or developer knowledge.

Approval Expectations

Reviewers should reject final reports that omit KIA.

If docs were skipped, the reason must be specific. "Not needed" is not enough.

Failure Cases

KIA fails when:

- 1 final report omits it,
- 1 meaningful behavior changed but docs did not,
- 1 docs changed but PDF/manifest were not regenerated,
- 1 ADR-worthy decisions were hidden in code changes,
- 1 module state changed without module history,
- 1 AI context became stale after major onboarding truth changed.
- 1 `.kms-impact.yml` is missing, stale, inconsistent with the Git diff, or uses an unexplained override.
- 1 generated-artifact validation fails.

Developer Responsibilities

Developers must:

- 1 assess KIA honestly,

- 1 update docs in the same task,
- 1 keep generated artifacts current,
- 1 avoid unrelated documentation churn,
- 1 preserve historical truth.

AI Responsibilities

AI assistants must:

- 1 read relevant docs before coding,
- 1 identify likely KMS impact,
- 1 not invent history,
- 1 not silently skip docs,
- 1 not commit or push unless requested,
- 1 include KIA in every implementation final report.

TIS Self-Evolving Workflow

This is the official engineering workflow for keeping TIS software and knowledge synchronized.

Workflow

Task

- > Implementation
- > Validation
- > Knowledge Impact Assessment
- > Update .kms-impact.yml
- > Documentation Updates
- > ADR if required
- > Module History if required
- > KMS Sync (PDF + Manifest + Freshness)
- > KMS Read-Only Check
- > Review
- > Commit
- > Push
- > Deployment

Task

Clarify:

- 1 objective,
- 1 allowed files,
- 1 forbidden files,
- 1 validation requirements,
- 1 documentation requirements,
- 1 commit/push/deployment instructions.

Implementation

Implement only the approved scope.

Protect:

- 1 tenant isolation,
- 1 identity boundaries,
- 1 payment/provisioning correctness,
- 1 owner-only controls,
- 1 landing boundaries,
- 1 database/migration safety.

Validation

Run focused checks:

- 1 compile checks,

- 1 relevant tests,
- 1 route/template smoke checks,
- 1 PDF generation for docs,
- 1 frontend checks if frontend is in scope.

Knowledge Impact Assessment

Complete KIA before final report.

Do not treat documentation as optional when knowledge changed.

Record the same assessment in `.kms-impact.yml`. The declaration must change with the task and match the actual Git diff.

Documentation Updates

Update relevant docs:

- 1 master context,
- 1 project state,
- 1 AI context,
- 1 engineering docs,
- 1 change history,
- 1 module history,
- 1 ADRs/rejected decisions.

ADR If Required

Create/update ADR when architecture or product boundaries change.

Module History If Required

Update module history when area-specific before/after context matters.

Synchronize And Check KMS

If included Markdown docs changed:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Final read-only confirmation:

```
.\.venv\Scripts\python.exe scripts\kms.py check
```

sync validates KIA before generating derived artifacts and validates freshness afterward. It never edits authoritative Markdown.

Review

Reviewers check:

- 1 scope,
- 1 behavior,

- 1 tests,
- 1 KMS updates,
- 1 generated artifacts,
- 1 KIA.

CI repeats these checks and blocks pull-request completion, dev integration, or master deployment when declaration or freshness validation fails.

Commit / Push / Deployment

These are explicit actions, not assumptions.

- 1 Commit only when requested.
- 1 Push only when requested.
- 1 Deploy only through approved process.

Why This Is Self-Evolving

The workflow makes knowledge updates part of engineering completion. The system evolves because every meaningful change carries its context, history, decisions, and generated reference forward.

docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md

TIS Documentation Dependency Map

This document explains how KMS documents relate to each other and how changes propagate.

Dependency Model

Master Context

- > Project State
- > Engineering Handbook
- > AI Project Context
- > ADR
- > History
- > Knowledge Center
- > Manifest
- > PDF

The arrows show common reading and propagation flow. They are not strict ownership hierarchy.

Master Context

Purpose: Durable product, architecture, workflow, roadmap, and critical-rule truth.

Update when:

- 1 long-term architecture changes,
- 1 product vision changes,
- 1 major workflow changes,
- 1 critical rules change,
- 1 KMS structure changes.

Project State

Purpose: Current branch, priority, milestone, known issue, and next-work truth.

Update when:

- 1 active priority changes,
- 1 phase status changes,
- 1 known issues change,
- 1 next planned work changes.

Engineering Handbook

Purpose: Developer-facing deep knowledge: modules, repository, flows, database, standards, design, roadmap, governance, AI guidance, rejected decisions.

Update when:

- 1 developer understanding changes,
- 1 module maps change,

- 1 architecture or data ownership changes,
- 1 standards/governance changes,
- 1 future onboarding would be incomplete.

AI Project Context

Purpose: Compact first-read file for future AI coding conversations.

Update when:

- 1 first-read onboarding changes,
- 1 major docs are added,
- 1 current priority changes,
- 1 critical boundaries change.

ADR

Purpose: Accepted major decision record.

Update when:

- 1 a major architectural/product decision is made,
- 1 a prior decision is superseded,
- 1 a decision needs formal traceability.

Rejected Decisions

Purpose: Record important alternatives that were declined.

Update when:

- 1 future teams are likely to reconsider a rejected path,
- 1 a rejected alternative explains the chosen architecture.

History

Purpose: Chronological and module-specific evolution.

Update when:

- 1 meaningful change occurs,
- 1 module before/after state matters,
- 1 project knowledge changes.

Knowledge Center

Purpose: Owner-facing app view of KMS health, freshness, and protected PDF access.

Update when:

- 1 KMS metadata behavior changes,

- 1 Knowledge Center UI/status behavior changes,
- 1 protected documentation access changes.

Manifest

Purpose: Generated metadata for PDF and source freshness.

Update when:

- 1 PDF is regenerated.

Do not edit manually.

PDF

Purpose: Generated handbook snapshot.

Update when:

- 1 included Markdown docs change.

Do not edit manually.

Propagation Examples

Feature changes workflow:

Implementation -> CHANGE_HISTORY -> module history -> master/project state if needed
-> PDF/manifest

Architecture decision workflow:

Decision -> ADR -> rejected decision if relevant -> master context -> engineering
handbook -> change history -> PDF/manifest

Roadmap change workflow:

Roadmap -> PRODUCT_ROADMAP -> PROJECT_STATE -> AI_PROJECT_CONTEXT if first-read truth
changed -> PDF/manifest

KMS structure change workflow:

KMS docs -> README -> MASTER_CONTEXT -> AI_PROJECT_CONTEXT -> CHANGE_HISTORY ->
PDF/manifest

docs/engineering/AI_CODING_WORKFLOW.md

TIS AI Coding Workflow

This guide defines how future AI assistants should work inside TIS.

Planning Before Coding

AI assistants must:

- 1 follow root AGENTS . md,
- 1 read docs / AI_PROJECT_CONTEXT . md,
- 1 read relevant engineering docs,
- 1 read relevant ADRs/module history,
- 1 inspect the codebase with rg,
- 1 identify allowed and forbidden files,
- 1 identify likely KMS impact,
- 1 update . kms - impact . yml for the current task,
- 1 avoid starting from assumptions.

Implementation Review Before Editing

Before editing:

- 1 confirm the module boundary,
- 1 confirm tenant/identity/payment/provisioning risk,
- 1 confirm tests or validation,
- 1 confirm docs to update,
- 1 confirm that app, SaaS, landing, database, or routes are in scope before touching them.

Implementation

During implementation:

- 1 keep changes narrow,
- 1 follow existing patterns,
- 1 avoid unrelated rewrites,
- 1 use apply_patch for manual edits,
- 1 do not commit or push,
- 1 do not modify tis . db unless explicitly approved.

Validation Expectations

Run appropriate validation:

- 1 compile checks for Python scripts/modules,
- 1 targeted tests for behavior,
- 1 PDF generation for docs,
- 1 template/route smoke checks for UI routes,
- 1 frontend checks for landing/frontend tasks.

If validation cannot be run, state why.

Documentation Updates

AI assistants must update KMS when knowledge changes:

- 1 change history,
- 1 project state,
- 1 master context,
- 1 AI context,
- 1 engineering docs,
- 1 ADRs/rejected decisions,
- 1 module history.

KIA

Every final response must include KIA.

Do not omit KIA because the task "felt small." Assess it.

The machine-readable declaration must agree with the final KIA and actual changed Markdown. Use the explicit major-change override only for a genuinely non-behavioral change and provide a specific explanation.

Commit Strategy

AI assistants must not commit unless the user explicitly asks.

When committing is approved:

- 1 summarize scope,
- 1 ensure generated docs are current,
- 1 avoid unrelated changes,
- 1 use clear commit messages.

Push Strategy

AI assistants must not push unless explicitly asked.

Before pushing:

- 1 confirm branch,
- 1 confirm tests/checks,
- 1 confirm KMS artifacts,

- 1 run `scripts/kms.py sync` when documentation changed and `scripts/kms.py check` for final read-only enforcement,
- 1 confirm no forbidden files changed.

Deployment Strategy

AI assistants must not deploy unless explicitly asked.

Before deployment:

- 1 confirm production branch/environment,
- 1 confirm database/migration implications,
- 1 confirm SaaS/payment/provisioning risks,
- 1 confirm KMS state,
- 1 confirm rollback or recovery expectations.

Final Response Pattern

Include:

- 1 files changed,
- 1 behavior changed or not,
- 1 validation,
- 1 known issues,
- 1 KIA,
- 1 no vague claims about tests that were not run.

TIS Future Automation Roadmap

This document lists possible future automation improvements. These are not implemented in Phase 3D.

Git Hooks

Potential:

- 1 pre-commit check for changed docs without regenerated PDF/manifest,
- 1 warning when KIA-related docs are stale,
- 1 formatting checks for Markdown.

Considerations:

- 1 hooks must not block emergency work without clear override,
- 1 hooks must be documented for Windows/dev environments.

CI Documentation Validation

Potential:

- 1 run PDF generator,
- 1 verify manifest is current,
- 1 verify required docs exist,
- 1 verify source hashes match,
- 1 fail pull requests when docs/PDF are stale.

Automatic PDF Regeneration

Potential:

- 1 CI-generated PDF artifacts,
- 1 deployment-time regeneration,
- 1 explicit owner-only regenerate action.

Constraint:

- 1 automatic regeneration must not rewrite Markdown source docs.

Automatic Manifest Generation

Potential:

- 1 generate manifest in CI,
- 1 compare manifest against committed PDF,
- 1 expose freshness status in Knowledge Center.

Documentation Diff Reports

Potential:

- 1 summarize doc changes between commits,
- 1 show which docs changed after a feature,
- 1 include KIA summary in release notes.

Stale Documentation Alerts

Potential:

- 1 Knowledge Center warning when manifest/source hashes differ,
- 1 CI alerts,
- 1 dashboard badge for owner users.

Knowledge Center Search

Potential:

- 1 search docs, ADRs, module history, and roadmap from owner page,
- 1 filter by module,
- 1 link to protected document views.

AI Semantic Search

Potential:

- 1 indexed semantic retrieval over KMS docs,
- 1 AI assistant prompt context packs,
- 1 module-specific context bundles.

Constraints:

- 1 preserve privacy,
- 1 do not expose internal docs publicly,
- 1 avoid unreviewed AI-generated source edits.

Documentation Analytics

Potential:

- 1 track stale areas,
- 1 track frequently referenced docs,
- 1 identify docs missing module ownership.

Release Documentation

Potential:

- 1 release notes generated from change history,
- 1 deployment checklist,
- 1 customer-facing change summaries,
- 1 internal technical release notes.

Architecture Dashboards

Potential:

- 1 owner-facing architecture/decision dashboard,
- 1 ADR status summaries,
- 1 module health and documentation freshness.

Priority Recommendation

Likely future order:

- 1 CI documentation validation.
- 2 Stale documentation alert in Knowledge Center.
- 3 Documentation diff report.
- 4 Owner-only explicit PDF regenerate action.
- 5 Search.
- 6 Semantic AI retrieval.

docs/adr/0001-separate-nextjs-landing-website.md

ADR 0001: Separate Next.js Landing Website

Status: Accepted

Date: 2026-06-26

Context

TIS needs a public marketing website and a protected operational application. The public site has different performance, design, routing, and deployment needs than the FastAPI operational portal.

Decision

Keep the public landing website in `tis-landing-website/` as a separate Next.js app. Keep the FastAPI app as the operational portal at `https://app.tisplatform.com`.

Alternatives Considered

- 1 Use FastAPI/Jinja for both marketing and app pages.
- 1 Put marketing pages inside the operational app templates.
- 1 Use a static site generated outside the repository.

Consequences

Positive:

- 1 Clear separation between marketing and operational concerns.
- 1 Landing page can evolve with a modern frontend stack.
- 1 Operational FastAPI app remains focused on authenticated workflows.

Tradeoffs:

- 1 Two runtimes must be understood.
- 1 Deployment and routing boundaries must be respected.

Related Docs / Files

- 1 `tis-landing-website/`
- 1 `docs/marketing/landing_page_source_of_truth.md`
- 1 `docs/TIS_MASTER_CONTEXT.md`
- 1 `templates/landing.html`
- 1 `static/landing/landing.css`

docs/adr/0002-separate-saas-identity-and-operational-users.md

ADR 0002: Separate SaaS Identity And Operational Users

Status: Accepted

Date: 2026-06-26

Context

TIS has SaaS account users who sign up, select plans, onboard organizations, and manage billing. It also has operational tenant users who work inside a provisioned school context. Platform owners and developers are separate platform identities.

Decision

Keep SaaS account identity, platform identity, and operational tenant identity distinct.

Alternatives Considered

- 1 Use one user table concept for all public signup, platform, and tenant users.
- 1 Automatically convert every SaaS signup into an operational tenant user.
- 1 Treat platform users as ordinary tenant administrators.

Consequences

Positive:

- 1 Clearer security boundaries.
- 1 Safer tenant isolation.
- 1 Billing/onboarding can proceed before operational provisioning.

Tradeoffs:

- 1 Identity flows require careful documentation.
- 1 Developers must avoid mixing account, platform, and tenant assumptions.

Related Docs / Files

- 1 `auth.py`
- 1 `models.py`
- 1 `saas/models.py`
- 1 `saas/service.py`
- 1 `saas/router.py`
- 1 `docs/TIS_MASTER_CONTEXT.md`

docs/adr/0003-paddle-payment-architecture.md

ADR 0003: Paddle Payment Architecture

Status: Accepted

Date: 2026-06-26

Context

TIS requires subscription payments and billing visibility while preserving a clean boundary between academic operations and payment provider behavior.

Decision

Use Paddle behind service/client boundaries in the `saas/` package. Keep payment, pricing, billing, and currency logic separate from academic modules.

Active branch-quantity changes use `saas.subscription_change_service` and durable `SubscriptionChangeRequest` records. TIS requests Paddle previews and never calculates monetary proration. Immediate increases use `prorated_immediately` and prevent applying the provider change when payment fails. Reductions are billed for the next period without an automatic refund; TIS preserves current local capacity until the renewal boundary is webhook-confirmed. Every update sends Paddle the complete retained recurring item list.

Plan upgrades and downgrades use the same provider-authoritative principles through `saas.subscription_plan_change_service`. Upgrades are immediate only after provider-confirmed payment lifecycle evidence; downgrades are scheduled and preserve current entitlements until effective confirmation. Scheduled changes may be replaced or canceled only while Paddle and local request state agree.

Cancellation and reversal use `saas.subscription_cancellation_service`. Cancellation is scheduled at period end, does not revoke current paid access early, and may be reversed before the effective boundary after provider validation. `saas.subscription_lifecycle_service` centralizes displayed lifecycle state and allowed customer actions.

Billing history and invoices remain provider-owned. `saas.billing_history_service` reads subscription-scoped Paddle transactions and requests fresh expiring invoice URLs without persisting a duplicate financial ledger or provider URL.

Paddle does not support arbitrary client-supplied idempotency keys. TIS therefore serializes unresolved requests per subscription, uses deterministic local request keys, locks request/subscription rows during submission, and reconciles webhook retries by provider event and request state.

Alternatives Considered

- 1 Inline Paddle calls directly inside route handlers.
- 1 Store provider-specific behavior across operational modules.
- 1 Delay payment architecture until after provisioning.

Consequences

Positive:

- 1 Provider integration is isolated.
- 1 Academic workflows stay independent of payment details.

- 1 Billing status can be reasoned about separately from onboarding and provisioning.

Tradeoffs:

- 1 Service boundaries must be maintained.
- 1 Webhook/payment state must be carefully tested.

Related Docs / Files

- 1 `saas/paddle_client.py`
- 1 `saas/payment_service.py`
- 1 `saas/billing_service.py`
- 1 `saas/pricing_service.py`
- 1 `saas/currency_service.py`
- 1 `saas/entitlement_service.py`
- 1 `saas/subscription_portal_service.py`
- 1 `saas/subscription_change_service.py`
- 1 `saas/subscription_plan_change_service.py`
- 1 `saas/subscription_cancellation_service.py`
- 1 `saas/subscription_lifecycle_service.py`
- 1 `saas/billing_history_service.py`
- 1 `saas/router.py`
- 1 `docs/TIS_MASTER_CONTEXT.md`

ADR 0004: Webhook-Only Payment Confirmation

Status: Accepted

Date: 2026-06-26

Context

Checkout return pages can be reached by browser redirects, refreshes, or user navigation. They should not be treated as authoritative proof that payment succeeded.

Decision

Treat provider webhook confirmation as the authoritative source for successful payment state. Checkout return pages may inform the user but should not independently confirm payment or trigger provisioning as if payment is verified.

The same authority boundary applies to M7 subscription mutations. Local requests and browser submissions are intent, not final financial truth. Webhook processing validates signatures, provider event identity, subscription/customer ownership, expected items/quantity, attributable transactions, and idempotent event state before confirming local outcomes. Ambiguous or mismatched evidence fails closed to manual review.

Initial checkout and post-activation subscription-change lifecycles remain distinct so a completed change transaction cannot accidentally replay initial provisioning/payment transitions. Guarded reconciliation may repair finalized local lifecycle fields only from attributable stored webhook evidence plus authoritative Paddle transaction data.

Alternatives Considered

- 1 Confirm payment based on checkout return route.
- 1 Allow manual UI confirmation from the SaaS account page.
- 1 Provision tenant records immediately after checkout redirect.

Consequences

Positive:

- 1 Reduces risk of false-positive payment confirmation.
- 1 Aligns billing state with provider-confirmed events.
- 1 Supports delayed provisioning after verified payment.

Tradeoffs:

- 1 Users may need clear pending-payment status while webhook processing completes.
- 1 Webhook handling must be reliable and observable.

Related Docs / Files

- 1 `saas/paddle_client.py`
- 1 `saas/payment_service.py`
- 1 `saas/billing_service.py`

```
1    saas/subscription_change_service.py
1    saas/subscription_plan_change_service.py
1    saas/subscription_cancellation_service.py
1    saas/payment_lifecycle_reconciliation_service.py
1    scripts/reconcile_finalized_payment_lifecycle.py
1    saas/router.py
1    docs/TIS_MASTER_CONTEXT.md
```

docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md

ADR 0005: Delayed Tenant Provisioning After Verified Payment

Status: Accepted

Date: 2026-06-26

Context

Public SaaS onboarding collects organization and branch information before an operational tenant exists. Provisioning creates operational school records and must be safe, reviewable, and recoverable.

Decision

Delay tenant provisioning until payment is verified and the organization is ready for provisioning. Use platform owner provisioning workflows and jobs to create or update operational tenant structures.

Alternatives Considered

- 1 Provision immediately after signup.
- 1 Provision immediately after checkout return.
- 1 Let public users create operational tenants directly.

Consequences

Positive:

- 1 Protects operational tenant data.
- 1 Keeps pending SaaS onboarding separate from live school records.
- 1 Allows platform owner oversight and retry behavior.

Tradeoffs:

- 1 More state transitions exist between signup and operational access.
- 1 Provisioning status must be communicated clearly.

Related Docs / Files

- 1 `saas/provisioning_service.py`
- 1 `saas/router.py`
- 1 `saas/service.py`
- 1 `templates/saas/admin_provisioning.html`
- 1 `templates/saas/admin_pending_organizations.html`
- 1 `docs/TIS_MASTER_CONTEXT.md`

docs/adr/0006-documentation-as-source-knowledge-management-system.md

ADR 0006: Documentation As Source / Knowledge Management System

Status: Accepted

Date: 2026-06-26

Context

TIS is growing across product, SaaS, architecture, payment, provisioning, landing, and operational modules. Future humans and AI coding assistants need reliable context and change history.

Decision

Use Markdown files under docs/ as the source of truth. Generate the PDF booklet as a snapshot. Maintain change history, ADRs, module history, project state, master context, and AI project context as part of every meaningful development task.

Alternatives Considered

- 1 Keep knowledge only in chat history.
- 1 Keep only a generated PDF.
- 1 Let the app rewrite docs automatically.
- 1 Store decisions only in commits.

Consequences

Positive:

- 1 Project knowledge is reviewable and version-controlled.
- 1 Future Codex and ChatGPT conversations can onboard quickly.
- 1 Major decisions are preserved.

Tradeoffs:

- 1 Developers must maintain the KIA workflow.
- 1 Docs and PDF can become stale if the policy is ignored.

Related Docs / Files

- 1 docs/README.md
- 1 docs/DOCUMENTATION_UPDATE_POLICY.md
- 1 docs/CHANGE_HISTORY.md
- 1 docs/AI_PROJECT_CONTEXT.md

- 1 scripts/generate_docs_pdf.py
- 1 static/docs/TIS_Project_Reference_Booklet.pdf

docs/adr/0007-landing-page-visual-system-strategy.md

ADR 0007: Landing Page Visual System Strategy

Status: Accepted

Date: 2026-06-26

Context

The public landing page must communicate trust, product maturity, and academic operations value. It should not be accidentally changed during backend or operational app work.

Decision

Treat the Next.js landing website and its visual system as a separate product surface. Keep marketing content references under docs/marketing/. Do not modify landing design, copy, or legacy FastAPI landing files unless explicitly approved.

Alternatives Considered

- 1 Let backend tasks freely alter landing page presentation.
- 1 Continue using legacy FastAPI landing files as the public source.
- 1 Keep marketing strategy only in informal notes.

Consequences

Positive:

- 1 Protects brand and public conversion strategy.
- 1 Keeps landing work deliberate and reviewable.
- 1 Reduces accidental coupling with operational app changes.

Tradeoffs:

- 1 Landing page changes need explicit planning.
- 1 Developers must check the source-of-truth docs before editing public website files.

Related Docs / Files

- 1 tis-landing-website/
- 1 docs/marketing/landing_page_source_of_truth.md
- 1 docs/marketing/tis_landing_page_master_content.md
- 1 docs/history/landing-page/README.md

docs/history/academic-calendar/README.md

Academic Calendar History

This folder tracks meaningful changes to academic calendar workflows, event types, permissions, exports, and branch/year behavior.

Related files:

- 1 routers/academic_calendar.py
- 1 templates/academic_calendar.html
- 1 permission_registry.py

docs/history/engineering-handbook/2026-06-26-kms-v3-engineering-handbook.md

2026-06-26 - KMS v3 Engineering Handbook

Module: Engineering Handbook

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Added KMS v3.0 Engineering Handbook

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for KMS v3.0 Phase 3A only.

Previous Documented State

The KMS had master context, project state, ADRs, module history, AI context, generated PDF, manifest, and Platform Owner Knowledge Center docs. It did not yet contain a complete engineering handbook for module ownership, repository architecture, and end-to-end flows.

New Documented State

The KMS includes docs/engineering/ with a module map, repository architecture guide, user/system flows, and developer onboarding index. The generated PDF now includes these docs as part of documentation version 3.0.

Reason For Change

New human developers and future Codex/ChatGPT conversations need a practical engineering handbook, not only a bundle of source documents.

User / Business Impact

Improves continuity, onboarding speed, technical review quality, and safer future implementation work.

Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page, database, migration, route, or runtime behavior changed.

Documentation Updated

- 1 docs/engineering/README.md
- 1 docs/engineering/TIS_MODULE_MAP.md
- 1 docs/engineering/REPOSITORY_ARCHITECTURE.md
- 1 docs/engineering/USER_AND_SYSTEM_FLOWS.md
- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/AI_PROJECT_CONTEXT.md
- 1 docs/PROJECT_STATE.md
- 1 docs/README.md
- 1 docs/CHANGE_HISTORY.md

PDF Regenerated

Yes

Follow-Up Needed

Review PDF readability and handbook completeness before any Phase 3B work.

docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3b-engineering-layers.md

2026-06-26 - KMS v3 Phase 3B Engineering Layers

Module: Engineering Handbook

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Added KMS v3.0 Phase 3B Engineering Layers

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for KMS v3.0 Phase 3B only.

Previous Documented State

The engineering handbook included the module map, repository architecture guide, user/system flows, and onboarding index. It did not yet include a database architecture overview, development standards, UI/UX philosophy, or product roadmap.

New Documented State

The engineering handbook includes Phase 3B layers:

- 1 database architecture overview,
- 1 development standards,
- 1 UI/UX design philosophy,
- 1 product roadmap,
- 1 stronger AI/human developer onboarding guidance.

Reason For Change

New senior developers, Codex conversations, ChatGPT conversations, and technical reviewers need stronger system boundaries, design expectations, roadmap context, and development rules before changing TIS.

User / Business Impact

Improves implementation safety, onboarding quality, technical review quality, and continuity across future development sessions.

Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page code, database, migration, route, or runtime behavior changed.

Documentation Updated

- 1 docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
- 1 docs/engineering/DEVELOPMENT_STANDARDS.md
- 1 docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md

- 1 docs/engineering/PRODUCT_ROADMAP.md
- 1 core KMS index/context files

PDF Regenerated

Yes

Follow-Up Needed

Review handbook completeness and readability before any Phase 3C work.

docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3c-governance-ai-traceability.md

2026-06-26 - KMS v3 Phase 3C Governance And AI Traceability

Module: Engineering Handbook

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Added KMS v3.0 Phase 3C Governance And AI Traceability

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for KMS v3.0 Phase 3C only.

Previous Documented State

The engineering handbook included module maps, repository architecture, user/system flows, database architecture, development standards, UI/UX philosophy, and roadmap. It did not yet document rejected decisions, visual documentation standards, definitive AI guidance, governance, or decision traceability.

New Documented State

The engineering handbook includes Phase 3C layers:

- 1 rejected architectural decisions,
- 1 visual documentation framework,
- 1 AI optimization guide,
- 1 project governance,
- 1 engineering decision traceability.

Reason For Change

Future developers and AI assistants need to understand why TIS became what it is, not only its current structure.

User / Business Impact

Improves continuity, reduces repeated architectural debates, improves AI readiness, and clarifies governance and review expectations.

Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page code, database, migration, route, or runtime behavior changed.

Documentation Updated

- 1 docs/engineering/REJECTED_DECISIONS.md
- 1 docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md
- 1 docs/engineering/AI_OPTIMIZATION_GUIDE.md

- 1 docs/engineering/PROJECT_GOVERNANCE.md
- 1 core KMS index/context files

PDF Regenerated

Yes

Follow-Up Needed

Future KMS improvements may add screenshots, diagrams, deployment runbooks, route inventories, test strategy docs, and deeper module guides. Do not implement those without a later approved phase.

docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3d-lifecycle-foundation.md

2026-06-26 - KMS v3 Phase 3D Lifecycle Foundation

Module: Engineering Handbook

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Completed KMS v3.0 Phase 3D Lifecycle Foundation

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved as KMS v3.0 Phase 3D final phase.

Previous Documented State

The engineering handbook documented module maps, repository architecture, flows, database architecture, standards, UI/UX philosophy, roadmap, rejected decisions, visual documentation, AI optimization, and governance. It did not yet define a full self-evolving lifecycle from task through deployment.

New Documented State

The engineering handbook now includes:

- 1 knowledge lifecycle,
- 1 documentation automation guide,
- 1 formal KIA standard,
- 1 self-evolving workflow,
- 1 documentation dependency map,
- 1 AI coding workflow,
- 1 future automation roadmap.

This completes the KMS v1.0 lifecycle foundation.

Reason For Change

Future implementation work needs a repeatable lifecycle that keeps KMS synchronized with software changes without automatic source-doc rewriting.

User / Business Impact

Improves long-term governance, reviewer confidence, AI coding discipline, and continuity across future phases.

Technical Impact

Documentation and PDF generation only. No SaaS flow, landing code, database model, migration, `tis.db`, app route, or runtime behavior changed.

Documentation Updated

- 1 docs/engineering/KNOWLEDGE_LIFECYCLE.md

- 1 docs/engineering/DOCUMENTATION_AUTOMATION.md
- 1 docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md
- 1 docs/engineering/SELF_EVOLVING_WORKFLOW.md
- 1 docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md
- 1 docs/engineering/AI_CODING_WORKFLOW.md
- 1 docs/engineering/FUTURE_AUTOMATION_ROADMAP.md
- 1 core KMS index/context files

PDF Regenerated

Yes

Follow-Up Needed

Review the final handbook for readability. Future improvements should be handled as new approved KMS phases.

docs/history/engineering-handbook/2026-06-27-production-memory-stability-guardrails.md

2026-06-27 - Production Memory Stability Guardrails

Module: Engineering Handbook and operational app stability

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-27 - Added Production Memory Stability Guardrails

Related ADRs: None. This is a standards and operational guardrail update, not a new architecture decision.

Reviewer/approval notes: Prepared after a Render restart/502 investigation. Local changes only; no commit, push, migration, database change, or deployment was performed.

Previous Documented State

The KMS required scoped changes, tenant isolation, validation, and documentation updates, but it did not explicitly define a production memory budget or prohibit common memory hazards such as:

- 1 full large-dataset parsing for normal picker requests,
- 1 duplicate template rendering in production routes,
- 1 warning-level diagnostic stage logs during normal traffic,
- 1 unbounded global caches,
- 1 startup-heavy work without memory review.

New Documented State

docs/engineering/DEVELOPMENT_STANDARDS.md now includes a mandatory Production Memory and Render Stability section.

The standards require future work to:

- 1 treat 512 MB Render memory as a hard production budget,
- 1 avoid full large-dataset loads when scoped or streaming lookup is available,
- 1 avoid duplicate template renders in production request paths,
- 1 keep diagnostics opt-in through explicit environment flags,
- 1 filter data before materializing result sets,
- 1 keep caches bounded and justified,
- 1 include memory/performance smoke checks for memory-sensitive changes.

Reason For Change

The 2026-06-27 production investigation found avoidable memory pressure risks after deployment:

- 1 /observations/ had diagnostic logging and extra template pre-renders in the normal route path.
- 1 Global location lookup could parse a 47 MB country/state/city reference dataset into a complete in-memory index for simple picker API calls.

These patterns can trigger process restarts and temporary 502 responses on constrained production instances.

User / Business Impact

The rule reduces the risk that normal navigation causes Render restarts, temporary 502s, or user-visible instability. It also creates a clear review standard for future SaaS onboarding, dashboard, reports, observations, and large-data changes.

Technical Impact

Future changes that add broad data loading, reference datasets, exports, diagnostics, or startup work must include memory-conscious design and validation. The local stabilization patch also reduces current risk by gating observation diagnostics and changing location lookup to scoped loading.

Documentation Updated

- 1 docs/engineering/DEVELOPMENT_STANDARDS.md
- 1 docs/PROJECT_STATE.md
- 1 docs/AI_PROJECT_CONTEXT.md
- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/CHANGE_HISTORY.md

PDF Regenerated

Yes

Follow-Up Needed

Review, commit, and deploy the stabilization changes when approved. After deployment, monitor Render memory, restart count, warning logs, and 502 frequency.

docs/history/engineering-handbook/2026-07-21-kms-automatic-enforcement.md

2026-07-21 - Automatic KMS Synchronization Enforcement

Previous State

KMS governance was documented but not mechanically enforced. Developers and AI assistants manually decided impact, edited Markdown, and regenerated artifacts. The Knowledge Center could detect stale hashes, but no pull-request or deployment gate consumed that status.

New State

TIS has a narrow repository enforcement layer:

- 1 `AGENTS.md` makes KMS onboarding and completion rules default for Codex.
- 1 `.kms-impact.yml` records task-level impact, affected areas, updated KMS files, and controlled no-impact overrides.
- 1 `scripts/check_kms_impact.py` compares the declaration with a Git range or local worktree, classifies likely major changes, and validates generated artifacts.
- 1 `scripts/generate_docs_pdf.py --check` verifies source coverage, source hashes, PDF hash/size, version, and manifest consistency without writing.
- 1 GitHub Actions run enforcement for pull requests and dev; production deployment from `master` depends on the same gate.

Cross-Platform Correction

The initial enforcement release exposed two checkout-dependent differences: Markdown hashes reflected raw CRLF/LF bytes, and dynamic source ordering followed native path comparison. The generator and Knowledge Center now hash normalized UTF-8/LF Markdown. The generator also records repository-relative POSIX paths in a deterministic order while continuing to fail on genuine source-list drift. Impact detection includes deleted files so removing behavior or documentation cannot bypass review.

Task-Boundary Correction

The initial push workflow compared only the previous pushed commit with the new head, while pull requests compared the complete feature branch. Follow-up commits therefore validated a task-level declaration against a commit-level diff. Push checks now derive the task base from the merge base of the default branch and pushed head. Pull-request and push checks consequently enforce the same cumulative KIA scope without allowing undeclared documentation changes.

Phase 6 Unified Command

The initial automation exposed generation, artifact checking, and KIA enforcement as separate commands. Phase 6 adds `scripts/kms.py sync` for KIA preflight, PDF/manifest generation, post-generation freshness validation, and completion reporting. `scripts/kms.py check` is the canonical read-only command used locally and by CI. Both commands delegate to the existing generator and checker; they do not duplicate policy or rewrite Markdown.

Guardrails

Automation does not generate or rewrite Markdown prose. Reviewed Markdown remains authoritative. KMS files must describe engineering design only and must not contain customer, organization, personal, transaction, invoice, production identifier, webhook payload, secret, credential, environment, or database-row data.

Related Files

```
1 AGENTS.md
1 .kms-impact.yml
1 scripts/check_kms_impact.py
1 scripts/generate_docs_pdf.py
1 scripts/kms.py
1 .github/workflows/kms-enforcement.yml
1 .github/workflows/deploy-on-master.yml
1 docs/DOCUMENTATION_UPDATE_POLICY.md
```

docs/history/engineering-handbook/README.md

Engineering Handbook History

This folder tracks meaningful changes to the TIS Engineering Handbook, including module maps, repository architecture, user/system flows, and onboarding guidance for humans and AI coding conversations.

Related files:

- 1 docs/engineering/README.md
- 1 docs/engineering/TIS_MODULE_MAP.md
- 1 docs/engineering/REPOSITORY_ARCHITECTURE.md
- 1 docs/engineering/USER_AND_SYSTEM_FLOWS.md

docs/history/landing-page/README.md

Landing Page History

This folder tracks meaningful changes to the public landing page, marketing positioning, visual system strategy, and source-of-truth boundaries.

Related docs:

- 1 docs/adr/0001-separate-nextjs-landing-website.md
- 1 docs/adr/0007-landing-page-visual-system-strategy.md
- 1 docs/marketing/landing_page_source_of_truth.md
- 1 docs/marketing/tis_landing_page_master_content.md

docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md

2026-06-26 - Platform Owner Knowledge Center

Module: Platform Knowledge Center

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Added Platform Owner Knowledge Center

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for Phase 2C. Regenerate button, public access, SaaS changes, database changes, migrations, landing page changes, commits, and pushes remain out of scope.

Previous Documented State

TIS had KMS Markdown source files, ADRs, module history, a generated PDF booklet, and a manifest, but there was no protected in-app owner utility for viewing KMS status or accessing the booklet.

New Documented State

TIS has a read-only Platform Owner Knowledge Center that shows KMS health, manifest metadata, PDF freshness, source document status, recent change-history entries, ADRs, module history areas, and the Knowledge Impact Assessment checklist.

Reason For Change

Platform owners need a protected internal view of KMS status and a safe way to view/download the generated PDF without linking directly to static public paths.

User / Business Impact

Platform owners can verify whether the KMS snapshot is current and access the reference booklet from inside the app.

Technical Impact

Adds read-only KMS service helpers, owner-protected FastAPI routes, one app-shell template, and an owner-only Platform Console card. No SaaS, database, migration, billing, provisioning, or landing page behavior changes.

Documentation Updated

- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/PROJECT_STATE.md
- 1 docs/CHANGE_HISTORY.md
- 1 docs/AI_PROJECT_CONTEXT.md
- 1 docs/history/platform-knowledge/README.md
- 1 docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md

PDF Regenerated

Yes

Follow-Up Needed

Consider a future explicit owner-only regenerate action after review. The app must not silently rewrite Markdown source docs.

docs/history/platform-knowledge/README.md

Platform Knowledge Center History

This folder tracks meaningful changes to the owner-only TIS Knowledge Center, protected documentation PDF access, KMS metadata display, freshness detection, and knowledge health reporting.

Related files:

```
1    knowledge_service.py
1    templates/platform_knowledge_center.html
1    templates/platform_console.html
1    main.py
```

docs/history/provisioning/2026-06-26-kms-foundation.md

2026-06-26 - KMS Foundation

Module: Documentation and knowledge management

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Established Knowledge Management System Foundation

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for Phase 2A and Phase 2B only.

Previous Documented State

TIS had Phase 1 documentation source files and a generated PDF booklet, but no formal KMS policy, ADR structure, module history foundation, or AI onboarding context.

New Documented State

TIS now has a KMS foundation with policy, change history, ADRs, module history folders, and AI project context. The PDF generator includes these source docs and produces manifest metadata.

Reason For Change

Future human developers, Codex conversations, ChatGPT conversations, project owners, platform owners, and reviewers need durable project context and history.

User / Business Impact

Improves continuity, reduces repeated rediscovery, and makes project decisions easier to review.

Technical Impact

No runtime app behavior changed. This is documentation and PDF generation foundation only.

Documentation Updated

- 1 docs/CHANGE_HISTORY.md
- 1 docs/DOCUMENTATION_UPDATE_POLICY.md
- 1 docs/AI_PROJECT_CONTEXT.md
- 1 docs/adr/
- 1 docs/history/
- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/PROJECT_STATE.md
- 1 docs/README.md

PDF Regenerated

Yes

Follow-Up Needed

Phase 2C can later add a protected Platform Owner Knowledge Center after review.

docs/history/provisioning/README.md

Provisioning History

This folder tracks meaningful changes to pending organizations, provisioning jobs, verified-payment readiness, retry behavior, and platform owner provisioning oversight.

Related docs:

- 1 docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md
- 1 docs/TIS_MASTER_CONTEXT.md

Related files:

- 1 saas/provisioning_service.py
- 1 saas/router.py
- 1 templates/saas/admin_provisioning.html

docs/history/README.md

TIS Module History

Module history stores deeper area-specific change records. It complements docs/CHANGE_HISTORY.md, which remains the chronological summary.

Use module history when a meaningful area changes and the previous documented state should be preserved.

Module Areas

- 1 subscriptions/
- 1 landing-page/
- 1 academic-calendar/
- 1 workforce-planning/
- 1 saas-onboarding/
- 1 provisioning/

Entry Template

```
# YYYY-MM-DD - Change Title

Module:
Related change-history entry:
Related ADRs:
Reviewer/approval notes:

## Previous Documented State

## New Documented State

## Reason For Change

## User / Business Impact

## Technical Impact

## Documentation Updated

## PDF Regenerated

## Follow-Up Needed
```

docs/history/saas-onboarding/README.md

SaaS Onboarding History

This folder tracks meaningful changes to signup, login, account, organization onboarding, contacts, branches, academic setup, review, and account self-service.

2026-07-16 - M7 Phase 2 Read-Only Subscription Management Portal

Verified SaaS customers now have a read-only Subscription Management page at `/saas/subscription`. The page presents the confirmed plan, safe subscription health, billing interval, next billing date, paid and active branch quantities, remaining capacity, grouped feature entitlements, and a compact plan comparison. All commercial data is supplied by `saas.entitlement_service`; the portal does not query Paddle or independently calculate subscription state or branch capacity.

Upgrade, branch-addition, billing-history, invoice, pending-change, and subscription-management controls are visible only as disabled Coming Soon options. This phase adds no subscription mutations, Paddle calls, proration, refunds, cancellation, or plan and quantity editing.

2026-07-16 - M7 Phase 1 Subscription Entitlement Foundation

Commercial access now resolves through `saas.entitlement_service` from the provisioned SchoolGroup's tenant link, paid operational contract, and one confirmed active Paddle subscription. Onboarding selections, checkout quotes, pending payment attempts, and page values are not entitlement sources. Missing, mismatched, or ambiguous subscription relationships fail closed as `manual_review`.

The initial normalized matrix authoritatively maps only existing plan metadata: Enterprise AI receives `module.ai`; Professional and Enterprise AI receive `feature.advanced_reporting`; Starter does not receive either. Paid active-branch capacity is derived only from `PaymentSubscription.quantity`. Teacher management, branch management, observations, hiring, core reporting, general exports, audit logs, and cross-branch reporting remain `owner_approval_required` until commercial rules are approved.

Pilot enforcement is limited to allocation-plan PDF/XLSX exports. Both the existing `reports.export` user permission and `feature.advanced_reporting` subscription entitlement must succeed. Platform Owner and Developer identities do not bypass subscription entitlements and must operate in a selected tenant scope.

Upgrades, downgrades, proration, refunds, Paddle subscription changes, branch-specific plans, and customer subscription-management UI remain later M7 work.

2026-07-14 - M6 Phase 3 Abandoned Draft Cleanup

Automatic cleanup applies only to unpaid, unprovisioned SaaS drafts. A draft becomes eligible after the globally configured inactivity period (30 days by default) and only after its final reminder was sent successfully for the current activity cycle. Any later meaningful activity restarts the lifecycle and prevents deletion.

Before deleting, the processor locks and rechecks each account and pending organization in its own transaction. Payment success or processing, subscription evidence, provisioning or tenant links, operational identities, protected accounts, shared ownership, and unresolved provider relationships block cleanup. Ambiguous records are retained for manual review. Successful payment evidence, Paddle webhooks and remote Paddle records, global plans/prices, reference data, and unrelated accounts or tenants are always preserved.

Run the bounded processor from the repository root:

```
PYTHONPATH=. python scripts/process_abandoned_draft_cleanup.py --dry-run --batch-size 100
PYTHONPATH=. python scripts/process_abandoned_draft_cleanup.py --batch-size 100
```

```
PYTHONPATH=. python scripts/process_abandoned_draft_cleanup.py --dry-run --account-  
email draft@example.com
```

--max-inactivity-days is available only for local testing and is rejected in production-like environments. For Render, configure a daily Cron Job using the deployed service environment and DATABASE_URL; dry-run should be used before enabling the live command. Each account commits independently, failures roll back completely, concurrent workers skip locked rows, and durable external audit events record deleted, skipped, manual-review, recovery, and rolled-back outcomes.

Platform Owner lifecycle analytics remain future M6 scope.

2026-07-14 - M6 Phase 2 Draft Onboarding Reminder Engine

Draft retention remains inactivity-based. The reminder engine sends at most one first, second, and final reminder per activity cycle using the globally configured lifecycle thresholds (defaults: 24 hours, 7 days, and 25 days). The final reminder shows the deletion-eligibility date derived from the configured retention period (default: 30 days). Meaningful customer activity continues to flow through `draft_lifecycle_service.record_meaningful_activity(...)`, which starts a new reminder cycle.

Run the bounded processor from the repository root:

```
PYTHONPATH=. python scripts/process_draft_reminders.py --batch-size 100  
PYTHONPATH=. python scripts/process_draft_reminders.py --dry-run  
PYTHONPATH=. python scripts/process_draft_reminders.py --stage final
```

For Render, use a Cron Job with the service's DATABASE_URL, RESEND_API_KEY, EMAIL_FROM, EMAIL_REPLY_TO, and TIS_PUBLIC_BASE_URL. TIS_SUPPORT_EMAIL is optional and falls back to EMAIL_REPLY_TO in reminder content. An hourly schedule is recommended. PostgreSQL row locking prevents overlapping workers from sending the same reminder stage.

Automatic draft deletion and Platform Owner lifecycle analytics are not enabled in Phase 2.

2026-06-27 - Subscription And Workspace Activation Guided Journey Phase 3C

Phase 3C subscription, payment, and activation page redesign is accepted.

What changed:

- 1 Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages now use the Phase 3A shared shell and Phase 3B guided style.
- 1 Each page now has one shared-shell primary CTA and keeps secondary actions visually secondary.
- 1 Secure Payment pages now clearly explain that browser return from checkout does not itself confirm payment.
- 1 Subscription and activation status pages now use concise customer-safe cards for payment, subscription, activation, and TIS Platform access state.
- 1 Customer-facing pages now consistently explain that TIS Platform access becomes available after Workspace Activation.

Scope notes:

- 1 Payment behavior, billing behavior, provisioning behavior, webhook logic, checkout start/launch behavior, stored statuses, database schema, migrations, operational modules, the Next.js landing website, OAuth behavior, internal /saas route names, and admin views were not changed.

Related files:

- 1 `saas/router.py`


```
1 templates/saas/plan_selection.html
1 templates/saas/checkout_summary.html
1 templates/saas/checkout_return.html
1 templates/saas/checkout_cancel.html
1 templates/saas/account_billing.html
1 templates/saas/billing_status.html
1 tests/test_saas_phase1.py
```

2026-06-27 - School Workspace Setup Guided Wizard Phase 3B

Phase 3B School Workspace Setup onboarding page redesign is accepted.

What changed:

- 1 Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup now use a consistent guided wizard structure on top of the Phase 3A shared shell.
- 1 Each onboarding page now has one shared-shell primary CTA and keeps Back/Save Draft actions visually secondary.
- 1 Organization Profile groups identity, logo upload, program/location, and estimated scale fields.
- 1 Branch Setup uses compact branch panels instead of heavy repeated blank blocks.
- 1 Academic Setup and Primary Contact use focused single-step sections with concise guidance.
- 1 Review School Workspace Setup now presents a clearer ready-to-continue summary before Subscription Selection.

Scope notes:

- 1 Form actions, field names, routes, validation behavior, draft behavior, onboarding progression, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, OAuth behavior, internal /saas route names, and admin views were not changed.
- 1 Subscription/payment/status pages remain future Phase 3 work.

Related files:

```
1 saas/router.py
1 templates/saas/base.html
1 templates/saas/onboarding_organization.html
1 templates/saas/onboarding_branches.html
1 templates/saas/onboarding_academic_setup.html
1 templates/saas/onboarding_contacts.html
1 templates/saas/onboarding_review.html
1 tests/test_saas_phase1.py
```

2026-06-27 - TIS Account Guided Setup Framework Phase 3A

Phase 3A shared guided setup framework is accepted.

What changed:

- 1 The shared customer account shell now supports a guided setup console when setup context is provided.
- 1 The TIS Account page now uses an 8-step customer journey: TIS Account, Email Verification, School Workspace Setup, Review & Confirmation, Subscription Selection, Secure Payment, Workspace Activation, and Enter TIS Platform.
- 1 The account page now focuses on the current step, one primary next action, concise account/workspace context, and guidance that TIS Platform access becomes available after Workspace Activation.
- 1 The old customer account dashboard statistics and session detail blocks were removed from the account landing page.
- 1 Journey state is calculated from existing account, onboarding, billing, payment, and activation data without changing stored statuses.

Scope notes:

- 1 Onboarding forms, subscription/payment pages, billing/status pages, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, internal /saas route names, admin views, and Google/Microsoft login were not changed.
- 1 This phase prepares the shared framework for later Phase 3 onboarding and payment page redesign work.

Related files:

- 1 `saas/router.py`
- 1 `saas/service.py`
- 1 `templates/saas/base.html`
- 1 `templates/saas/account.html`
- 1 `tests/test_saas_phase1.py`

2026-06-27 - TIS Account Customer-Facing Wording And Logo Cleanup

Phase 2 customer-facing wording cleanup is accepted.

What changed:

- 1 Customer account and school workspace setup pages now use professional labels such as "TIS Account", "Account Dashboard", "School Workspace Setup", "Organization Profile", "Branch Setup", "Academic Setup", "Subscription Setup", "Secure Payment", and "Workspace Activation".
- 1 Customer views use customer-safe display labels for internal onboarding, billing, payment, and activation statuses instead of exposing raw tenant/provisioning/checkout status values.
- 1 Customer-facing billing and subscription views hide provider transaction/subscription IDs, attempt UUIDs, checkout session internals, plan IDs, and school group IDs.
- 1 The shared customer account shell now includes the official full-color horizontal TIS logo image so inherited customer account/setup pages carry official branding.
- 1 TIS Account transactional emails use an existing official dark-blue TIS wordmark asset.
- 1 Activation email copy now uses "School Workspace", "Workspace Activation", and "TIS Account" language.

Scope notes:

- 1 Internal /saas route/module/model names and stored statuses were not renamed.

- 1 Payment, billing, provisioning behavior, database schema, migrations, operational modules, and the Next.js landing website were not changed.
- 1 Google/Microsoft login remains future work and was not implemented.
- 1 Phase 3 account setup UI redesign was not implemented.

Related files:

- 1 `saas/router.py`
- 1 `saas/service.py`
- 1 `saas/provisioning_service.py`
- 1 `email_templates.py`
- 1 `templates/saas/`
- 1 `tests/test_saas_phase1.py`
- 1 `tests/test_saas_phase5.py`
- 1 `tests/test_email_templates.py`

2026-06-27 - TIS Account Email Verification Recovery And Setup Gate

Phase 1 TIS Account email verification recovery is accepted.

What changed:

- 1 Valid email verification links now mark the SaaS account email verified/active and redirect to TIS Account login with a professional success notice.
- 1 Expired or invalid verification links now show a recovery page with a resend verification option.
- 1 Resend verification supports unverified accounts, already verified accounts, and unknown-email cases with safe customer-facing messaging that does not reveal account existence.
- 1 Password-based accounts that remain unverified are blocked from starting or continuing school workspace setup.
- 1 New visible wording in this verification flow uses "TIS Account" and "school workspace setup".

Scope notes:

- 1 Payment, billing, provisioning, database schema, migrations, operational modules, and the Next.js landing website were not changed.
- 1 Google/Microsoft login remains future work and was not implemented.
- 1 Phase 2 customer-facing wording cleanup and Phase 3 account setup UI redesign were not implemented as part of this change.

Related files:

- 1 `saas/router.py`
- 1 `saas/service.py`
- 1 `templates/saas/`
- 1 `docs/adr/0002-separate-saas-identity-and-operational-users.md`

[docs/history/subscriptions/README.md](#)

Subscription History

2026-07-20 - M7 Phase 5 Lifecycle Policy, Cancellation, And Reversal

The Subscription Management portal now resolves customer-visible state and allowed actions through `saas.subscription_lifecycle_service`. Upgrade, downgrade, branch increase/reduction, cancellation, and cancellation reversal controls are exposed only when authorization, provider subscription state, pending requests, effective dates, and local relationships permit them.

Authorized billing administrators can schedule cancellation at period end and reverse it before the effective boundary. Current paid access remains active until authoritative provider evidence confirms the end of the paid period. Provider conflicts, unknown outcomes, and local/provider mismatches fail closed to manual review.

2026-07-18 - M7 Payment Lifecycle And Reconciliation Protections

Initial checkout and post-activation subscription-change transactions are resolved as distinct payment lifecycles. Webhook processing remains signature-verified and idempotent, validates attributable provider subscription/transaction evidence, and prevents change transactions from replaying initial provisioning transitions.

Diagnostics and the sandbox-guarded finalized-lifecycle reconciliation script expose sanitized evidence and default to dry-run behavior. Reconciliation applies only when stored completed webhook evidence and authoritative Paddle transaction data agree; unexpected or ambiguous state is blocked rather than guessed.

2026-07-17 - M7 Phase 4 Plan Upgrades And Scheduled Downgrades

Authorized billing administrators can preview and submit plan transitions from `/saas/subscription`. Paddle supplies monetary previews and proration outcomes. Upgrades use immediate provider proration with payment-failure prevention and do not become local entitlement truth before verified completion. Downgrades are scheduled for the next billing period and preserve the current plan until provider/webhook confirmation at the effective boundary.

Customers can cancel or replace an eligible scheduled plan change before it becomes effective. TIS sends complete retained recurring items and enters manual review on provider/local ambiguity.

2026-07-20 - M7 Phase 6 Billing History and Invoice Management

The customer Subscription Management page now retrieves billing history directly from Paddle's paginated `GET /transactions` API, scoped to the customer's confirmed provider subscription. TIS displays provider transaction totals, statuses, origins, invoice numbers, and returned credit/refund adjustments without creating local financial records or calculating replacement amounts.

Eligible invoice downloads use Paddle's `GET /transactions/{transaction_id}/invoice` API. TIS reauthorizes the current billing user, resolves the invoice against the confirmed customer subscription, and requests a fresh expiring provider URL at download time. The provider URL is not stored locally. No billing-history cache, schema change, migration, or webhook behavior is introduced; temporary provider failures render customer-safe retry states.

2026-07-16 - M7 Phase 3 Active Branch Quantity Management

Authorized organization billing administrators can now preview and submit paid branch-quantity changes from `/saas/subscription`. Paddle remains the sole source of financial calculations: TIS sends the complete retained subscription item list to Paddle's subscription-update preview endpoint and stores only customer-safe charge, credit, net, recurring-total, and effective-date summaries.

Increases use prorated_immediately with on_payment_failure=prevent_change. Local paid capacity does not increase until verified subscription.updated and successful transaction.completed evidence confirms the requested quantity and subscription-update payment. Reductions use prorated_next_billing_period, issue no immediate refund, and remain locally scheduled until a renewal-boundary subscription webhook confirms the reduced quantity. Reductions below active operational branch usage are rejected, and scheduled reductions can be restored before their effective date.

PaymentSubscription.quantity remains the only entitlement-capacity authority. Branch creation and individual or bulk reactivation now fail closed for provisioned SaaS tenants when confirmed paid capacity is exhausted. Provider mismatches, unsupported state, stale previews, or ambiguous ownership enter a customer-safe blocked/manual-review path without exposing provider diagnostics.

This folder tracks meaningful changes to TIS subscription plans, pricing, billing status, payment behavior, checkout assumptions, and provider boundaries.

Related docs:

- 1 docs/adr/0003-paddle-payment-architecture.md
- 1 docs/adr/0004-webhook-only-payment-confirmation.md
- 1 docs/TIS_MASTER_CONTEXT.md

2026-07-11 - Paddle Transaction Payment Launcher

Paddle transaction checkout now uses a dedicated public SaaS payment launcher page at /saas/payment instead of the app root or operational login page. Server-side checkout still creates Paddle transactions through the existing payment service and still redirects to Paddle's returned transaction.checkout.url; PADDLE_CHECKOUT_BASE_URL should point to https://app.tisplatform.com/saas/payment so Paddle appends _ptxn to the launcher page.

The launcher page loads Paddle.js from the official Paddle CDN, initializes Paddle with the public PADDLE_CLIENT_TOKEN, uses PADDLE_ENVIRONMENT for sandbox/live mode, reads _ptxn, and opens checkout for the transaction. It does not require SaaS or operational login, does not expose PADDLE_API_KEY, and does not change webhook-confirmed payment state, subscription activation, provisioning, pricing, or billing transitions.

2026-06-30 - Paddle Initial Checkout Price Mapping Configuration

Initial Paddle checkout now has a script-based configuration path for mapping TIS subscription plan prices to Paddle provider price IDs. The runtime source of truth remains subscription_plan_prices.provider_price_id; Paddle credentials and endpoints remain environment variables.

Added configuration support:

- 1 scripts/sync_paddle_price_ids.py
- 1 config/paddle/paddle_prices.sandbox.example.json
- 1 config/paddle/paddle_prices.production.example.json

The sync script validates the six required plan/interval mappings: Starter monthly, Starter annual, Professional monthly, Professional annual, Enterprise AI monthly, and Enterprise AI annual. Real local, sandbox, and production mapping files are ignored by Git so sandbox and live Paddle price IDs remain separated.

If a selected plan price still lacks a Paddle provider price ID, checkout remains fail-closed before calling Paddle. Customers see a support-oriented Secure Payment message while internal diagnostics retain plan code, billing interval, and currency context.

docs/history/workforce-planning/README.md

Workforce Planning History

This folder tracks meaningful changes to teacher information, workload planning, staffing gaps, planning sections, timetable relationships, and related reports.

Related files:

```
1 routers/teachers.py
1 routers/planning.py
1 routers/timetable.py
1 teacher_capacity.py
1 timetable_logic.py
```

docs/marketing/landing_page_source_of_truth.md

Landing Page Source of Truth

The official source of truth for the public TIS landing website is:

`tis-landing-website/`

Public Website

- 1 **Domain:** `https://tisplatform.com`
- 1 **Runtime:** Next.js / Node
- 1 **Local testing URL:** `http://localhost:3000`

All future marketing landing page changes must be made inside `tis-landing-website/`.

Application Portal

The TIS application portal remains separate from the public landing website:

- 1 **Domain:** `https://app.tisplatform.com`
- 1 **Runtime:** FastAPI / Python with PostgreSQL

Legacy Landing Page Files

The former FastAPI/Jinja landing page files are now legacy:

- 1 `templates/landing.html`
- 1 `static/landing/landing.css`

Codex and other developers must not modify these legacy files unless explicitly instructed.

docs/marketing/tis_landing_page_master_content.md

TIS Landing Page Master Content Approved Marketing Foundation

1 Main Positioning

TIS is a developing SaaS academic operations platform designed to connect school leadership, supervisors, teachers, branches, calendars, observations, staffing data, and decision-making inside one integrated system. The platform helps schools organize teacher information, analyze teaching loads, identify staffing gaps, coordinate shared academic calendars, manage structured observation cycles, support supervisor-teacher communication, and build clearer visibility across academic operations. As TIS continues to grow, advanced AI-powered features will be introduced through subscription-based plans, including curriculum-driven assessment generation, academic analytics, planning support, and intelligent recommendations based on verified school data.

1 Hero Section

Hero Badge Built for Schools Moving Beyond Spreadsheets Main Headline When Academic Decisions Are Scattered, Schools Lose Time, Clarity, and Control. Subheadline TIS brings teacher data, workloads, staffing gaps, academic calendars, observations, and leadership decisions into one connected SaaS platform - helping schools see what is happening, what is missing, and what needs action. CTA Buttons Request Early Access See How TIS Works Small Supporting Line A developing academic operations platform with advanced AI capabilities planned through subscription-based growth.

1 Problem Section

Section Label The Problem Main Heading Schools Are Not Short of Data. They Are Drowning in Disconnected Files. Section Text In many schools and educational organizations, every academic process becomes another spreadsheet. Teacher loads, subject coverage, observations, calendars, reports, staffing needs, and follow-up tasks are often managed in separate files, created by different people, in different formats. Over time, this creates a hidden operational burden. Staff members spend more time updating files than understanding the data. Leaders receive information late, inconsistently, or from multiple disconnected sources. In multi-branch organizations, the problem becomes even larger: every branch may work differently, every file may tell a different story, and every decision requires extra checking. Problem Cards

1 Too Many Spreadsheets

Academic work is scattered across different files, formats, and personal tracking systems.

1 Inconsistent Data

Each person or branch may structure information differently, making comparison and reporting difficult.

1 Heavy Staff Workload

Teachers, supervisors, and coordinators carry the burden of repeatedly entering and updating data.

1 Delayed Visibility

Leaders often discover staffing gaps, missing coverage, or academic issues after the problem has already grown.

1 Disconnected Decisions

Calendars, observations, staffing, reports, and academic planning are managed separately instead of working as one connected system.

1 Solution Section

Section Label The Solution Main Heading One Connected Platform to Bring Academic Operations Back Under Control.
Section Text TIS brings the scattered pieces of school operations into one structured academic ecosystem. Instead of chasing separate spreadsheets, comparing different file formats, and waiting for updates from multiple sources, school leaders can work from a clearer, more connected view of what is happening across their school or organization. Teacher data, workloads, subject coverage, academic calendars, observations, supervision follow-up, branch structures, and leadership decisions become part of one system. This allows schools to reduce manual confusion, improve visibility, standardize academic processes, and make decisions with greater confidence. For multi-branch organizations, TIS helps create consistency across campuses while still allowing each branch to manage its own academic operations within a secure, organized, and tenant-isolated platform. **Solution Cards**

1 One Source of Academic Truth

Bring teacher data, workloads, observations, calendars, and staffing information into one connected system.

1 Clearer Leadership Visibility

Help principals, academic leaders, and supervisors see what is happening, what is missing, and what requires action.

1 Standardized Academic Processes

Reduce personal spreadsheet formats and create a more consistent way to manage academic operations across branches.

1 Smarter Staffing Decisions

Identify teaching load issues, uncovered hours, subject gaps, and staffing needs before they become larger problems.

1 Connected School Communication

Support better coordination between leadership, supervisors, teachers, and branches through shared academic workflows.

1 Ready for Future Intelligence

Build the structured data foundation needed for advanced AI-powered features as the platform continues to grow.

1 Core Platform Capabilities

Section Label Core Platform Capabilities Main Heading Everything Your Academic Operation Needs - Connected in One Platform. **Section Intro** TIS is designed to bring the most important academic operations into one structured SaaS environment. From teacher planning and branch management to observations, calendars, dashboards, and future AI-powered intelligence, the platform helps schools move from scattered work to connected academic control. **Capability Cards**

1 Academic Workforce Planning

Plan teacher assignments, analyze weekly loads, identify uncovered teaching hours, and understand staffing needs with greater clarity.

1 Academic Calendars & Shared Coordination

Coordinate academic events, timelines, school priorities, and branch-level planning through shared calendars that keep teams aligned.

1 Observation & Supervision Workflows

Manage structured teacher observations, supervisor feedback, shared records, and signing workflows inside one organized process.

1 Multi-Branch Organization Management

Support school groups, campuses, branches, users, roles, and tenant-isolated structures from one scalable SaaS platform.

1 Dashboards & Decision Visibility

Give leaders a clearer view of coverage, staffing gaps, teacher utilization, subject sufficiency, and academic follow-up needs.

1 Future AI-Powered Intelligence

Prepare for advanced subscription-based AI capabilities, including curriculum-driven assessment generation, answer keys, planning support, analytics, and intelligent recommendations based on verified academic data.

1 Teacher Workforce Planning Engine

Section Label Teacher Workforce Planning Engine Main Heading Know Exactly Who Is Teaching What - and Where Support Is Needed. Section Text One of the strongest current capabilities of TIS is its ability to help schools understand their teaching workforce with clarity. Instead of manually calculating teacher loads, subject coverage, and staffing needs across separate files, TIS brings this information into one structured planning engine. School leaders can track teacher qualifications, majors, subject specialties, weekly teaching loads, assigned subjects, uncovered hours, and additional staffing requirements. This helps academic teams identify gaps earlier, use existing teachers more effectively, and make staffing decisions based on clear data rather than assumptions. For schools and multi-branch organizations, this creates a more reliable way to plan academic coverage, compare staffing needs, and understand whether each subject, grade, and branch has the right teaching support. **Key Capabilities**

1 Teacher Load Visibility

See teacher workloads clearly and understand whether teaching hours are balanced, underused, or overloaded.

1 Subject Coverage Analysis

Identify which subjects are fully covered and which still have uncovered teaching hours.

1 Staffing Gap Detection

Detect shortages early before they affect schedules, instruction, or academic quality.

1 Qualification-Based Planning

Use teacher degrees, majors, and subject specialties to support smarter assignment decisions.

1 New Teacher Requirement Insights

Understand how many additional teachers may be needed based on uncovered hours and school workload requirements.

1 Multi-Branch Workforce Clarity

Support better staffing visibility across branches, campuses, and larger educational organizations.

1 Observation & Supervision Workflows

Section Label Observation & Supervision Workflows Main Heading Turn Teacher Observation into a Clear, Shared, and Accountable Process. Section Text TIS is designed to help schools move teacher observation away from scattered forms, disconnected files, and informal follow-up. Observation records, supervisor feedback, teacher responses, signatures, and approval steps can become part of one structured academic workflow. This gives supervisors a clearer way to document classroom visits, teachers a more transparent way to receive and respond to feedback, and school leaders a stronger view of supervision quality across departments and branches. As the platform continues to grow, advanced AI-assisted supervision features are planned to support supervisors by suggesting improvement plans based on observation notes, evaluation results, teacher performance evidence, and school-approved criteria. Key Capabilities

1 Structured Observation Records

Create organized observation forms and records that follow a clear school-approved process.

1 Shared Teacher-Supervisor Visibility

Allow teachers and supervisors to access relevant observation records, feedback, and follow-up actions.

1 Signing and Approval Workflow

Support formal documentation through review, acknowledgement, signing, and approval steps.

1 Supervision Follow-Up

Track feedback, recommendations, and improvement actions after classroom observations.

1 AI-Suggested Improvement Plans

Future AI-assisted tools may help supervisors generate suggested teacher improvement plans based on observation notes, evaluation data, and approved academic criteria.

1 Leadership Oversight

Give academic leaders better visibility over observation activity, supervision quality, and follow-up completion.

1 Reduced Observation Paperwork

Replace scattered observation files with a more consistent, traceable, and connected supervision system.

1 Academic Calendars & Shared Coordination

Section Label Academic Calendars & Shared Coordination Main Heading Turn the Academic Calendar into a Shared Planning Engine. Section Text TIS is designed to help schools move beyond static calendars and disconnected event planning. Academic leaders, supervisors, teachers, and branches can work from a shared academic calendar connected to the selected academic year, helping the whole school coordinate priorities, events, activities, observations, deadlines, and academic follow-up. Future calendar capabilities can support default worldwide and international academic events, allowing schools to plan meaningful activities around important global occasions. This gives international, national, private, and public schools a stronger way to connect school events with classroom learning, curriculum objectives, student engagement, and whole-school awareness. As TIS continues to grow,

AI-assisted calendar planning can help schools generate suggested activities for selected events, including classroom activities, outdoor activities, entertainment activities, curriculum-integrated tasks, and lesson-based ideas. The calendar vision also includes support for Sustainable Development Goals awareness planning, helping schools design year-round activities connected to SDGs, global citizenship, and curriculum integration. Key Capabilities

1 Academic-Year-Based Calendar

Build and manage school calendars according to the selected academic year, such as 2026-2027.

1 Shared School Coordination

Connect leaders, supervisors, teachers, and branches through one shared academic planning calendar.

1 International Event Awareness

Support worldwide and international academic events that schools can use for activities, campaigns, and curriculum connections.

1 AI-Suggested Activity Planning

Future AI-assisted tools may suggest activity plans based on selected events, school needs, grade levels, and curriculum links.

1 Curriculum-Integrated Events

Help schools connect calendar events with classroom learning, lesson planning, and academic objectives.

1 SDG Awareness Planning

Support year-round planning for Sustainable Development Goals awareness, student engagement, and global citizenship activities.

1 Branch-Level Visibility

Give school groups and multi-branch organizations a clearer view of academic events, priorities, and planning across campuses.

1 Multi-Branch Organization Management

Section Label Multi-Branch Organization Management Main Heading Built for Schools That Operate Across Branches, Campuses, and Teams. Section Text TIS is designed to support schools and educational organizations that need structure beyond a single location. Whether an organization manages one school, several branches, multiple campuses, or different academic pathways, TIS provides a scalable environment where users, roles, branches, and academic operations can be organized with greater clarity. School leadership and organization administrators can manage branches, users, permissions, branding, academic structures, and operational workflows inside a tenant-isolated SaaS environment. This helps every branch work within a shared organizational framework while still maintaining its own operational identity and academic needs. For educational groups, TIS can support stronger management-level visibility across branches. Leaders can monitor branch progress, compare completion levels, identify delayed tasks, and recognize where follow-up is needed across areas such as curriculum planning, exams, observations, calendars, reporting, and academic operations. Key Capabilities

1 Organization and Branch Structure

Manage organizations, schools, branches, campuses, and academic pathways inside one scalable platform.

1 Role-Based User Management

Assign users according to their responsibilities, such as organization administrator, branch leader, academic leader, supervisor, teacher, or future custom roles.

1 Tenant-Isolated Environment

Keep each organization's data, users, branding, and academic operations securely separated.

1 Branch Progress Intelligence

Give management-level users a clearer view of each branch's completion, progress, delays, and operational follow-up needs.

1 Branch Comparison Indicators

Compare branch performance across connected academic areas such as curriculum tasks, exams, observations, calendars, planning, and reports.

1 Custom Branding by Organization

Support organization and branch branding, including logos, visual identity, and school-specific presentation.

1 Scalable SaaS Structure

Prepare schools and educational groups for growth, future modules, subscription plans, and advanced AI-powered capabilities.

1 Dashboards & Decision Visibility

Section Label Dashboards & Decision Visibility Main Heading See Academic Progress, Risks, and Priorities Before They Become Problems. Section Text TIS dashboards are designed to give school leaders more than static reports. They provide a connected view of academic operations, helping leadership understand what has been completed, what is delayed, what needs follow-up, and where intervention may be required. As the platform continues to grow, dashboards can reflect progress across annual plans, weekly plans, curriculum coverage, assessment blueprints, quizzes, exams, academic calendar deadlines, teacher observations, supervision follow-up, staffing gaps, and branch-level performance. For branch management, dashboards can show the academic health of one school or campus. For organization-level leadership, dashboards can compare branches, highlight delayed tasks, identify performance gaps, and support faster, evidence-based decisions. Key Capabilities

1 Planning Completion Indicators

Track progress for annual plans, weekly plans, lesson planning, curriculum coverage, and completed academic tasks.

1 Assessment Progress Visibility

Monitor assessment blueprints, quizzes, exams, term-based assessment requirements, and calendar-linked due dates.

1 Observation & Supervision Insights

See observation progress, teacher follow-up status, supervisor actions, and areas requiring improvement plans.

1 Academic Calendar Integration

Connect events, deadlines, assessments, activities, and required tasks to dashboard indicators.

1 Teacher & Academic Performance Indicators

Support leadership with visual indicators related to teacher follow-up, academic progress, and school-level performance trends.

1 Branch and Organization Dashboards

Provide branch-level dashboards for school management and organization-level dashboards for comparing progress across multiple campuses.

1 Risk and Delay Alerts

Highlight delayed tasks, incomplete requirements, missing academic evidence, and branches or departments that need attention. Supporting Line: From reports to real-time academic visibility.

1 Future AI-Powered Intelligence

Section Label Future AI-Powered Intelligence Main Heading AI That Works Inside the Academic Workflow - Not Outside It. Section Text TIS is being developed with a future AI vision that goes beyond simple content generation. Instead of using AI as a separate tool, future AI-powered capabilities are planned to work inside the platform's academic workflows, supporting schools through the data already stored, reviewed, and approved in TIS. This future intelligence layer can support academic calendars, observation follow-up, curriculum planning, teacher support, assessment generation, dashboards, branch progress indicators, and leadership decision-making. At the center of this vision is a curriculum-to-classroom workflow. Uploaded curriculum data, standards, learning outcomes, annual plans, weekly plans, lesson plans, and confirmed instructional coverage can become the foundation for guided planning, AI-assisted recommendations, assessment generation, answer keys, and academic analytics. Future AI capabilities will be introduced progressively through subscription-based plans, allowing schools to access more advanced intelligence features as their needs grow. Key Capabilities

1 Curriculum-to-Classroom AI Support

Assist teachers and academic leaders in moving from uploaded curriculum to annual plans, weekly plans, lesson plans, and verified coverage.

1 AI-Assisted Assessment Generation

Support future generation of exams, quizzes, assessment blueprints, and answer keys based on approved curriculum data and confirmed instructional coverage.

1 AI-Supported Observation Follow-Up

Help supervisors generate suggested teacher improvement plans based on observation notes, evaluation results, and school-approved criteria.

1 AI-Powered Calendar Planning

Suggest activities for international events, SDG awareness, curriculum links, classroom activities, and school-wide academic events.

1 Academic Analytics and Recommendations

Provide future intelligent insights related to planning progress, teacher follow-up, assessment readiness, curriculum coverage, and branch performance.

1 Subscription-Based AI Growth

Introduce advanced AI features progressively through higher subscription plans, giving schools flexibility to grow into more powerful capabilities.

1 From Curriculum to Classroom - With Guided AI Support

Section Label From Curriculum to Classroom Main Heading Support Teachers Without Adding Another Burden. Section Text TIS is being developed to support teachers, not overload them. Instead of asking teachers to build annual plans, weekly plans, lesson plans, and curriculum tracking documents from scratch, future AI-assisted planning tools will help guide them through structured steps using approved templates, uploaded curriculum data, learning outcomes, standards, objectives, and school-specific requirements. Teachers will be able to review, adjust, complete, and submit their plans through the system. Academic leaders can then review and approve the work, turning planning documents into reliable, official academic data inside TIS. This helps reduce teacher workload, improve planning consistency, and give school leadership clearer visibility over what has been planned, approved, taught, and covered.

1 Subscription Plans Preview

Section Label Subscription Plans Main Heading Choose the Level of Academic Intelligence Your School Needs. Section Text TIS is being developed as a subscription-based SaaS platform, allowing schools and educational organizations to start with essential academic operations and grow into more advanced capabilities over time. As the platform expands, subscription plans can support different levels of access, from core teacher and academic operations to advanced dashboards, multi-branch management, supervision workflows, and future AI-powered intelligence. Plan Cards

1 Core

For schools that need a structured way to organize teacher data, academic operations, basic planning visibility, and essential workflows.

1 Professional

For schools that need stronger dashboards, shared academic calendars, observation workflows, staffing visibility, and improved coordination across teams.

1 Enterprise AI

For school groups and larger organizations that need multi-branch visibility, advanced customization, branch progress intelligence, AI-assisted planning, assessment generation, analytics, and strategic decision support. Supporting Note AI-powered capabilities will be introduced progressively and may vary according to subscription level, school needs, and platform development stage. CTA Request Early Access Request Pricing

1 Final Call-to-Action

Section Label Start Your TIS Journey Main Heading Ready to Bring Clarity, Control, and Intelligence to Your Academic Operations? Section Text TIS is being built for schools and educational organizations that want to move beyond scattered files, disconnected workflows, and delayed decisions. Start exploring how one connected academic operations platform can support your leadership team, supervisors, teachers, branches, and future AI-powered growth. CTA Buttons Request Early Access Book a Demo Supporting Line TIS is currently in active development, with selected features and advanced AI capabilities planned for progressive release through subscription-based plans.

docs/location-data-roadmap.md

TIS Location Data Roadmap

Phase 1: Global Form Usability

Status: implemented in the current working tree.

Phase 1 intentionally keeps location data attached directly to an organization or branch. It does not create shared locality records.

Included:

- 1 Controlled country selection from the local dataset.
- 1 Controlled region/state/province selection when dataset coverage exists.
- 1 Dataset-backed city/locality selection.
- 1 Other / manual entry fallback for missing regions and cities.
- 1 Optional district and neighborhood fields, stored separately from city.
- 1 Local cached APIs; no external location API dependency at runtime.
- 1 Existing text location fields remain compatible with legacy Saudi data.

Manual region and city values are record-level values. They are not published to other tenants and are not added to the imported dataset.

Deferred Architecture

The following phases are roadmap items only. They are not part of Phase 1 and must not be inferred from the current text-based storage model.

Locality Registry

Create stable TIS locality identifiers and canonical locality records that are independent from vendor dataset identifiers. Organizations and branches would eventually reference these records while retaining display-name snapshots for audit and migration compatibility.

Locality Aliases

Support native names, translated names, transliterations, spelling variants, and historical names. Alias matching should be region-aware and must not merge places solely because their normalized names match.

GeoNames and Official Dataset Enrichment

Build an offline enrichment process using GeoNames country dumps and, where available, official national gazetteers. Source priority, licensing, attribution, place classification, and duplicate resolution must be defined before combining records.

Tenant-Private City Additions

Allow an authorized tenant user to suggest a missing locality. New records should initially be visible only to that tenant and must be constrained to the tenant's organization scope.

Platform Moderation

Provide a Platform Owner moderation queue for reviewing proposed localities, potential duplicates, coordinates, place type, aliases, and source evidence.

City Verification and Promotion

Support explicit states such as tenant-private, pending, verified, deprecated, and merged. Promotion to the global catalog must be an audited platform action, never an automatic consequence of a tenant entering free text.

Dataset Refresh and ETL Workflow

Introduce versioned, offline imports with source checksums, staging tables, coverage metrics, diff review, attribution records, and rollback support. Referenced localities should be deprecated or merged rather than deleted.

Guardrails for Future Work

- 1 Preserve tenant isolation for all tenant-created locality data.
- 1 Keep stable internal IDs separate from external source IDs.
- 1 Retain source and license provenance for every imported record.
- 1 Do not call public geocoding services from normal page requests.
- 1 Do not modify the vendor JSON to store tenant-entered values.
- 1 Migrate existing organization and branch text values without data loss.